

Framework/AutoExec/STFPOSAuto.prw

672 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "STPOS.CH"
3 #INCLUDE "STIRECESEL.CH"
4
5 //Tratamento para identificação da posição do Array aNCCs - Fonte: STINCCSelection
6 #DEFINE NCCSELECTED 1 // Campo logico que indica se a NCC foi selecionada
7 #DEFINE SEINUM 3 // Posicao do campo E1_NUM do Array aNCCsCli
8 #DEFINE SEIRECNO 5 // Posicao do Recno do registro dentro do Array aNCCsCli
9 #DEFINE SEISALDO 6 // Posicao do campo E1_SALDO do Array aNCCsCli
10 #DEFINE SEIPREFIXO 9 // Posicao do campo E1_PREFIXO do Array aNCCsCli
11 #DEFINE SEIPARCELA 10 // Posicao do campo E1_PARCELA do Array aNCCsCli
12
13 Static lOpenPDV := .T. //Controla se faz abertura do PDV perifericos, menus etc
14
15 //-----
16 /*{Protheus.doc} STFPosAuto
17 Executa operações do TOTVS PDV de forma automática sem exibição de Tela.
18 Pode ser usado para testes automatizados etc.
19 @param aParFunc - Parametros utilizados a depender da rotina a ser executada
20 @param cTipoOper - Tipo de operação a ser executada conforme tabela
21 VD = Venda
22 CA = Cancelamento Venda
23 AC = Abertura de caixa
24 FC = Fechamento de caixa
25 RT = Recebimento de Título
26 ET = Estorno de Título
27 IO = Importação Orcamento
28 SA = Sangria de caixa
29 SU = Suprimento de caixa
30 CC = Cadastro cliente
31 @type function
32 @param
33 @return lRet - Retorna se executou corretamente
34 */
35 //-----
36 Function STFPosAuto( aParFunc , cTipoOper )
37
38 Local lRet := .T.
39 Local aMenus := {}
40 Local cErro := ""
41 Local nX := 1
42 Default aParFunc := Array(5)
43 Default cTipoOper := "VD"
44
45 //Sempre completa o array com 5 posições para suprir os parâmetros não passados
46 While Len(aParFunc) < 5
47     aAdd(aParFunc,{})
48 Enddo
49
50 If lOpenPDV
51     STFSetPOS(.T.)//Seta(Força) que é TOTVS PDV
52

```

```

53 //Inicialização padrão TOTVS PDV
54 lRet := STFStart()
55
56 // Carregar menus somente apos o STFStart
57 If lRet
58     aMenus := STBLstOperCashier()
59     lOpenPDV := .F. //nao faz abertura do PDV perifericos, menus etc
60 EndIf
61 EndIf
62
63 If lRet
64     Do case
65         case cTipoOper == "VD"
66             If STWP0S0OpenCash()
67                 STFSaleAuto( aParFunc[1] , aParFunc[2] , aParFunc[3], aParFunc[4] )
68             Endif
69         case cTipoOper == "CA" //Cancelar venda
70             If STWP0S0OpenCash()
71                 lRet := STFSaleCan( aParFunc[1] )
72             Endif
73         case cTipoOper == "AC"
74             STWP0SCloseCash()
75             lRet := STWP0S0OpenCash()
76         case cTipoOper == "FC"
77             lRet := STWP0SCloseCash()
78             //Fecha comunicacao com perifericos
79             STWCcloseDevice()
80             lOpenPDV := .T. //Marca para fazer abertura da proxima vez
81             //Libera o objeto de totais da venda
82             If ExistFunc("STFTotRelease")
83                 STFTotRelease()
84             EndIf
85         case cTipoOper == "CC"
86             If(STWP0S0OpenCash(),lRet := STFIccAuto(aParFunc,@cErro),(cErro := "",lRet := .F.))
87         case cTipoOper == "RT"
88             If STWP0S0OpenCash()
89                 STFRecAuto( aParFunc[1] , aParFunc[2] , aParFunc[3], aParFunc[4], aParFunc[5] )
90             Endif
91         case cTipoOper == "ET"
92             If STWP0S0OpenCash()
93                 STFEsRAuto( aParFunc[1] , aParFunc[2] )
94             Endif
95         otherwise
96             lRet := .F.
97             cErro := "Operação: " + cTipoOper + " Inválida!."
98     EndCase
99
100 EndIf
101
102 EndIf
103
104 If !lRet

```

```

105     Conout(cErro)
106     lMsErroAuto := .T. //Sinaliza Erro para ExecAuto
107     Help( " ", 1, "Help",, cErro, 1, 0 )
108 EndIf
109
110 Return lRet
111
112
113 //-----
114 /*{Protheus.doc} STFSaleAuto
115 Executa operação de venda no TOTVS PDV de forma automática sem exibição de Tela.
116 Pode ser usado para testes automatizados etc.
117 @param aCab - Cabeçalho da venda
118 @param aItens - Itens da venda
119 @param aPgts - Pagamentos da venda
120
121 @type function
122 @param
123 @return lRet - Retorna se executou corretamente
124 */
125 //-----
126 Static Function STFSaleAuto( aCab , aItens , aPgts, aNCCsCli )
127
128     Local lRet           := .T.
129     Local cCliCode       := ""
130     Local cCliStore      := ""
131     Local cCliType       := ""
132     Local cFilRes        := ""
133     Local cOrcRes        := ""
134     Local cPedRes        := ""
135     Local cCondPg        := ""
136     Local cVendedor      := ""
137     Local lIsRetPost     := .F.
138     Local nPosPRODUTO    := 0
139     Local nPosVALDESC    := 0
140     Local nPosTES        := 0
141     Local nPosPRCTAB     := 0
142     Local nPosQUANT      := 0
143     Local nPosCODBAR     := 0
144     Local nPosITEMGAR    := 0
145     Local nNextItem      := 0
146     Local lItemFiscal    := .T.
147     Local cCliente       := ""
148     Local cLojaCli       := ""
149     Local cNomeCli       := ""
150     Local cEndCli        := ""
151     Local cCgcCli        := ""
152     Local lFinServ       := AliasIndic("MG8") .AND. SuperGetMV("MV_LJCSF",,.F.) // Valida implementação do serviço financeiro
153     Local aItemFind      := {}
154     Local cCodProdReg     := ""
155     Local cCodItemGar     := ""
156     Local lRegistrou     := .F.

```

```

157 Local nScanJuros      := 0
158 Local nScanVLJur      := 0
159 Local aAcres          := {}
160 Local nI              := 0
161 Local nJ              := 0
162 Local nX              := 0
163 Local nSaldoNCCs      := 0
164
165 Local oCliModel        := Nil
166 Local oModelCesta      := Nil // Model Cesta
167 Local oModelPay        := Nil // Model Formas de Pagamento
168 Local oMdlGrd          := Nil // Model Parcelas
169 Local oMdlPaym         := Nil // Model APAYMENTS
170 Local oModel           := Nil
171 Local aCopyPaym        := STIGetSx5()[2]
172 Local cFormaPG         := ""
173 Local nValorPG         := 0
174
175 Local nPosFrete        := AScan( aCab , { |x| x[1] == "L1_FRETE" } )
176 Local nVlrFrete       := 0
177
178 Default aCab           := {}
179 Default aItens         := {}
180 Default aPgts         := {}
181 Default aNCCsCli       := {}
182
183 //Adiciona cabeçalho da venda
184 If lRet
185
186     cCliCode      := aCab[AScan( aCab , { |x| x[1] == "L1_CLIENTE" } )][2]
187     cCliStore     := aCab[AScan( aCab , { |x| x[1] == "L1_LOJA" } )][2]
188     cCliType      := aCab[AScan( aCab , { |x| x[1] == "L1_TIPOCLI" } )][2]
189     cCondPg       := aCab[AScan( aCab , { |x| x[1] == "L1_CONDPG" } )][2]
190     cVendedor     := aCab[AScan( aCab , { |x| x[1] == "L1_VEND" } )][2]
191     nVlrFrete     := IIF(nPosFrete > 0, aCab[nPosFrete][2], 0 )
192
193     oCliModel := STWCustomerSelection(cCliCode+cCliStore)
194
195     If Len(aNCCsCli) > 0 // tem NCC para ser baixado
196
197         STDSetNCCs("1",aNCCsCli) //Alimenta o array aNCCs
198
199         For nX := 1 To Len(aNCCsCli)
200             If aNCCsCli[nX,NCCSELECTED]
201                 //STDSetNCCs("2",aNCCsCli[nX,SE1SALDO])
202                 nSaldoNCCs += aNCCsCli[nX,SE1SALDO]
203             EndIf
204         Next nX
205
206         // alimentando as variaveis utilizadas nas rotinas do STiPosAuto
207         // aNCCs := STDGetNCCs("1", aNCCsCli)
208         STDGetNCCs("2", nSaldoNCCs)

```

```

209
210     EndIf
211
212     STDSPBasket( "SL1" , "L1_CLIENTE" , cCliCode )
213     STDSPBasket( "SL1" , "L1_LOJA" , cCliStore )
214     STDSPBasket( "SL1" , "L1_TIPOCLI" , cCliType )
215     STDSPBasket( "SL1" , "L1_CONDPG" , cCondPg )
216     STDSPBasket( "SL1" , "L1_FRETE" , nVlrFrete )
217
218     oVendModel := STWSalesmanSelection(cVendedor)
219     STDSPBasket("SL1","L1_VEND" ,oVendModel:GetValue("SA3MASTER","A3_COD"))
220     STDSPBasket("SL1","L1_COMIS",STDGComission( oVendModel:GetValue("SA3MASTER","A3_COD") ))
221
222     cCliente := oCliModel:GetValue("SA1MASTER", "A1_COD" )
223     cLojaCli := oCliModel:GetValue("SA1MASTER", "A1_LOJA")
224     cNomeCli := oCliModel:GetValue("SA1MASTER", "A1_NOME")
225     cEndCli := oCliModel:GetValue("SA1MASTER", "A1_END" )
226     cCgcCli := oCliModel:GetValue("SA1MASTER", "A1_CGC" )
227     STICGCConfirm(@cCgcCli, cNomeCli, cEndCli, .T. , .F. )
228
229 EndIf
230
231 // Adicionar os Itens na Venda
232 If lRet
233
234     nPosPRODUTO := AScan( aItens[1] , { |x| x[1] == "L2_PRODUTO" } )
235     nPosVALDESC := AScan( aItens[1] , { |x| x[1] == "L2_VALDESC" } )
236     nPosQUANT := AScan( aItens[1] , { |x| x[1] == "L2_QUANT" } )
237     nPosTES := AScan( aItens[1] , { |x| x[1] == "L2_TES" } )
238     nPosPRCTAB := AScan( aItens[1] , { |x| x[1] == "L2_PRCTAB" } )
239
240     For nI := 1 To Len(aItens)
241
242         // Seta quantidade
243         STBSetQuant( aItens[nI][nPosQUANT][2] )
244
245         nNextItem := STDPBLength("SL2") + 1
246
247         // Busca item na base de dados
248         aItemFind := STWFindItem( aItens[nI][nPosPRODUTO][2] )
249
250         // Caso nao encontre o item na base de dados
251         If !aItemFind[ITEM_ENCONTRADO]
252             lRet:= .F.
253         EndIf
254
255         If nPosCODBAR > 0 .And. !Empty(aItens[nI][nPosCODBAR][2])
256             cCodProdReg := aItens[nI][nPosCODBAR][2]
257         Else
258             cCodProdReg := aItens[nI][nPosPRODUTO][2]
259         EndIf
260

```

```

261      //STWSetIsOpenReceipt(.T.)
262
263      lRet := STWItemReg(      nNextItem      , ;      // Item
264                             cCodProdReg      , ;      // Codigo Prod
265                             cCliCode         , ;      // Codigo Cli
266                             cCliStore        , ;      // Loja Cli
267                             1                , ;      // Moeda
268                             aItens[nI][nPosVALDESC][2] , ;      // Valor desconto
269                             "V"              , ;      // Tipo desconto ( Percentual ou Valor )
270                             NIL              , ;      // Item adicional?
271                             aItens[nI][nPosTES][2] , ;      // TES
272                             cCliType         , ;      // Tipo do cliente (A1_TIPO)
273                             lItemFiscal      , ;      // Registra item no cupom fiscal?
274                             aItens[nI][nPosPRCTAB][2] , ;      // Preço
275                             "IMP"            , ;      // Indica que é importação de orçamento
276                             .T.              , ;      // Se deve imprimir o CNPJ, informacao sera passada na abertura do cupom
277                             ,,,,,,          , ;      // lRecovery,nSecItem,lServFinal,lProdBonif,lListProd,cCodList,cCodListIt,cCodMens,cEntrega
278                             cCodItemGar      )      // Código do item de produto garantia vinculado
279
280      If lRet
281          //Atualiza Cesta
282          iif ( ExistFunc("STBISL2Refresh"), STBISL2Refresh( aItens[nI] , nNextItem ), nil)
283          STDSaveSale(nNextItem)
284      Else
285          Exit //Caso ocorra algum erro ao registrar o item não é necessário executar os passos seguintes
286      EndIf
287
288      Next nI
289
290  EndIf
291
292
293  //Atualiza campos do cabeçalho apos inclusão dos itens
294  If lRet
295
296      //Chamada Desconto no Total
297      If AScan( aCab , { |x| x[1] == "L1_DESCONT" } ) > 0
298          STBTotDiscApply( aCab[AScan( aCab , { |x| x[1] == "L1_DESCONT" } )][2] , "V" , , .T.)
299      EndIf
300
301      // Chamada componente de frete
302      STWAddFreight(aItens)
303
304      //Chamada Acrescimo Financeiro
305      nScanJuros := AScan( aCab , { |x| x[1] == "L1_JUROS" } )
306
307      If SL1->(ColumnPos("L1_VLRJUR")) > 0
308
309          nScanVLJur := AScan( aCab , { |x| x[1] == "L1_VLRJUR" } )
310          If nScanVLJur > 0
311              STFSetTot( "L1_VLRJUR" , aCab[nScanVLJur][2])
312              STDSPBasket("SL1","L1_VLRJUR",STDGPBasket('SL1','L1_VLRJUR') + aCab[nScanVLJur][2])

```

```

313         Else
314             STFSerTot( "L1_VLRJUR",0)
315             STDSPBasket("SL1","L1_VLRJUR",0)
316         EndIf
317
318         AADD ( aAcres , STDGPBasket("SL1","L1_VLRJUR"))
319         If nScanJuros > 0
320             AADD ( aAcres , aCab[nScanJuros][2])
321         EndIf
322
323     ElseIf nScanJuros > 0
324         //Calcula o valor do acrescimo (Legado)
325         //A função abaixo irá retornar o valor correto apenas para juros simples ou multi-negociação
326         //Caso seja Condição negociada (juros composto ou Price) o valor total da venda não irá bater com o Total da SL4
327         aAcres := STBDiscConvert( aCab[nScanJuros][2] , 'P' )
328     EndIf
329
330     STWAddIncrease( aAcres[1] , IIF(nScanJuros > 0,aCab[nScanJuros][2],0) )//Add acrescimo
331
332     STFRefTot()//Atualiza Totalizador
333
334 EndIf
335
336 //Atualiza SL4
337 If lRet
338
339     ModelPayme()
340     oModel := STISetMdlPay()
341     oMdlGrd := oModel:GetModel("PARCELAS")
342     oMdlGrd:Activate()
343     oMdlPaym := oModel:GetModel("APAYMENTS")
344     oMdlPaym:Activate()
345
346     For nI := 1 To Len(aPgts)
347
348         cFormaPG := aPgts[nI][AScan( aPgts[nI] , { |x| x[1] == "L4_FORMA" } )][2]
349         nValorPG := aPgts[nI][AScan( aPgts[nI] , { |x| x[1] == "L4_VALOR" } )][2]
350
351         lAddPay := STIAddPay(AllTrim(cFormaPG), Nil, 1, Nil, Nil, nValorPG)
352         lRet := lRet .AND. lAddPay
353
354     Next nI
355
356     If nSaldoNCCs > 0
357         // alimentando as variaveis utilizadas nas rotinas do STiPosAuto
358         STDSetNCCs("1", aNCCsCli ) //Alimenta o array aNCCs
359         STDGetNCCs("2", nSaldoNCCs)
360
361         STIAddPay("CR", Nil, 1, Nil, Nil, nSaldoNCCs)
362     EndIf
363
364 //Finaliza venda

```

```

365         lRet := STBConfPay(oMdlGrd, aCopyPaym,oMdlPaym)
366
367     EndIf
368
369     If !lRet
370         lMsErroAuto := .T. //Sinaliza Erro para ExecAuto
371         cErro := "STFPOSAUTO ERROR: " + Chr(10)+ Chr(13)
372         cErro += STFLatestMsg()
373         ConOut("cErro")
374         Help( " ", 1, "Help",, cErro, 1, 0 )
375     EndIf
376
377     FreeObj(oCliModel)
378     FreeObj(oModelCesta)
379     FreeObj(oModelPay)
380     FreeObj(oMdlGrd)
381     FreeObj(oMdlPaym)
382     FreeObj(oModel)
383
384 Return lRet
385 //-----
386 /*{Protheus.doc} STFSaleCan
387 Executa operação de venda no TOTVS PDV de forma automática sem exibição de Tela.
388 Pode ser usado para testes automatizados etc.
389 @param aCab - Cabeçalho da venda
390 @param aItens - Itens da venda
391 @param aPgts - Pagamentos da venda
392
393 @type function
394 @param
395 @return lRet - Retorna se executou corretamente
396 */
397 //-----
398 Static Function STFSaleCan( aCancSale )
399
400 Private cGetCanc := aCancSale[1]
401 private oStruGrd
402 Private oPanel
403
404 //carrega os dados da venda
405 aValidCancel := STBCSCanCancel(cGetCanc)
406 lRet := aValidCancel[1]
407
408 If lRet
409     /*
410     Seta Informações cancelamento
411     */
412     STISetCancel( aValidCancel )
413     /*
414     Ação botão de cancelar venda
415     */
416     IIF(ExistFunc("STBActionCancel"),STBActionCancel(cGetCanc), .F. )

```



```

417     EndIf
418 Return( lRet )
419
420 //-----
421 /*{Protheus.doc} STFiccAuto
422 Executa operação de inclusão de um novo usuário no TOTVS PDV de forma automática sem exibição de Tela.
423 Pode ser usado para testes automatizados etc.
424 @param  aDadosCli   - Dados do cliente
425
426 @type function
427 @param
428 @return   lRet - Retorna se executou corretamente
429 */
430 //-----
431 Static Function STFiccAuto(aDadosCli,cErro)
432
433 Local lRet := .T.
434 Local nX := 1
435 Local nPos := 0
436 Local lAutoGenCod := .T.
437 Local cCodCliente := ""
438 Local lOpenRegItem := .F.
439 Local oLojCliente := Nil
440 Local cLojCliente := ""
441 Local aCliente := {"A1_LOJA" , ""},; //1
442                  {"A1_NOME" , ""},; //2
443                  {"A1_NREDUZ" , ""},; //3
444                  {"A1_PESSOA" , ""},; //4
445                  {"A1_TIPO" , ""},; //5
446                  {"A1_CGC" , ""},; //6
447                  {"A1_END" , ""},; //7
448                  {"A1_EST" , ""},; //8
449                  {"A1_MUN" , ""},; //9
450                  {"A1_TEL" , ""},; //10
451                  {"A1_EMAIL" , ""},; //11
452                  {"A1_DDD" , ""} //12
453
454 Default aDadosCli := {}
455 DeFault cErro := ""
456
457 For nX := 1 to len(aCliente)
458     nPos = aScan(aDadosCli,{|x| x[1]==aCliente[nX][1]})
459     If nPos > 0
460         aCliente[nX][2] := aDadosCli[nPos][2]
461     Endif
462 Next nX
463
464 If(STICGCvld(aCliente[6][2],aCliente[4][2],@cCodCliente,oLojCliente,.T.,aCliente[1][2])[3],,(cErro := "CPF: "+aCliente[6][2]+", invalido!",lRet := .F.))
465 If(STDExistChav("SA1",cCodCliente+aCliente[1][2],,""),,(cErro := "Cliente já cadastrado!",lRet := .F.))
466 If lRet
467     lRet := STIConfCustomer(cCodCliente ,aCliente[1][2] ,aCliente[2][2] ,aCliente[3][2] ,;
468                            aCliente[4][2] ,aCliente[6][2] ,aCliente[5][2] ,aCliente[7][2] ,;

```

```

469         aCliente[8][2]      ,aCliente[9][2]      ,lAutoGenCod,lOpenRegItem,;
470         ,,{ "A1_FILIAL","A1_COD","A1_LQJA","A1_NOME","A1_NREDUZ","A1_END","A1_TIPO","A1_EST","A1_MUN","A1_PESSOA","A1_CGC","A1_DDD","A1_TEL","A1_EMAIL"},;
471         {{xFilial("SA1"),cCodCliente,aCliente[1][2],aCliente[2][2],aCliente[3][2],aCliente[7][2],aCliente[5][2],aCliente[8][2],aCliente[9][2],;
472         aCliente[4][2],aCliente[6][2],aCliente[12][2],aCliente[10][2],aCliente[11][2]}})
473 EndIf
474
475 FreeObj(oLojCliente)//Libera objeto
476
477 Return lRet
478
479 //-----
480 /*/{Protheus.doc} STFRecAuto
481
482
483 @param  aDados    - Dados
484
485 @type function
486 @return  lRet - Retorna se executou corretamente
487 */
488 //-----
489 Static Function STFRecAuto(aCli,aTit,aPgts,aNCC,aCheque)
490
491 Local lRet := .T.
492 Local oCliModel
493 Local oMdl      := Nil                      //Recupera o model ativo
494 Local oMdlGrd   := Nil                      //Seta o model do grid
495 Local oMdlPaym  := Nil                      //Seta o model do pagamento
496 Local aCopyPaym := STIGet5x5()[2]
497 Local aListTitles := {}
498 Local oWFReceipt
499 Local nX := 1
500 Local nY := 1
501 Local nPos := 0
502 Local cFormaPG := ""
503 Local nValorPG := 0
504 Local cAdminis := ""
505 Local nParc := 1
506 Local oMdlC := Nil
507 Local cGetNSU := ""
508 Local cGetAutoriz := ""
509 Local lContTef := .F.
510 Local dData := ctod("")
511
512 Default aCli := {}
513 Default aTit := {}
514 Default aPgts := {}
515 Default aNCC := {}
516 Default aCheque := {}
517
518 oMdl      := ModelPayme()
519 oMdlGrd   := oMdl:GetModel("PARCELAS")
520 oMdlGrd:Activate()

```

```

521 oMdlPaym      := oMdl:GetModel("APAYMENTS")
522 oMdlPaym:Activate()
523
524 STISetRecTit(.T.)
525
526 oWFRceipt := STWReceiptTitle():STWReceiptTitle(aCli[1],aCli[2],aCli[3],"")
527 If (lRet := oWFRceipt:LoadTitles())
528
529     oCliModel := STWCustomerSelection(aCli[1]+aCli[2],"")
530
531     STWReceTit()
532
533     For nX := 1 To Len(aPgts)
534         For nY := 1 to Len(aPgts[nX])
535             Do Case
536                 Case aPgts[nX][nY][1] == "L4_FORMA"
537                     cFormaPG := aPgts[nX][nY][2]
538                 Case aPgts[nX][nY][1] == "L4_VALOR"
539                     nValorPG := aPgts[nX][nY][2]
540                 Case aPgts[nX][nY][1] == "L4_ADMINIS"
541                     cAdminis := aPgts[nX][nY][2]
542                 Case aPgts[nX][nY][1] == "L4_NSUTEF"
543                     cGetNSU := aPgts[nX][nY][2]
544                 Case aPgts[nX][nY][1] == "L4_AUTORIZ"
545                     cGetAutoriz := aPgts[nX][nY][2]
546                 Case aPgts[nX][nY][1] == "L4_DATA"
547                     dData := aPgts[nX][nY][2]
548                 Case aPgts[nX][nY][1] == "L4_PARC"
549                     nParc := aPgts[nX][nY][2]
550             Otherwise
551             End Case
552         Next nY
553
554     If cFormaPG $("CC|CD")
555         oMdlC := ModelCard()//oModel:GetModel("CARDMASTER")
556         oMdlC := oMdlC:GetModel("CARDMASTER")
557
558         oMdlC:DeActivate()
559         oMdlC:Activate()
560
561         oMdlC:LoadValue("L4_FILIAL", xFilial("SL4"))
562         oMdlC:LoadValue("L4_DATA", dData)
563         oMdlC:LoadValue("L4_VALOR", nValorPG)
564         oMdlC:LoadValue("L4_PARC", nParc)
565         oMdlC:LoadValue("L4_ADMINIS", cAdminis)
566
567         If !Empty(cGetNSU)
568             oMdlC:LoadValue("L4_NSUTEF", cGetNSU)
569         EndIf
570         If !Empty(cGetAutoriz)
571             oMdlC:LoadValue("L4_AUTORIZ", cGetAutoriz)
572         EndIf

```

```

573
574         oTEF20 := STBGetTEF()
575
576         STISetCard(.T.)
577         STISetTef(oTEF20)
578     Elseif cFormaPG == "CH"
579         STWSetCkRet(aCheque)
580     Endif
581
582     lRet := STIAddPay(AllTrim(cFormaPG), oMdlC, nParc, cFormaPG $("CC|CD") , Nil, nValorPG)
583 Next nX
584
585 //Marca os títulos a serem baixados
586 For nX := 1 to len(aTit)
587     If (nPos := aScan(oWfReceipt:aListTitles[4],{|x| x[16]==aTit[nX][16]}) > 0
588         oWfReceipt:aListTitles[4][nPos][1] := .T.//Selecionado
589         oWfReceipt:aListTitles[4][nPos][7] := aTit[nX][7]//Multa
590         oWfReceipt:aListTitles[4][nPos][8] := aTit[nX][8]//Juros
591         oWfReceipt:aListTitles[4][nPos][9] := aTit[nX][9]//Desconto
592         oWfReceipt:aListTitles[4][nPos][10] := aTit[nX][6]+aTit[nX][7]+aTit[nX][8]-aTit[nX][9]
593     Endif
594 Next nX
595
596 If lRet .AND. oWfReceipt:DropTitles(oMdlGrd,STIGetIsCont())
597     STISetCard(.F.)
598     oWfReceipt:Print(oMdlGrd)
599     lRet := .T.
600     STFSetTot( 'L1_VLRTOT' , 0 )
601     STISetRecTit(.F.) // Depois de finalizar, seta que nao eh mais recebimento
602 Else
603     lRet := .F.
604 EndIf
605
606 If lRet
607     STISetRecTit(.F.)
608     STFRestart()
609 EndIf
610
611 Endif
612
613 FreeObj(oWfReceipt)
614
615 Return lRet
616
617 //-----
618 /*/{Protheus.doc} STFESRAuto
619
620
621 @param  aDados    - Dados
622
623 @type function
624 @return      lRet - Retorna se executou corretamente

```

```

625 /*/
626 //-----
627 Static Function STFESRAuto(aCli,aTit)
628
629 Local aNCCs := STDGetNCCs("1") //NCCs do cliente
630 Local aNCCcli := {}
631 Local cTpOp := STIGetTpOp() //Tipo de operacao
632 Local nParcTotal := 0
633 Local nX := 1
634 Local oWFReceipt
635 Local oObjRec
636 Local lRet := .T.
637
638 Default aTit := {}
639 Default aCli := Array(3)
640
641 oWFReceipt := STWReceiptTitle():STWReceiptTitle(aCli[1],aCli[2],aCli[3],"")
642 oObjRec := STIRetObjTit()
643
644 oWFReceipt:SetReverseMode(.T.)
645
646 If (lRet := oWFReceipt:LoadTitles())
647     //Marca os títulos a serem baixados
648     For nX := 1 to len(aTit)
649         If (nPos := aScan(oWFReceipt:aListTitles[4],{|x| x[16]==aTit[nX][16]})) > 0
650             oWFReceipt:aListTitles[4][nPos][1] := .T.//Selecionado
651             STISetTitSelecionado(nPos,.T.)
652             nParcTotal := nParcTotal + oWFReceipt:GetParcTotal(nX)
653             oWFReceipt:SetParcSelecte44368d(nX, .T.)
654             AADD(aNCCcli,{oWFReceipt:aListTitles[4][nX][13],oWFReceipt:aListTitles[4][nX][14]}) ///Codigo de Cliente e Loja do Titulo
655         Endif
656     Next nX
657
658 STFSetTot( 'L1_VLRTOT', nParcTotal )
659
660 If Len(oWFReceipt:aListTitles[4]) > 0 .AND. !Empty(nParcTotal) .AND. lRet
661     If oWFReceipt:ReverseDropTitles()
662         oWFReceipt:Print()
663         STFRestart()
664     Else
665         //Necessario zerar o total para poder efetuar outras operações no caixa.
666         STFSetTot( 'L1_VLRTOT', 0 )
667         lRet := .F.
668     EndIf
669 Endif
670 Endif
671
672 Return lRet

```

Framework/Cache/STFWCLimitedMemCache.prw

45 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2
3 Function STFWCLimMemCache() ; Return
4
5 Class STFWCLimitedMemCache From STFWCMemCache
6     Data nCacheLimit
7
8     Method STFWCLimitedMemCache()
9     Method Put(cKey, uData, nMinutesToExpire)
10    Method LimitReached()
11    Method ClearExpiredCache()
12 EndClass
13
14 Method STFWCLimitedMemCache(nCacheLimit) Class STFWCLimitedMemCache
15     Default nCacheLimit := 0
16     Self:nCacheLimit := nCacheLimit
17     _Super:STFWCMemCache()
18 Return
19
20 Method Put(cKey, uData, nMinutesToExpire) Class STFWCLimitedMemCache
21     Local lRet := .F.
22     If Self:LimitReached()
23         If Self:ClearExpiredCache()
24             lRet := _Super:Put(cKey, uData, nMinutesToExpire)
25         EndIf
26     Else
27         lRet := _Super:Put(cKey, uData, nMinutesToExpire)
28     EndIf
29 Return lRet
30
31 Method LimitReached() Class STFWCLimitedMemCache
32 Return Self:nCacheLimit != 0 .And. Self:nCacheLimit <= Len(Self:aMemCache)
33
34 Method ClearExpiredCache() Class STFWCLimitedMemCache
35     Local nCount := 0
36     Local lRet := .F.
37
38     For nCount := 1 To Len(Self:aMemCache)
39         If !Self:IsValidCache(nCount)
40             Self:RemoveByPos(nCount)
41             lRet := .T.
42         Exit
43     EndIf
44 Next
45 Return lRet

```

Framework/Cache/STFWCMemCache.prw

94 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "TOTVS.CH"
2
3 Function STFWCMemCache() ; Return
4
5 Class STFWCMemCache
6     Data aMemCache
7
8     Method STFWCMemCache()
9     Method Put(cKey, uData, nMinutesToExpire)
10    Method Contains(cKey)
11    Method Get(cKey)
12    Method Find(cKey)
13    Method IsValidCache(nPos)
14    Method Invalidate()
15    Method Remove(cKey)
16    Method RemoveByPos(nPos)
17 EndClass
18
19 Method STFWCMemCache() Class STFWCMemCache
20     Self:aMemCache := {}
21 Return
22
23 Method Put(cKey, uData, nMinutesToExpire) Class STFWCMemCache
24     Local oNow := Nil
25     Local nPos := 0
26     Default nMinutesToExpire := -1
27
28     If nMinutesToExpire > 0
29         oNow := TMKDateTime():Now()
30         oNow:PlusMinutes(nMinutesToExpire)
31     EndIf
32
33     nPos := AScan(Self:aMemCache, {|x| x[1] == cKey })
34     If nPos > 0
35         Self:aMemCache[nPos][2] := uData
36         Self:aMemCache[nPos][3] := oNow
37     Else
38         AAdd(Self:aMemCache, {cKey, uData, oNow})
39     EndIf
40 Return .T.
41
42 Method Contains(cKey) Class STFWCMemCache
43 Return IF(Self:Find(cKey) > 0, .T., .F.)
44
45 Method Get(cKey) Class STFWCMemCache
46     Local nPos := 0
47     Local uRet := Nil
48
49     nPos := Self:Find(cKey)
50     // Localizou a chave
51     If nPos > 0
52         // Verifica se o cache é válido

```

```

53     If Self:IsValidCache(nPos)
54         uRet := Self:aMemCache[nPos][2]
55     Else
56         Self:RemoveByPos(nPos)
57     EndIf
58 EndIf
59 Return uRet
60
61 Method Find(cKey) Class STFWCMemCache
62     Local nPos := 0
63     nPos := AScan(Self:aMemCache, {|x| x[1] == cKey })
64 Return nPos
65
66 Method IsValidCache(nPos) Class STFWCMemCache
67     Local oNow      := Nil
68     Local lRet      := .F.
69
70     If Self:aMemCache[nPos][3] != Nil
71         // Verifica se já expirou o cache
72         oNow := TMKDateTime():Now()
73         lRet := oNow:LessThan(Self:aMemCache[nPos][3])
74     Else
75         lRet := .T.
76     EndIf
77 Return lRet
78
79 Method Invalidate() Class STFWCMemCache
80     Self:aMemCache := {}
81 Return
82
83 Method Remove(cKey) Class STFWCMemCache
84     Local nPos := 0
85     nPos := AScan(Self:aMemCache, {|x| x[1] == cKey })
86     If nPos > 0
87         Self:RemoveByPos(nPos)
88     EndIf
89 Return
90
91 Method RemoveByPos(nPos) Class STFWCMemCache
92     ADel(Self:aMemCache, nPos)
93     ASize(Self:aMemCache, Len(Self:aMemCache)-1)
94 Return

```


Framework/Cache/STFWCTestLimitedMemCache.prw

60 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2
3 Class STFWCTestLimitedMemCache From FWDefaultTestCase
4     Method STFWCTestLimitedMemCache()
5     Method TestPut()
6     Method TestPutWithExpired()
7     Method TestUnlimited()
8 EndClass
9
10 Method STFWCTestLimitedMemCache() Class STFWCTestLimitedMemCache
11     _Super:FWDefaultTestCase()
12     Self:AddTestMethod("TestPut")
13     Self:AddTestMethod("TestPutWithExpired")
14     Self:AddTestMethod("TestUnlimited")
15 Return
16
17 Method TestPut() Class STFWCTestLimitedMemCache
18     Local oResult      := FWTestResult():FWTestResult()
19     Local oMemCache := STFWCLimitedMemCache():STFWCLimitedMemCache(10)
20
21     oResult:AssertTrue(oMemCache:Put("KEY1", 1), "Não incluiu o KEY1, mas deveria.")
22     oResult:AssertTrue(oMemCache:Put("KEY2", 1), "Não incluiu o KEY2, mas deveria.")
23     oResult:AssertTrue(oMemCache:Put("KEY3", 1), "Não incluiu o KEY3, mas deveria.")
24     oResult:AssertTrue(oMemCache:Put("KEY4", 1), "Não incluiu o KEY4, mas deveria.")
25     oResult:AssertTrue(oMemCache:Put("KEY5", 1), "Não incluiu o KEY5, mas deveria.")
26     oResult:AssertTrue(oMemCache:Put("KEY6", 1), "Não incluiu o KEY6, mas deveria.")
27     oResult:AssertTrue(oMemCache:Put("KEY7", 1), "Não incluiu o KEY7, mas deveria.")
28     oResult:AssertTrue(oMemCache:Put("KEY8", 1), "Não incluiu o KEY8, mas deveria.")
29     oResult:AssertTrue(oMemCache:Put("KEY9", 1), "Não incluiu o KEY9, mas deveria.")
30     oResult:AssertTrue(oMemCache:Put("KEY10", 1), "Não incluiu o KEY10, mas deveria.")
31     oResult:AssertFalse(oMemCache:Put("KEY11", 1), "Incluiu o KEY11, mas não deveria.")
32 Return oResult
33
34 Method TestPutWithExpired() Class STFWCTestLimitedMemCache
35     Local oResult      := FWTestResult():FWTestResult()
36     Local oMemCache := STFWCLimitedMemCache():STFWCLimitedMemCache(10)
37
38     oResult:AssertTrue(oMemCache:Put("KEY1", 1, 0.02), "Não incluiu o KEY1, mas deveria.")
39     oResult:AssertTrue(oMemCache:Put("KEY2", 1, 0.02), "Não incluiu o KEY2, mas deveria.")
40     oResult:AssertTrue(oMemCache:Put("KEY3", 1), "Não incluiu o KEY3, mas deveria.")
41     oResult:AssertTrue(oMemCache:Put("KEY4", 1), "Não incluiu o KEY4, mas deveria.")
42     oResult:AssertTrue(oMemCache:Put("KEY5", 1), "Não incluiu o KEY5, mas deveria.")
43     oResult:AssertTrue(oMemCache:Put("KEY6", 1), "Não incluiu o KEY6, mas deveria.")
44     oResult:AssertTrue(oMemCache:Put("KEY7", 1), "Não incluiu o KEY7, mas deveria.")
45     oResult:AssertTrue(oMemCache:Put("KEY8", 1), "Não incluiu o KEY8, mas deveria.")
46     oResult:AssertTrue(oMemCache:Put("KEY9", 1), "Não incluiu o KEY9, mas deveria.")
47     oResult:AssertTrue(oMemCache:Put("KEY10", 1), "Não incluiu o KEY10, mas deveria.")
48     Sleep(1000)
49     oResult:AssertTrue(oMemCache:Put("KEY11", 1), "Não incluiu o KEY11, mas deveria.")
50     oResult:AssertTrue(oMemCache:Put("KEY12", 1), "Não incluiu o KEY12, mas deveria.")
51     oResult:AssertFalse(oMemCache:Put("KEY13", 1), "Incluiu o KEY13, mas não deveria.")
52 Return oResult

```

```
53
54 Method TestUnlimited() Class STFWCTestLimitedMemCache
55     Local oResult      := FWTestResult():FWTestResult()
56     Local oMemCache := STFWCLimitedMemCache():STFWCLimitedMemCache()
57
58     oResult:AssertTrue(oMemCache:Put("KEY1", 1), "Não incluiu o KEY1, mas deveria.")
59     oResult:AssertTrue(oMemCache:Put("KEY2", 1), "Não incluiu o KEY2, mas deveria.")
60 Return oResult
```

Framework/Cache/STFWCTestMemCache.prw

87 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2
3 Class STFWCTestMemCache From FWDefaultTestCase
4     Method STFWCTestMemCache()
5
6     Method TestPut()
7     Method TestContains()
8     Method TestNotContains()
9     Method TestGetValid()
10    Method TestGetInvalid()
11    Method TestRemove()
12    Method TestInvalidate()
13 EndClass
14
15 Method STFWCTestMemCache() Class STFWCTestMemCache
16     _Super:FWDefaultTestCase()
17     Self:AddTestMethod("TestPut")
18     Self:AddTestMethod("TestContains")
19     Self:AddTestMethod("TestNotContains")
20     Self:AddTestMethod("TestGetValid")
21     Self:AddTestMethod("TestGetInvalid")
22     Self:AddTestMethod("TestRemove")
23     Self:AddTestMethod("TestInvalidate")
24 Return
25
26 Method TestPut() Class STFWCTestMemCache
27     Local oResult      := FWTestResult():FWTestResult()
28     Local oMemCache := STFWCMemCache():STFWCMemCache()
29
30     oResult:AssertTrue(oMemCache:Put("KEY1", 1))
31 Return oResult
32
33 Method TestContains() Class STFWCTestMemCache
34     Local oResult      := FWTestResult():FWTestResult()
35     Local oMemCache := STFWCMemCache():STFWCMemCache()
36
37     oMemCache:Put("KEY1", 1)
38
39     oResult:AssertTrue(oMemCache:Contains("KEY1"))
40 Return oResult
41
42 Method TestNotContains() Class STFWCTestMemCache
43     Local oResult      := FWTestResult():FWTestResult()
44     Local oMemCache := STFWCMemCache():STFWCMemCache()
45
46     oMemCache:Put("KEY1", 1)
47
48     oResult:AssertFalse(oMemCache:Contains("KEY2"))
49 Return oResult
50
51 Method TestGetValid() Class STFWCTestMemCache
52     Local oResult      := FWTestResult():FWTestResult()

```

```
53     Local oMemCache := STFWCMemCache():STFWCMemCache()
54
55     oMemCache:Put("KEY1", 1, 0.02)
56
57     oResult:AssertTrue(oMemCache:Get("KEY1") == 1)
58 Return oResult
59
60 Method TestGetInvalid() Class STFWCTestMemCache
61     Local oResult      := FWTestResult():FWTestResult()
62     Local oMemCache := STFWCMemCache():STFWCMemCache()
63
64     oMemCache:Put("KEY1", 1, 0.02)
65     Sleep(1000)
66     oResult:AssertTrue(oMemCache:Get("KEY1") == Nil)
67 Return oResult
68
69 Method TestRemove() Class STFWCTestMemCache
70     Local oResult      := FWTestResult():FWTestResult()
71     Local oMemCache := STFWCMemCache():STFWCMemCache()
72
73     oMemCache:Put("KEY1", 1, 0.10)
74     oMemCache:Remove("KEY1")
75     oResult:AssertFalse(oMemCache:Contains("KEY1"))
76 Return oResult
77
78 Method TestInvalidate() Class STFWCTestMemCache
79     Local oResult      := FWTestResult():FWTestResult()
80     Local oMemCache := STFWCMemCache():STFWCMemCache()
81
82     oMemCache:Put("KEY1", 1)
83     oMemCache:Put("KEY2", 1, 1)
84     oMemCache:Invalidate()
85     oResult:AssertFalse(oMemCache:Contains("KEY1"))
86     oResult:AssertTrue(oMemCache:Get("KEY2") == Nil)
87 Return oResult
```

Framework/CadFiles/STFCadFile.prw

36 linhas após remoção de metadados de autoria/data

```
1 #Include 'Protheus.ch'
2
3 //-----
4 /*/{Protheus.doc} STFCadFile
5
6
7 @param
8 @return
9 @obs      Nil
10 @sample   Nil
11 */
12 //-----
13 Function STFCadFile()
14 Return
15
16 //-----
17 /*/{Protheus.doc} STFRetDlls
18 Retorna os arquivos utilizados pelo sistema
19
20 @param
21 @return   aRet - Retorna um array com os arquivos que foram cadastrados
22 @obs      Nil
23 @sample   Nil
24 */
25 //-----
26 Function STFGetFiles()
27
28 Local aRet := {}
29
30 Aadd(aRet, {'Totvs','SigaLoja.dll','0.3.135.13'})
31 Aadd(aRet, {'Totvs','TotvsApi.dll','1.0.14.2'})
32 Aadd(aRet, {'Totvs','AutoCom.dll','3.4.1.3'})
33 Aadd(aRet, {'Bematech','BemaFI32.dll',''})
34 Aadd(aRet, {'Bematech','BemaFI32.ini',''})
35
36 Return aRet
```

Framework/Carga/STFLoadData.prw

1069 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "Protheus.ch"
2 #INCLUDE "TbiConn.ch"
3 #INCLUDE "STFLoadData.ch"
4
5 /*
6     Estrutura da Tabela MH1
7
8     X3_ARQUIVO  X3_ORDEM  X3_CAMPO  X3_TIPO  X3_TAMANHO  X3_TITULO  X3_DESCRIC  X3_PICTURE
9     MH1        01       MH1_FILIAL  C        2          Filial  Sucursal  Filial do Sistema  @!
10    MH1        02       MH1_COD  C        6          Cod. Carga  Cod. Carga  @!
11    MH1        03       MH1_TIME  N        5          Tempo  Tempo  99999
12    MH1        04       MH1_STATUS C        1          Status  Status  @!
13    MH1        05       MH1_HORAI  C        5          Hora Inicial  Hora Inicial  99:99
14    MH1        06       MH1_HORAF  C        5          Hora Final  Hora Final  99:99
15 */
16
17
18 //User function para testes dos jobs em debug.
19 /*
20 User Function TestLoadRet()
21 STFLoadRet( "T1", "D MG 01")
22 Return .T.
23 */
24
25 User Function TestLoadpdv()
26 STFLoadPdv("EMP_PDV", "FIL_PDV", "AMB_RET", "EMP_RET", "FIL_RET")
27 Return .T.
28
29
30 //-----
31 /*{Protheus.doc} STFLoadRet
32 Gera as cargas incrementais na retaguarda, automaticamente via Scheduler
33
34 [STFLoadRet]
35 Main=STFLoadRet
36 Environment=<Ambiente>
37 nParms=6
38 Parm1=<cEmp>
39 Parm2=<cFil>
40 Parm3=<cInterval>
41 Parm4=<cIpType>
42 Parm5=<cLoadDel>
43 Parm6=<cLoAutDel>
44 [OnStart]
45 Jobs=STFLoadRet
46 RefreshRate=30
47
48 @param
49 @return .T.
50 */
51 //-----
52

```

```

53 Function STFLoadRet( cEmp      , cFil      , cInterval, cIpType      ,;
54                     cLoadDel    , cLoAutDel )
55
56 Local cTimeLoad := "" //Controla se o horario da carga eh permitido para entrar no JOB
57 Local cIpExt    := "" //Endereco IP externo
58 Local cRetAmb   := "" //Ambiente retaguarda
59 Local cFileName := "" //Nome do arquivo de Semaforo
60 Local nSleep    := 0  //Tempo de intervalo
61 Local nLoads    := 0  //Contador de cargas
62 Local nSizeLoad := 0  //Retorna o numero de cargas geradas na tabela MBU
63 Local nI        := 0  //Contador
64 Local aLoads    := {} //Array de cargas
65 Local aFiliais  := {} //Filiais para processamento
66 Local aTables   := {"MBU", "MD3", "MD4", "MH1"} //Tabelas de trabalho
67 Local nMvLjThrea := 0
68
69 Default cEmp      := "" //Empresa de trabalho
70 Default cFil      := "" //Filial de trabalho
71 Default cInterval := "3600000" //Tempo de intervalo. Padrao para gerar carga 1 hora = 3600000 Milisegundos
72 Default cIpType   := "2" //Tipo do endereco IP
73 Default cLoadDel   := "0" //Limite para iniciar o processo de exclusao das cargas processadas em todos os pdvs
74 Default cLoAutDel  := "0" //Define quantas cargas ira excluir sem analisar os PDVs essa exclusão garante que a carga nao trava ao atingir limite do param MV_LJILQTD
75
76 /*
77   Ex Job no appserver.ini no ambiente da Retaguarda
78
79   [STFLoadRet]
80   Main=STFLoadRet
81   Environment=<Ambiente> - Ambiente Local Retaguarda
82   nParms=6
83   Parm1=<cRetEmp> - Empresa da Retaguarda
84   Parm2=<cRetFils> - Filiais da Retaguarda podendo ser separados por ponto e virgula
85   Parm3=<3600000> - Tempo para repetição da execução do Job LOJA1156 Job em milissegundos
86   Parm4=<cIpType> - 1=Dinâmico (Ip Atualizado automaticamente); 2=Estático (0 Ip permanece com a configuração Inicial)
87   Parm5=<cLoadDel> - Limite para exclusao apos baixas dos PDVs
88   Parm6=<cDelAut> - Define quantas cargas ira excluir sem analisar os PDVs essa exclusão garante que a carga nao trava ao atingir limite do param MV_LJILQTD
89
90   [ONSTART]
91   JOBS=STFLoadRet
92 */
93
94 aFiliais := StrTokArr(AllTrim(cFil), ",")
95
96 If !Empty(cEmp) .AND. Len(aFiliais) > 0
97   RPCSetType(3)
98   RpcSetEnv(cEmp, aFiliais[1],, "FRT",, aTables)
99
100   nMvLjThrea := SuperGetMv("MV_LJTHREA", .F., 0)
101 EndIf
102
103 If nMvLjThrea == 1
104   STFThreads(cLoadDel, cLoAutDel)

```

```

105 Else
106   LjGrvLog( "Carga","STFLoadRet Início ")
107   Conout("STFLoadRet Início ")
108   //Armazena tempo de intervalo
109   nSleep := Val(cInterval)
110
111   If nSleep < 3600000
112     Conout( "Atencao, Intervalo para geracao da carga(" + AllTrim(cInterval) + ") menor que o recomendado(3600000).")
113     LjGrvLog( "Carga","Atenção, Intervalo para geração da carga(" + AllTrim(cInterval) + ") menor que o recomendado(3600000).")
114   EndIf
115
116   If !Empty(cEmp)
117
118     For nI := 1 To Len(aFiliais)
119       //Se o arquivo de semáforo existir, pode ser que o server tenha sido interrompido no meio e neste caso exclui o arquivo.
120       If File("LJAUTOLOAD" + cEmp + aFiliais[nI] + ".txt")
121         //Apaga arquivo de controle Semaforo
122         FErase("LJAUTOLOAD" + cEmp + aFiliais[nI] + ".txt")
123       EndIf
124     Next nI
125
126     For nI := 1 To Len(aFiliais)
127       //Nao consome licenca
128       RPCSetType(3)
129       RpcSetEnv(cEmp, aFiliais[nI],,"FRT",,aTables)
130       //Nome do arquivo de Semaforo
131       cFileName := cEmp + aFiliais[nI]
132
133       //Semaforo do controle de execucao da filial
134       If !File("LJAUTOLOAD" + cFileName + ".txt")
135         Conout("Job LOJA1156job Iniciado para a filial " + aFiliais[nI])
136         LjGrvLog( "Carga","Job LOJA1156job Iniciado para a filial " + aFiliais[nI])
137
138
139         //Cria o arquivo de Semaforo
140         MSFCreat("LJAUTOLOAD" + cFileName + ".txt")
141
142         //Define a quantidade de cargas geradas
143         nSizeLoad := LJ1156CountLoads()
144
145
146         //Se o numero de cargas for superior ao desejado, aplica o processo para apagar os dados da MBU com base nos PDVs filhos
147         If Val(cLoadDel) > 0 .And. nSizeLoad > Val(cLoadDel)
148           If !STFLoadDel()
149             Conout("JOB LOJA1156job: STFLoadDel(): Aguardando a liberacao do(s) PDV(s) para excluir o proximo lote na retaguarda. Sera feita uma nova tentativa
para a filial " + aFiliais[nI])
150             LjGrvLog( "Carga","JOB LOJA1156job: STFLoadDel(): Aguardando a liberacao do(s) PDV(s) para excluir o proximo lote na retaguarda. Sera feita uma nova
tentativa para a filial " + aFiliais[nI])
151             EndIf
152           EndIf
153
154           //Apaga as cargas independente de os PDVs terem baixado por ter atingido o limite do parametro MV_LJILQTD

```



```

155      If Val(cLoAutDel) > 0 .AND. nSizeLoad >= SuperGetMV("MV_LJILQTD", .F., 200)
156          STFLoAutDel( Val(cLoAutDel ), nSizeLoad )
157      EndIf
158
159      //Armazena cargas
160      aLoads := STFLoadItem()
161
162      //Efetua cargas
163      For nLoads := 1 To Len(aLoads)
164          //Aguardando o tempo de espera, de acordo com o temp->FUTURO, armazenado em aLoads[nLoads][2]
165          While (aLoads[nLoads][3] == Date() .And. Round(Seconds(), 0) < (aLoads[nLoads][2]))
166              Inkey(1)
167          EndDo
168
169          //Se o tempo e a data atual for igual ou maior que o solicitado na Carga executa o JOB
170          If aLoads[nLoads][3] == Date() .And. Round(Seconds(), 0) >= (aLoads[nLoads][2])
171              cTimeLoad := Time()
172
173              //Se o horario atual permitir, sera gerada a carga
174              If cTimeLoad >= aLoads[nLoads][4] .And. cTimeLoad <= aLoads[nLoads][5]
175
176                  //Verifica e atualiza IP dinamico
177                  If cIpType == "1"
178                      MD3->(dbSetOrder(1))
179                      cIpExt := LjIpPrxy()
180                      cRetAmb := PAdr(SuperGetMv("MV_LJAMBIE", .F., ""), TamSx3("MD3_CODAMB")[1])
181
182                      //Atualiza endereco IP na MD3
183                      If !Empty(cIpExt) .And. !Empty(cRetAmb) .And. MD3->(dbSeek(xFilial("MD3") + cRetAmb + "R"))
184                          MD3->(RecLock("MD3", .F.))
185                          MD3->MD3_IP := cIpExt
186                          MD3->(MsUnlock())
187
188                          //Atualiza parametro do Ip Assistente de Carga
189                          PutMv("MV_LJILLIP", cIpExt)
190                      EndIf
191                  EndIf
192
193                  //Executa a carga na Retaguarda
194                  LOJA1156Job(aLoads[nLoads][1])
195
196                  Conout("Job LOJA1156Job executado para a carga " + aLoads[nLoads][1])
197                  LjGrvLog( "Carga","Job LOJA1156Job executado para a carga " + aLoads[nLoads][1])
198              Else
199                  Conout("Job LOJA1156Job nao foi executado pois o periodo esta fora do configurado para a carga " + aLoads[nLoads][1])
200                  LjGrvLog( "Carga","Job LOJA1156Job nao foi executado pois o periodo esta fora do configurado para a carga " + aLoads[nLoads][1])
201              EndIf
202          EndDo
203      EndIf
204      Next nLoads
205
206      //Apaga arquivo de controle Semaforo

```

```

207         FErase("LJAUTOLOAD" + cFileName + ".txt")
208
209         Conout("Job LOJA1156job Executado para a filial " + aFiliais[nI])
210         LjGrvLog( "Carga","Job LOJA1156job Executado para a filial " + aFiliais[nI])
211
212         EndIf
213         RPCClearEnv()
214     Next nI
215 EndIf
216
217 Sleep(nSleep)
218 LjGrvLog( "Carga","STFLoadRet Fim ")
219
220 EndIf
221
222
223 Return .T.
224
225 //-----
226 /*/{Protheus.doc} STFLoadPdv()
227 Roda no Onstart dos PDVs e importa as cargas incrementais
228
229 [STFLoadPdv]
230 Main=STFLoadPdv
231 Environment=<Ambiente>
232 nParms=8
233 Parm1=<cPdvEmp>
234 Parm2=<cPdvFil>
235 Parm3=<cRetAmb>
236 Parm4=<cRetEmp>
237 Parm5=<cRetFil>
238 Parm6=<cInterval>
239 Parm7=<cIpType>
240 Parm8=<cGrpTab>
241
242 [OnStart]
243 Jobs=STFLoadPdv
244 RefreshRate=30
245
246 @param
247 @return .T.
248 */
249 //-----
250
251 Function STFLoadPdv(cPdvEmp, cPdvFil, cRetAmb, cRetEmp, cRetFil, cInterval, cIpType, cGrpTab)
252
253 Local cIP           := ""           //Ip
254 Local cPorta        := ""           //Porta
255 Local cIpExt         := ""           //Endereco IP externo
256 Local cFilAmb        := ""           //Ambiente filial
257 Local nSleep         := 0            //Tempo de intervalo
258 Local lImport        := .T.          //Se efetuara a importacao da carga de dados

```

```

259 Local lDownload      := .T.      //Se efetuara o download da carga de dados
260 Local lActInChildren := .F.      //Se as acoes serao replicadas nos dependentes
261 Local lKillOtherThreads := .T.    //Se derrubara os outros processos que estao em execucao caso nao consiga abrir as tabelas exclusivamente
262 Local lOnlyIfNewer    := .T.      //Somente atualiza se a carga disponivel no servidor for mais atual que a carga anteriormente baixada
263 Local nMvLjThrea      := 0        //Habilita a carga com as multiplas threads
264 Local lContinua      := .T.      //Verifica se continua o processamento da carga
265
266 Default cPdvEmp       := ""        //Empresa do Pdv
267 Default cPdvFil       := ""        //Filial do PDV
268 Default cRetAmb       := ""        //Ambiente da Retaguarda
269 Default cRetEmp       := ""        //Empresa da Retaguarda
270 Default cRetFil       := ""        //Filial da Retaguarda
271 Default cInterval     := "600000" //Tempo de intervalo. Padrao para buscar carga 10 minutos = 600000 Milisegundos
272 Default cIpType       := "2"       //Tipo do endereco IP
273 Default cGrpTab       := ""        //Grupo de tabela
274
275 LjGrvLog( "Carga","STFLoadPdv Início ")
276 Conout("STFLoadPdv Inicio ")
277 /*
278     Ex Job no appserver.ini no ambiente do PDV
279
280     [STFLoadPdv]
281     Main=STFLoadPdv
282     Environment=<Ambiente> - Ambiente Local PDV
283     nParms=8
284     Parm1=<cPdvEmp> - Empresa do Pdv
285     Parm2=<cPdvFil> - Filial do Pdv
286     Parm3=<cRetAmb> - Ambiente da Retaguarda
287     Parm4=<cRetEmp> - Empresa da Retaguarda
288     Parm5=<cRetFil> - Filial da Retaguarda
289     Parm6=<600000> - Tempo para repetição da execução do Job LOJA1157Job em milissegundos
290     Parm7=<nIpType> - 1=Dinâmico (Ip Atualizado automaticamente); 2=Estático (0 Ip permanece com a configuração Inicial)
291     Parm8=<cGrpTab> - Informa o código do grupo de tabelas que a carga devera executar, esse oitavo parâmetro só tem efeito se o parâmetro MV_LJTHREA for igual a 1
292
293     [ONSTART]
294     JOBS=STFLoadPdv
295 */
296
297 //Armazena tempo de intervalo
298 nSleep := Val(cInterval)
299
300 If nSleep < 600000
301     Conout( "Atencao, Intervalo para Importacao da carga(" + AllTrim(cInterval) + ") menor que o recomendado(600000).")
302     LjGrvLog( "Carga", "Atenção, Intervalo para Importação da carga(" + AllTrim(cInterval) + ") menor que o recomendado(600000).")
303 EndIf
304
305 If !Empty(cPdvEmp) .AND. !Empty(cPdvFil)
306     RPCSetType(3)
307     RpcSetEnv(cPdvEmp,cPdvFil,Nil,Nil,"FRT")
308
309     nMvLjThrea := SuperGetMv("MV_LJTHREA", .F., 0)
310

```

```

311 If nMvLjThrea == 1
312     If Empty(cGrpTab)
313         Conout("Atenção, a funcionalidade de multiplas threads da carga esta ativada (MV_LJTHREA = 1) mas o oitavo parâmetro no JOB STFLoadPdv não foi informado. A carga
nã sera executada!")
314         LjGrvLog("Carga","Atenção, a funcionalidade de multiplas threads da carga esta ativada (MV_LJTHREA) mas o oitavo parâmetro no JOB STFLoadPdv não foi informado. A
carga não sera executada!")
315         lContinua := .F.
316     EndIf
317 Else
318     cGrpTab := ""
319 EndIf
320
321 //Valida se existe alguma venda em andamento
322 If lContinua .AND. !STBGetAct()
323     Conout("Job LOJA1157Job Iniciado")
324     LjGrvLog( "Carga","Job LOJA1157Job Iniciado")
325
326
327 //Verifica e atualiza IP dinamico
328 If cIpType == "1"
329     MD3->(dbSetOrder(1))
330     cIpExt := LjIpPrxy()
331     cFilAmb := Padr(SuperGetMv("MV_LJAMBIE", .F., ""), TamSx3("MD3_CODAMB")[1])
332
333     //Atualiza endereco IP na MD3
334     If !Empty(cIpExt) .And. !Empty(cFilAmb) .And. MD3->(dbSeek(xFilial("MD3") + cFilAmb + "R"))
335         MD3->(RecLock("MD3", .F.))
336         MD3->MD3_IP := cIpExt
337         MD3->(MsUnlock())
338
339         //Atualiza parametro do Ip Assistente de Carga
340         PutMv("MV_LJILLIP", cIpExt)
341     EndIf
342 EndIf
343
344 cIP := SuperGetMV("MV_LJILLIP", .F., "") //Ip
345 cPorta := SuperGetMV("MV_LJILLP0", .F., "") //Porta
346
347 //Executa a carga no PDV
348 LOJA1157Job(cIP, Val(cPorta), cRetAmb , cRetEmp, cRetFil, lImport, lDownload,;
349     lActInChildren, lKillOtherThreads, lOnlyIfNewer,, cGrpTab)
350
351 Conout("Job LOJA1157Job Executado")
352 LjGrvLog( "Carga","Job LOJA1157Job Executado")
353 EndIf
354
355 RPCClearEnv()
356 EndIf
357
358 Sleep(nSleep)
359 LjGrvLog( "Carga","STFLoadPdv Fim ")
360

```

```

361 Return .T.
362
363 //-----
364 /*{Protheus.doc} STFLoadDel
365 Exclui as cargas na Retaguarda de acordo com cada PDV ou lista de cargas recebidas
366 @param      aMbyGeral , array , Cargas a apagar
367 @return lRet - Retorna se excluiu as cargas
368 */
369 //-----
370
371 Static Function STFLoadDel(aMbyGeral)
372
373 Local aAreaMD4 := MD4->(GetArea()) //Guarda area
374 Local aAreaMD3 := MD3->(GetArea()) //Guarda area
375 Local cPath      := GetPvProfString("LJFileServer", "Path", "\LJFileServer\", GetAdv97()) //Caminho da carga
376 Local nCount     := 0              //Contador
377 Local nCountPdv  := 0              //Variavel que controla o numero de PDVs lidos
378 Local nLoads     := 0              //Numero de carga
379 Local nPosPdv    := 0              //Pesquisa carga no novo PDV pesquisado
380 Local nI         := 0              //Contador
381 Local aMBYPdv    := {}             //Retorno da consulta no Pdv com base na Tabela MBY
382 Local aAuxMby    := {}             //Auxiliar do controle de cargas dos Pdv
383 Local aPdv      := {}             //Armazena todos os PDVs cadastrados na Retaguarda
384 Local aDirectory := {}             //Array de diretorios
385 Local lRet       := .T.            //Retorno
386 Local oServer    := Nil            //Objeto de conexao com o server
387 Local bOldError := {}             //Variavel de backup de erro de execucao para proteger a comunicacao RPC caso algum PDV caia durante a consulta.
388
389 Default aMbyGeral := {}            //Array com cargas a deletar
390
391 LjGrvLog( "Carga","STFLoadDel Início ")
392
393 If Len(aMbyGeral) == 0
394
395     MD4->(dbSetOrder(1))
396     //Posiciona no primeiro registro com Filial + Codigo
397     If MD4->(dbSeek(xFilial("MD4")))
398         While MD4->(!Eof()) .AND. ( MD4->MD4_FILIAL == xFilial("MD4") )
399             //Processa somente ambiente PDV
400             If !Empty(MD4->MD4_AMBPAI)
401                 Aadd(aPdv, MD4->MD4_CODIGO)
402             EndIf
403
404             MD4->(DbSkip())
405         EndDo
406     EndIf
407
408     //Busca os dados de cada PDV para realizar a pesquisa das cargas processadas
409     MD3->(dbSetOrder(1))
410
411     If Len(aPdv) > 0
412         For nCount := 1 To Len(aPdv) //Realiza a leitura de todos os PDVs encontrados

```

```

413 If MD3->(dbSeek(xFilial("MD3") + aPdvs[nCount] + "R"))
414 //Cria objeto da conexao RPC
415 oServer := TRPC():New(AllTrim(MD3->MD3_NOMAMB))
416
417 If oServer:Connect(AllTrim(MD3->MD3_IP), Val(MD3->MD3_PORTA))
418 bOldError := ErrorBlock({|x| STFLoadError(x)}) //Muda code-block de erro
419 //Este tratamento protege o JOB caso a comunicacao caia durante este processo de consulta. Do contrario o JOB ficaria inativo
420
421 Begin Sequence
422 oServer:CallProc("RPCSetType", 3) //Tipo de Licenca consumida
423 oServer:CallProc("RPCSetEnv", MD3->MD3_EMP, MD3->MD3_FIL, Nil, Nil, "FRT", "", {"MBY"}) //Abre conexao com outra empresa
424
425 //Retorna as Cargas Liberadas neste PDV
426 aMBYPdv := oServer:CallProc("STFLoadGetPdv")
427
428 oServer:CallProc("RpcClearEnv") //Limpa Thread
429 oServer:Disconnect() //Encerra conexao
430 nCountPdv++
431 Recover
432 Conout("JOB LOJA1156Job: Ocorreu um erro inesperado durante a consulta com o PDV " + MD3->MD3_NOMAMB + ". Pode estar off-line")
433 LjGrvLog( "Carga","JOB LOJA1156Job: Ocorreu um erro inesperado durante a consulta com o PDV " + MD3->MD3_NOMAMB + ". Pode estar off-line")
434
435 Conout("JOB LOJA1156Job: Detalhes do ambiente PDV : MD3_IP " + AllTrim(MD3->MD3_IP) + " | MD3_PORTA : " + MD3->MD3_PORTA)
436 LjGrvLog( "Carga","JOB LOJA1156Job: Detalhes do ambiente PDV : MD3_IP " + AllTrim(MD3->MD3_IP) + " | MD3_PORTA : " + MD3->MD3_PORTA)
437
438 //Como ocorreu um erro dentro da conexao, nada sera feito pois nao tem como garantir os dados ate nova tentativa
439 aMbyGeral := {}
440 End Sequence
441
442 ErrorBlock(bOldError) //Restaura rotina de erro anterior
443 Else
444 Conout("JOB LOJA1156Job: Nao foi possivel estabelecer uma conexao com o PDV " + MD3->MD3_NOMAMB + "," + "nenhuma carga foi apagada!")
445 LjGrvLog( "Carga","JOB LOJA1156Job: Nao foi possivel estabelecer uma conexao com o PDV " + MD3->MD3_NOMAMB + "," + "nenhuma carga foi apagada!")
446
447 Conout("JOB LOJA1156Job: Detalhes do ambiente PDV : MD3_IP " + AllTrim(MD3->MD3_IP) + " | MD3_PORTA : " + MD3->MD3_PORTA)
448 LjGrvLog( "Carga","JOB LOJA1156Job: Detalhes do ambiente PDV : MD3_IP " + AllTrim(MD3->MD3_IP) + " | MD3_PORTA : " + MD3->MD3_PORTA)
449
450 aMbyGeral := {} //Se algum Pdv nao conectar, aborta o processamento e nao apaga nenhuma carga
451 Exit
452 EndIf
453
454 //Se existe carga liberada no PDV
455 If Len(aMBYPdv) > 0
456 If nCountPdv > 1 //Leitura do segundo PDV em diante
457 aAuxMby := {}
458
459 For nLoads := 1 To Len(aMBYPdv)
460 nPosPdv := Ascan(aMbyGeral, {|x| x == aMBYPdv[nLoads]}) //Pesquisa se existe a carga do novo PDV no anterior pesquisado
461
462 If nPosPdv > 0
463 Add(aAuxMby,aMBYPdv[nLoads]) //Inclui apenas os registros encontrados nos 2 ambientes
464 EndIf

```

```

465         Next nLoads
466
467         aMbyGeral := aClone(aAuxMby) //Vetor aMbyGeral consolida as validacoes a cada leitura de um novo PDV
468     Else
469         aMbyGeral := aClone(aMBYPdv) //Faz uma copia da primeiro PDV para usar como base para os proximos
470     Endif
471 EndIf
472 EndIf
473 Next nCount
474 EndIf
475
476 EndIf
477
478 //Apos analise em cada Pdv apenas as cargas encontradas em TODOS os ambientes poderao ser excluidas
479 //Ou caso tenha recebido a lista de cargas, deleta diretamente mesmo sem analisar os PDVs
480 If Len(aMbyGeral) > 0
481
482     LjGrvLog( "Carga","Deleta cargas")
483
484     MBU->(dbSetOrder(1)) //MBU_FILIAL+MBU_CODIGO
485     MBV->(dbSetOrder(1)) //MBV_FILIAL+MBV_CODGRP+MBV_TABELA
486     MBW->(dbSetOrder(1)) //MBW_FILIAL+MBW_CODGRP+MBW_TABELA
487     MBX->(dbSetOrder(1)) //MBX_FILIAL+MBX_CODGRP+MBX_TABELA+MBX_FIL
488
489     For nCount := 1 To Len(aMbyGeral)
490         //Se achar a carga na Retaguarda apaga o registro
491         If MBU->(dbSeek(xFilial("MBU")+ aMbyGeral[nCount]))
492             //Limpa MBU
493             If RecLock("MBU", .F.)
494                 MBU->(dbDelete())
495                 MBU->(MsUnLock())
496                 //Exclui os registros fisicamente da tabela MBU//LjPackTb("MBU")
497             //Limpa MBV
498             If MBV->(dbSeek(xFilial("MBV")+ aMbyGeral[nCount]))
499                 While MBV->(!EOF()) .And. MBV->(MBV_FILIAL + MBV_CODGRP) == xFilial("MBV") + aMbyGeral[nCount]
500                     RecLock("MBV", .F.)
501                     MBV->(dbDelete())
502                     MBV->(MsUnLock())
503
504                     MBV->(dbSkip())
505                 EndDo
506                 //Exclui os registros fisicamente da tabela MBV//LjPackTb("MBV")
507             EndIf
508
509             //Limpa MBW
510             If MBW->(dbSeek(xFilial("MBW")+ aMbyGeral[nCount]))
511                 While MBW->(!EOF()) .And. MBW->(MBW_FILIAL + MBW_CODGRP) == xFilial("MBW") + aMbyGeral[nCount]
512                     RecLock("MBW", .F.)
513                     MBW->(dbDelete())
514                     MBW->(MsUnLock())
515
516                     MBW->(dbSkip())

```

```

517         EndDo
518
519         //Exclui os registros fisicamente da tabela MBW//LjPackTb("MBW")
520     EndIf
521
522     //Limpa MBX
523     If MBX->(dbSeek(xFilial("MBX")+ aMbyGeral[nCount]))
524         While MBX->(!EOF()) .And. MBX->(MBX_FILIAL + MBX_CODGRP) == xFilial("MBX") + aMbyGeral[nCount]
525             RecLock("MBX", .F.)
526             MBX->(dbDelete())
527             MBX->(MsUnLock())
528
529             MBX->(dbSkip())
530         EndDo
531
532         //Exclui os registros fisicamente da tabela MBX//LjPackTb("MBX")
533     EndIf
534
535
536     //Limpa MH1
537     If MH1->( DbSeek(xFilial("MH1")+ aMbyGeral[nCount]))
538
539         RecLock("MH1", .F.)
540         MH1->( dbDelete() )
541         MH1->( MsUnLock() )
542         // Exclui os registros fisicamente da tabela MH1 -- Agendamento de Cargas
543
544     EndIf
545
546
547     Conout("JOB LOJA1156Job: Carga " + aMbyGeral[nCount] + " apagado com sucesso")
548     LjGrvLog( "Carga","JOB LOJA1156Job: Carga " + aMbyGeral[nCount] + " apagado com sucesso")
549
550     //Limpa os arquivos físicos do servidor
551     aDirectory := Directory(cPath + aMbyGeral[nCount] + "\" + ".*.*", "D" ) //Leitura de todos os niveis da pasta da carga
552
553     For nI := 1 To Len(aDirectory)
554         //Se for um arquivo
555         If aDirectory[nI][5] == "A"
556             Conout("JOB LOJA1156Job: Apagando arquivo " + cPath + aMbyGeral[nCount] + "\" + aDirectory[nI][1])
557             LjGrvLog( "Carga","JOB LOJA1156Job: Apagando arquivo " + cPath + aMbyGeral[nCount] + "\" + aDirectory[nI][1])
558             FErase( cPath + aMbyGeral[nCount] + "\" + aDirectory[nI][1] )
559         ElseIf aDirectory[nI][5] == "D" //Se for um diretorio auxiliar usado pelo Ctree
560             Conout("JOB LOJA1156Job: Apagando Diretorio " + cPath + aMbyGeral[nCount] + "\" + aDirectory[nI][1])
561             LjGrvLog( "Carga","JOB LOJA1156Job: Apagando Diretorio " + cPath + aMbyGeral[nCount] + "\" + aDirectory[nI][1])
562             DirRemove(cPath + aMbyGeral[nCount] + "\" + aDirectory[nI][1]) //Apaga diretorio especifico do ambiente Ctree
563         EndIf
564     Next nI
565
566     DirRemove(cPath + aMbyGeral[nCount]) //Apaga a pasta principal com o numero da carga
567 Else
568     Conout("JOB LOJA1156Job: Nao foi possivel obter acesso exclusivo da carga " + aMbyGeral[nCount] + ". Este registro sera apagado posteriormente" )

```



```

569         LjGrvLog( "Carga","JOB LOJA1156Job: Nao foi possivel obter acesso exclusivo da carga " + aMbyGeral[nCount] + ". Este registro sera apagado posteriormente")
570     EndIf
571 EndIf
572 Next nCount
573 Else
574     lRet := .F.
575 Endif
576
577 RestArea(aAreaMD4)
578 RestArea(aAreaMD3)
579
580 LjGrvLog( "Carga","STFLoadDel Fim Retorno: " ,lRet )
581
582 Return lRet
583
584
585 //-----
586 /*{Protheus.doc} STFLoAutDel
587 Exclui as cargas na Retaguarda sem analisar se os PDVs baixaram.
588 Para evitar atingir o numero maximo de cargas permitidos(MV_LJILQTD) e travar as cargas automaticas.
589 Apaga somente cargas incrementais
590 @type function
591 @param      nQtdeDel , numérico, Quantidade de cargas para apagar
592 @param      nSizeLoad , numérico, Quantidade de cargas ativas
593 @return lRet - Retorna se excluiu as cargas
594 */
595 //-----
596 Static Function STFLoAutDel( nQtdeDel , nSizeLoad )
597
598 Local aArea      := GetArea()      // Guarda area
599 Local aDelLoad   := {}             // Array de cargas a apagar
600 Local lRet       := .T.            // Retorno
601
602 Default nQtdeDel := 0
603 Default nSizeLoad := 0
604
605 LjGrvLog( "Carga","STFLoAutDel Início" )
606
607 If (nQtdeDel > 0 .AND. nSizeLoad > 0 )
608
609     DbSelectArea("MBU")
610     MBU->(dbSetOrder(2)) //MBU_FILIAL+MBU_TIPO
611     If MBU->(dbSeek(xFilial("MBU")+ "2" ))
612         While MBU->(!EOF()) .AND. (MBU->MBU_FILIAL + MBU->MBU_TIPO == xFilial("MBU") + "2") .AND. (Len(aDelLoad) < nQtdeDel)
613             If AllTrim(MBU->MBU_INTINC) == "2"
614                 Add(aDelLoad, MBU->MBU_CODIGO )
615             EndIf
616             MBU->(dbSkip())
617         EndDo
618     EndIf
619
620 EndIf

```

```

621
622 If Len(aDelLoad) > 0
623     LjGrvLog( "Carga","Cargas a deletar " ,aDelLoad )
624     STFLoadDel(aDelLoad)
625 EndIf
626
627 RestArea(aArea)
628
629 LjGrvLog( "Carga","STFLoadDel Fim Retorno: " ,lRet )
630
631 Return lRet
632
633
634 //-----
635 /*{Protheus.doc} STFLoadError
636 Trata erro na consulta Rpc para nao cair o JOB
637 @param
638 @return Nil
639 */
640 //-----
641
642 Static Function STFLoadError(e)
643
644 Local lRet := .F. //Retorno da funcao
645
646 //Verifica se encontrou erros
647 If e:gencode > 0
648     Conout("Ocorreu o erro: " + e:DESCRIPTION)
649     LjGrvLog( "Carga","Ocorreu o erro: " + e:DESCRIPTION )
650
651     Conout("Pilha de chamada: " + e:ERRORSTACK)
652     LjGrvLog( "Carga","Pilha de chamada: " + e:ERRORSTACK )
653
654     lRet := .T.
655     Break
656 EndIf
657
658 Return(lRet)
659
660 //-----
661 /*{Protheus.doc} STFLoadGetPdv
662 Retorna as cargas atualizadas no PDV
663 @param
664 @return aRet - Retorna as cargas atualizadas no PDV
665 */
666 //-----
667
668 Function STFLoadGetPdv()
669
670 Local aArea := GetArea() //Guarda area
671 Local aRet := {} //Array de retorno
672

```

```

673 MBY->(DbSetOrder(1)) //MBY_FILIAL + MBY_CODGRP
674
675 //Posiciona no primeiro registro
676 If MBY->(dbSeek(xFilial("MBY")))
677     While MBY->(!Eof())
678         //Processa apenas cargas recebidas e atualizadas no PDV
679         If MBY->MBY_STATUS == "2" .AND. MBY->MBY_INTINC == "2"
680             Aadd(aRet, MBY->MBY_CODGRP)
681         EndIf
682     EndWhile
683     MBY->(dbSkip())
684 EndDo
685 Endif
686
687 RestArea(aArea)
688
689 Return aRet
690
691 //-----
692 /*{Protheus.doc} STFLoadItem
693 Retorna o Vetor com a informações da carga e seu tempo
694 @param
695 @return aRet - Array de cargas e tempo de execucao
696 */
697 //-----
698
699 Static Function STFLoadItem()
700
701 Local nTimeFut := 0 //Tempo de execucao futuro
702 Local aRet := {} //Array de retorno
703 Local dDateTemp := Date() //Data temporaria
704 Local cFileTemp := GetNextAlias() //Alias de Trabalho
705 Local oTempTable := Nil //Objeto para criação da tabela temporaria.
706 Local lRet := STFLoadTemp(@cFileTemp,@oTempTable) //Retorno
707 Local aMBU := MBU->( GetArea() ) //Guarda Area MBU
708 Local lDelLoadAut := .F. //Controla se deleta a carga automatica
709
710 //Verifica se nao foram encontrados erros
711 If lRet
712     MH1->(dbSetOrder(1))
713     MBU->(dbSetOrder(1)) //MBU_FILIAL+MBU_CODIGO
714
715     //Busca cargas agendadas
716     If MH1->(dbSeek(xFilial("MH1")))
717         While !MH1->(EOF())
718
719             If MBU->( DbSeek( xFilial("MBU") + MH1->MH1_COD ) )
720                 //Se achar a carga verifica se eh tipo template e incremental
721                 If !( AllTrim(MBU->MBU_TIPO) = "1" .AND. AllTrim(MBU->MBU_INTINC) = "2")
722                     lDelLoadAut := .T.
723                 EndIf
724             Else

```

```

725         //Se não Achar a Carga deleta Geracao automatica(MH1)
726         lDelLoadAut := .T.
727     EndIf
728
729     If lDelLoadAut
730
731         RecLock("MH1", .F. )
732         MH1->( DbDelete() )
733         MH1->( MsUnLock() )
734
735         lDelLoadAut := .F.
736         MH1->( DbSkip() )
737         Loop
738     EndIf
739
740     If MH1->MH1_STATUS == "A"
741         dbSelectArea(cFileTemp)
742
743         If !(cFileTemp)->(dbSeek(MH1->MH1_COD)) //Inclui na temporaria um novo Registro
744             nTimeFut := MH1->MH1_TIME + Round(Seconds(), 0)
745             STFLoadTime(@dDateTemp, @nTimeFut)
746
747             //Gera arquivo temporario
748             RecLock(cFileTemp , .T.)
749             (cFileTemp)->CODIGO := MH1->MH1_COD
750             (cFileTemp)->TEMPO := MH1->MH1_TIME
751             (cFileTemp)->FUTURO := nTimeFut
752             (cFileTemp)->DATACG := dDateTemp
753             (cFileTemp)->HORAI := MH1->MH1_HORAI
754             (cFileTemp)->HORAF := MH1->MH1_HORAF
755             (cFileTemp)->(MsUnLock())
756
757             Aadd(aRet, {(cFileTemp)->CODIGO, (cFileTemp)->FUTURO, (cFileTemp)->DATACG, (cFileTemp)->HORAI, (cFileTemp)->HORAF})
758             //A unica coisa a ser avaliada e a mudanca do tempo da carga, porque se ela ficar inativa o vetor aRet nao sera montado
759             ElseIf (cFileTemp)->TEMPO <> MH1->MH1_TIME .Or. (cFileTemp)->HORAI <> MH1->MH1_HORAI .Or. (cFileTemp)->HORAF <> MH1->MH1_HORAF
760                 nTimeFut := MH1->MH1_TIME + Round(Seconds(), 0) //Atualiza o novo tempo da proxima carga
761                 nTimeFut := STFLoadTime(@dDateTemp, @nTimeFut)
762
763                 //Altera o arquivo temporario
764                 RecLock(cFileTemp, .F.)
765                 (cFileTemp)->TEMPO := MH1->MH1_TIME
766                 (cFileTemp)->FUTURO := nTimeFut
767                 (cFileTemp)->DATACG := dDateTemp
768                 (cFileTemp)->HORAI := MH1->MH1_HORAI
769                 (cFileTemp)->HORAF := MH1->MH1_HORAF
770                 (cFileTemp)->(MsUnLock())
771
772                 Conout("JOB LOJA1156job: 0 tempo para geracao da carga " + MH1->MH1_COD + " foi alterado. Novo valor em segundos: " + Alltrim(Str(MH1->MH1_TIME)))
773                 LjGrvLog( "Carga", "JOB LOJA1156job: 0 tempo para geracao da carga " + MH1->MH1_COD + " foi alterado. Novo valor em segundos: " +
Alltrim(Str(MH1->MH1_TIME)) )
774
775                 Aadd(aRet,{(cFileTemp)->CODIGO, (cFileTemp)->FUTURO, (cFileTemp)->DATACG, (cFileTemp)->HORAI, (cFileTemp)->HORAF})

```

```

776         Else
777             Aadd(aRet,{(cFileTemp)->CODIGO, (cFileTemp)->FUTURO, (cFileTemp)->DATACG, (cFileTemp)->HORAI, (cFileTemp)->HORAFA}) //Alimenta o vetor com o conteudo
            anterior do temporario
778         EndIf
779     Endif
780
781     MH1->(DbSkip())
782     EndDo
783 EndIf
784
785 // -- Apago a tabela temporaria criada na função STFLoadTemp
786 oTempTable:Delete()
787
788 EndIf
789
790 RestArea(aMBU)
791
792 Return aRet
793
794 //-----
795 /*{Protheus.doc} STFLoadTemp
796 Constroe e valida arquivo temporario
797 @param      cFileTemp - Arquivo temporario de trabalho
798 @return lRet - Retorna se conseguiu construir o arquivo temporario
799 */
800 //-----
801
802 Static Function STFLoadTemp(cFileTemp,oTempTable)
803
804 Local aStruct      := {}                //Estrutura
805 Local nSizeCOD     := 0                //Tamanho do campo codigo
806 Local nSizeTime    := 0                //Tamanho campo tempo
807 Local nTimeLim     := 0                //Tamanho do campo hora limite
808 Local lRet         := .T.              //Retorno da funcao
809
810 Default cFileTemp := ""
811
812
813 oTempTable := FWTemporaryTable():New()
814
815
816 If AliasInDic("MH1")
817     DbSelectArea("MH1")
818     nSizeCOD     := TamSx3("MH1_COD")[1] //Tamanho do campo codigo
819     nSizeTime    := TamSx3("MH1_TIME")[1] //Tamanho campo tempo
820     nTimeLim     := TamSx3("MH1_HORAI")[1] //Tamanho do campo hora limite
821 Else
822     lRet := .F.
823     LjGrvLog( "Carga",STR0001 )// "JOB L0JA1156job: tabela MH1 não encontrada, a falta da tabela implica diretamente no funcionamento das cargas automaticas."
824 EndIf
825
826 If lRet

```

```

827 //Estrutura do arquivo temporario
828 Aadd(aStruct, {"CODIGO", "C", nSizeCOD , 0})
829 Aadd(aStruct, {"TEMPO", "N", nSizeTime , 0})
830 Aadd(aStruct, {"FUTURO", "N", nSizeTime , 0})
831 Aadd(aStruct, {"DATACG", "D", 8 , 0})
832 Aadd(aStruct, {"HORAI", "C", nTimeLim , 0})
833 Aadd(aStruct, {"HORA", "C", nTimeLim , 0})
834
835 // -- alimenta estrutura da tabela temporaria para ser criado posteriormente
836 oTempTable:SetFields(aStruct)
837
838 // -- alimenta indice da tabela temporario para ser criado posteriormente
839 oTempTable:AddIndex("01", {"CODIGO"} )
840
841 // -- Cria tabela temporaria
842 oTempTable:Create()
843
844 // -- Retorna a Alias criada.
845 cFileTemp := oTempTable:GetAlias()
846
847 If Empty(cFileTemp)
848     Conout("JOB L0JA1156job: Nao foi possivel criar o arquivo temporario. O processo sera interrompido para nova tentativa.")
849     LjGrvLog( "Carga","JOB L0JA1156job: Nao foi possivel criar o arquivo temporario. O processo sera interrompido para nova tentativa." )
850     lRet := .F.
851 EndIf
852
853 EndIf
854
855 Return lRet
856
857 //-----
858 /*{Protheus.doc} STFLoadTime
859 Retorna o tempo da próxima carga
860 @param      dDateTemp - Data do arquivo temporario
861 @param      nTimeFut - Valor do tempo futuro
862 @return Nil
863 */
864 //-----
865
866 Static Function STFLoadTime(dDateTemp, nTimeFut)
867
868 If nTimeFut >= 86339 //Limite para o dia seguinte
869     dDateTemp := Date() + 1 //Proximo dia
870     nTimeFut -= 86339
871 Else
872     dDateTemp := Date()
873 EndIf
874
875 Return Nil
876
877 //-----
878 /*{Protheus.doc} STFThreads

```

```

879 Essa função faz a chamada para a geração da carga para cada
880 agendamento ativo.
881
882 Para cada agendamento, sera criado uma nova thread.
883
884 @param
885 @return .T.
886 /*/
887 //-----
888 Static Function STFTThreads(cLoadDel, cLoAutDel)
889
890 Local cQuery      := ""
891 Local cAlias      := ""
892 Local nContador   := 0
893 Local lContinua   := .T.
894 Local aThreads    := {}
895 Local nSizeLoad   := 0
896 Local lExclui1    := .F.
897 Local lExclui2    := .F.
898 Local nPosThread  := 0
899
900 cQuery := "SELECT R_E_C_N_O_REC "
901 cQuery += " FROM " + RetSqlName("MIO")
902 cQuery += " WHERE MIO_FILIAL = '" + xFilial("MIO") + "'"
903 cQuery += " AND MIO_ATIVO = '1'"
904 cQuery += " AND D_E_L_E_T_ = ' '"
905
906 PutGlbValue("cThread", "0")
907
908 While lContinua
909     cAlias := GetNextAlias()
910     DbUseArea(.T., "TOPCONN", TcGenQry( , , cQuery), cAlias, .T., .F.)
911
912     //----Inicio para exclusão das cargas----
913     aThreads := GetUserInfoArray(.T.)
914     nPosThread := aScan(aThreads, {|x| cValToChar(x[3]) == GetGlbValue("cThread")})
915
916     If !(nPosThread > 0 .AND. aThreads[nPosThread][5] == "STFDEL CARGA")
917
918         //Quantidade de cargas geradas
919         nSizeLoad := LJ1156CountLoads()
920
921         //Se o numero de cargas for superior ao desejado, aplica o processo para apagar os dados da MBU com base nos PDVs filhos
922         If Val(cLoadDel) > 0 .AND. nSizeLoad > Val(cLoadDel)
923             lExclui1 := .T.
924         EndIf
925
926         //Apaga as cargas independente de os PDVs terem baixado por ter atingido o limite do parametro MV_LJILQTD
927         If Val(cLoAutDel) > 0 .AND. nSizeLoad >= SuperGetMV("MV_LJILQTD", .F., 200)
928             lExclui2 := .T.
929         EndIf
930

```

```

931      If lExclui1 .OR. lExclui2
932          StartJob("STFDeLCarga", GetEnvServer(), .F., cEmpAnt, cFilAnt, lExclui1, lExclui2, cLoAutDel, nSizeLoad)
933      EndIf
934
935  EndIf
936
937  lExclui1 := .F.
938  lExclui2 := .F.
939  //----Final para exclusão das cargas----
940
941
942  While !(cAlias)->( Eof() )
943      MIO->(dbGoto((cAlias)->REC))
944
945      If STFRecorre(aThreads)
946          StartJob("LOJA1156Job", GetEnvServer(), .F., AllTrim(MIO->MIO_GRPCAR), AllTrim(MIO->MIO_SEQ), cEmpAnt, cFilAnt )
947      EndIf
948      Sleep(2000)
949      (cAlias)->( DbSkip() )
950  EndDo
951
952  (cAlias)->( DbCloseArea() )
953
954  nContador++
955
956  //Reinicia a thread a cada 50 mil execuções
957  If nContador <= 50000
958      Sleep(2000)
959  Else
960      lContinua := .F.
961  EndIf
962 End
963
964 Return .T.
965
966 //-----
967 /*{Protheus.doc} STFRecorre
968 Função para verificar a recorrência do agendamento,
969 verifica se deve ou não executar a carga.
970
971 @param
972 @return      .T.
973 */
974 //-----
975 Static Function STFRecorre(aThreads)
976
977 Local aDiaSemana      := {{ "SUNDAY", "MIO_DOM"}, {"MONDAY", "MIO_SEG"}, {"TUESDAY", "MIO_TER"}, {"WEDNESDAY", "MIO_QUA"}, {"THURSDAY", "MIO_QUI"}, {"FRIDAY", "MIO_SEX"}, {"SATURDAY", "MIO_S
AB"}}}
978 Local cDia           := ""
979 Local nPos           := 0
980 Local lRet           := .F.
981 Local cDif           := ""

```



```

982 Local nPosThread      := 0
983 Local cDtFim           := SubStr(MIO->MIO_HRFIM,7,4) + SubStr(MIO->MIO_HRFIM,4,2) + SubStr(MIO->MIO_HRFIM,1,2)
984
985 //Retorna o dia da semana em inglês
986 cDia := AllTrim(Upper(CDow(Date())))
987
988 If !Empty(cDia)
989     nPos := aScan(aDiaSemana,{|x| x[1] == cDia})
990
991     If nPos > 0
992         If MIO->&(aDiaSemana[nPos][2])
993
994             If Empty(MIO->MIO_THREAD)
995                 lRet := .T.
996             Else
997                 //Caso a thread não exista, significa que já foi encerrada desde a última geração.
998                 nPosThread := aScan(aThreads,{|x| cValToChar(x[3]) == AllTrim(MIO->MIO_THREAD)})
999
1000                 If !((nPosThread > 0) .AND. (aThreads[nPosThread][5] == "LOJA1156JOB"))
1001
1002                     //Caso o campo Hora Fim for vazio, pode ter ocorrido algum problema na gravação,
1003                     //portanto considero que devera gerar a carga novamente.
1004                     If !Empty(MIO->MIO_HRFIM)
1005
1006                         //Pega a diferença entre hora final e a atual
1007                         cDif := ElapTime(SubStr(MIO->MIO_HRFIM,12),Time())
1008
1009                         If MIO->MIO_PERIOD == "1" //Hora
1010                             If (MIO->MIO_RECRR == 24 .AND. DateDiffDay(SToD(cDtFim),SToD(SubStr(FWTimeStamp(1),1,8))) >= 1) .OR. ;
1011                                 (Val(SubStr(cDif,1,2)) >= MIO->MIO_RECRR)
1012                                 lRet := .T.
1013                             EndIf
1014                         Else //Minuto
1015                             If (Val(SubStr(cDif,1,2)) >= 1) .OR. (Val(SubStr(cDif,4,2)) >= MIO->MIO_RECRR) .OR. (DateDiffDay(SToD(cDtFim),SToD(SubStr(FWTimeStamp(1),1,8)))
1016                                 >= 1)
1017                                 lRet := .T.
1018                             EndIf
1019                         EndIf
1020                     Else
1021                         lRet := .T.
1022                     EndIf
1023                 EndIf
1024             EndIf
1025         EndIf
1026     EndIf
1027 EndIf
1028 EndIf
1029
1030 EndIf
1031
1032 Return lRet

```

```

1033
1034 //-----
1035 /*{Protheus.doc} STFDelCarga
1036 Função que faz a exclusão das cargas conforme a configuração no
1037 arquivo appserver.ini na seção [STFLoadRet]
1038
1039 @param
1040 @return .T.
1041 */
1042 //-----
1043 Function STFDelCarga(cEmpCarga, cFilCarga, lExclui1, lExclui2, cLoAutDel, nSizeLoad)
1044
1045 //Inicia o ambiente
1046 RPCSetType(3)
1047 RpcSetEnv(cEmpCarga, cFilCarga,,,"FRT")
1048
1049 PutGlbValue("cThread",cValToChar(ThreadID()))
1050
1051 LjGrvLog( "Carga", "Inicio da exclusão da carga - Thread: " + cValToChar(ThreadID()) )
1052 CoNout("Inicio da exclusao da carga - Thread: " + cValToChar(ThreadID()))
1053
1054 If lExclui1
1055     If !STFLoadDel()
1056         Conout("JOB L0JA1156job: STFLoadDel(): Aguardando a liberacao do(s) PDV(s) para excluir o proximo lote na retaguarda. Sera feita uma nova tentativa para a filial " +
1057             cFilCarga)
1058         LjGrvLog( "Carga","JOB L0JA1156job: STFLoadDel(): Aguardando a liberacao do(s) PDV(s) para excluir o proximo lote na retaguarda. Sera feita uma nova tentativa para a
1059             filial " + cFilCarga)
1060         EndIf
1061     EndIf
1062
1063 //Apaga as cargas independente de os PDVs terem baixado por ter atingido o limite do parametro MV_LJILQTD
1064 If lExclui2
1065     STFLoAutDel( Val(cLoAutDel ), nSizeLoad )
1066 EndIf
1067
1068 LjGrvLog( "Carga", "Fim da exclusão da carga - Thread: " + cValToChar(ThreadID()) )
1069 CoNout("Fim da exclusao da carga - Thread: " + cValToChar(ThreadID()))
1070
1071 Return .T.

```

Framework/Collection/STFWACollection.prw

160 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "MSOBJECT.CH"
3
4 //UAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA;
5 //³0 Protheus necessita ter ao menos uma funcao para que o      ³
6 //³fonte seja exibido na inspecao de fontes do RPO.              ³
7 //AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAU
8 User Function STFWACollec ; Return
9
10 //-----
11 /*/{Protheus.doc} STFWACollection
12 Classe abstrata responsavel em guardar os metodos comuns das colecoes esta classe nao pode ser
13 instanciada, somente herdada
14
15 @param
16 @see    Veja tambem as classes STFWCHashTable, STFWCList, STFWCStack, STFWCQueue.
17 @return Self
18 @todo
19 @obs
20 @sample
21 /*/
22 //-----
23
24 Class STFWACollection
25
26     Data aColecao                                //Array que ira armazenar os dados
27
28     Method STFWACollection()                      //Metodo construtor
29     Method Count()                                //Metodos que ira retornar o numero de objetos da Colecao
30     Method Erase()                                //Limpa o array aColecao (Inicializa)
31     Method Elements(nIndex)                       //Metodo que ira retornar um elemento de um determinado indice
32     Method ToArray()                              //Metodo que retorna os elementos da colecao em um array
33     Method Clonar()                               //Metodo abstrato que ira fazer um clone de um objeto, tem que ser sobrescrito pelas classes filhos
34
35 EndClass
36
37 //-----
38 /*/{Protheus.doc} STFWACollection
39 Construtor da classe STFWACollection.
40
41 @param
42 @see    Veja tambem as classes STFWCHashTable, STFWCList, STFWCStack, STFWCQueue.
43 @return Self
44 @todo
45 @obs
46 @sample
47 /*/
48 //-----
49
50 Method STFWACollection() Class STFWACollection
51
52     Self:aColecao := {}

```

```

53
54 Return Self
55
56 //-----
57 /*{Protheus.doc} Count
58 Retorna a quantidade de elementos da colecao.
59
60 @param
61 @see    Veja tambem as classes STFWCHashTable, STFWCList, STFWCStack, STFWCQueue.
62 @return Conta a quantidade de registros no aColecao.
63 @todo
64 @obs
65 @sample
66 */
67 //-----
68
69 Method Count() Class STFWACollection
70
71     Local nRetorno := 0                //Variavel de retorno do metodo
72
73     nRetorno := Len(Self:aColecao)
74
75 Return nRetorno
76
77 //-----
78 /*{Protheus.doc} Erase
79 Limpa o array aColecao (Inicializa).
80
81 @param
82 @see    Veja tambem as classes STFWCHashTable, STFWCList, STFWCStack, STFWCQueue.
83 @return Conta a quantidade de registros no aColecao.
84 @todo
85 @obs
86 @sample
87 */
88 //-----
89
90 Method Erase() Class STFWACollection
91
92     //Inicializa o array
93     ::aColecao := {}
94
95 Return Nil
96
97 //-----
98 /*{Protheus.doc} Elements
99 Retorna um elemento da colecao atraves de um indice.
100
101 @param  ExpN1 (1 - nIndex) - Posicao do array.
102 @see    Veja tambem as classes STFWCHashTable, STFWCList, STFWCStack, STFWCQueue.
103 @return Objeto oElement
104 @todo

```

```

105 @obs
106 @sample
107 /*/
108 //-----
109
110 Method Elements(nIndex) Class STFWACollection
111
112     Local oElement := Nil           //Objeto que sera retornado na funcao
113
114     oElement := Self:aColecao[nIndex]
115
116 Return oElement
117
118 //-----
119 /*/{Protheus.doc} ToArray
120 Responsavel em retornar o conteudo da colecao em um array
121
122 @param
123 @see     Veja tambem as classes STFWCHashTable, STFWCList, STFWCStack, STFWCQueue.
124 @return  aRetorno com o conteudo do objeto
125 @todo
126 @obs
127 @sample
128 /*/
129 //-----
130
131 Method ToArray() Class STFWACollection
132
133     Local aRetorno := {}           //Retorno do metodo
134     Local nCount   := 0           //Variavel auxiliar contador
135
136     //Atribui ao array o conteudo do objeto
137     For nCount := 1 To Self:Count()
138         AADD(aRetorno, Self:aColecao[nCount])
139     Next
140
141 Return aRetorno
142
143 //-----
144 /*/{Protheus.doc} Clonar
145 Metodo abstrato que ira fazer um clone de um objeto, tem que ser sobrescrito pelas classes filhos
146
147 @param
148 @see     Veja tambem as classes STFWCHashTable, STFWCList, STFWCStack, STFWCQueue.
149 @return  Objeto oReturn
150 @todo
151 @obs
152 @sample
153 /*/
154 //-----
155
156 Method Clonar() Class STFWACollection

```

```
157
158     Local oRetorno := Nil           //Retorno do metodo
159
160 Return oRetorno
```

Framework/Collection/STFWCHashTable.prw

262 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "MSOBJECT.CH"
3
4 User Function STFWCHashT ; Return // "dummy" function - Internal Use
5
6 //-----
7 /*{Protheus.doc} STFWCHashTable
8 Classe responsavel em guardar uma colecao de objetos.
9
10 @param
11 @see    Veja tambem a classe STFWACollection
12 @return Self
13 @todo
14 @obs
15 @sample
16 */
17 //-----
18
19 Class STFWCHashTable From STFWACollection
20
21     Method STFWCHashTable()           //Metodo construtor
22     Method Add(oKey, oObject)         //Metodo que ira adicionar um objeto na Colecao
23     Method Contains(oKey)             //Metodo que ira verificar se existe um determinado elemento na Colecao
24     Method Remove(oKey)               //Metodo para remover um objeto da Colecao
25     Method ElementKey(oKey)           //Metodo que ira retornar um elemento da Colecao
26     Method Elements(nIndice)          //Metodo que ira retornar um elemento de um determinado indice
27     Method ElementPar(nIndice)
28     Method ToArray()                  //Metodo que retorna os elementos da colecao em um array
29     Method Clonar()                   //Metodo que ira fazer um clone do objeto LJCHashTable usando recursividade
30
31 EndClass
32
33 //-----
34 /*{Protheus.doc} STFWCHashTable
35 Construtor da classe STFWCHashTable
36
37 @param
38 @see    Veja tambem a classe STFWACollection
39 @return Self
40 @todo
41 @obs
42 @sample
43 */
44 //-----
45
46 Method STFWCHashTable() Class STFWCHashTable
47
48     _Super:STFWACollection()
49
50 Return Self
51
52 //-----

```

```

53 */{Protheus.doc} Add
54 Adiciona um elemento a colecao.
55
56 @param   ExpV1 (1 - oKey)      - Chave da colecao.
57 @param   Exp01 (2 - oObject) - Elemento da colecao.
58 @see     Veja tambem a classe STFWACollection
59 @return  Self
60 @todo
61 @obs     Metodo da classe STFWCHashTable
62 @sample
63 */
64 //-----
65
66 Method Add(oKey, oObject) Class STFWCHashTable
67
68     //Verifica se o elemento ja existe na colecao, se sim, remove
69     If Self:Contains(oKey)
70         Self:Remove(oKey)
71     EndIf
72
73     //Adiciona um elemento na colecao
74     AADD(Self:aColecao, {oKey, oObject})
75
76 Return Nil
77
78 //-----
79 */{Protheus.doc} Contains
80 Verifica se existe um determinado elemento na colecao.
81
82 @param   ExpV1 (1 - oKey)      - Chave da colecao.
83 @see     Veja tambem a classe STFWACollection
84 @return  Logico se encontro um elemento
85 @todo
86 @obs     Metodo da classe STFWCHashTable
87 @sample
88 */
89 //-----
90
91 Method Contains(oKey) Class STFWCHashTable
92
93     Local lRetorno := .F.          //Variavel de retorno do metodo
94     Local nPosicao  := 0            //Posicao do array
95
96     //Verifica se possui algum elemento na colecao
97     If Self:Count() > 0
98         Begin Sequence
99             //Procura o elemento na colecao
100             nPosicao := Ascan(Self:aColecao, {|x| x[1] == oKey})
101
102             //Verifica se encontrou o elemento
103             If nPosicao > 0
104                 lRetorno := .T.

```



```

105         EndIf
106         Recover
107         lRetorno := .F.
108     End Sequence
109 EndIf
110
111 Return lRetorno
112
113 //-----
114 /*{Protheus.doc} Remove
115 Remove um elemento da colecao.
116
117 @param ExpV1 (1 - oKey)      - Chave da colecao.
118 @see    Veja tambem a classe STFWACollection
119 @return
120 @todo
121 @obs     Metodo da classe STFWCHashTable
122 @sample
123 */
124 //-----
125
126 Method Remove(oKey) Class STFWCHashTable
127
128     Local nPosicao := 0                //Posicao do produto no array
129
130     //Procura o elemento na colecao
131     nPosicao := Ascan(Self:aColecao, {|x| x[1] == oKey})
132     //Apaga o elemento da colecao
133     ADel(Self:aColecao, nPosicao)
134     //Redimensiona a colecao
135     ASize(Self:aColecao, Self:Count() - 1)
136
137 Return Nil
138
139 //-----
140 /*{Protheus.doc} ElementKey
141 Retorna um elemento da colecao atraves da chave.
142
143 @param ExpV1 (1 - oKey)      - Chave da colecao.
144 @see    Veja tambem a classe STFWACollection
145 @return Objeto oElement
146 @todo
147 @obs     Metodo da classe STFWCHashTable
148 @sample
149 */
150 //-----
151
152 Method ElementKey(oKey) Class STFWCHashTable
153
154     Local oElement := Nil            //Objeto que sera retornado na funcao
155     Local nPosicao := 0                //Posicao do elemento no array
156

```

```

157 //Procura o elemento na colecao
158 nPosicao := Ascan(Self:aColecao, {|x| x[1] == oKey})
159
160 //Pega o elemento
161 oElement := Self:Elements(nPosicao)
162
163 Return oElement
164
165 //-----
166 /*{Protheus.doc} Elements
167 Retorna um elemento da colecao atraves de um indice.
168
169 @param ExpV1 (1 - oKey) - Chave da colecao.
170 @see Veja tambem a classe STFWACollection
171 @return Objeto oElement
172 @todo
173 @obs Metodo da classe STFWCHashTable
174 @sample
175 */
176 //-----
177
178 Method Elements(nIndice) Class STFWCHashTable
179
180 Local oElement := Nil //Objeto que sera retornado na funcao
181
182 oElement := Self:aColecao[nIndice, 2]
183
184 Return oElement
185
186 //-----
187 /*{Protheus.doc} ElementPar
188 Retorna um elemento da colecao atraves de um indice.
189
190 @param Indice de busca dentro do elemento.
191 @see Veja tambem a classe STFWACollection e a classe STFWCKeyValuePar
192 @return Objeto oPar
193 @todo
194 @obs Metodo da classe STFWCHashTable
195 @sample
196 */
197 //-----
198
199 Method ElementPar(nIndice) Class STFWCHashTable
200
201 Local oPar := Nil //Objeto que sera retornado na funcao
202
203 oPar := STFWCKeyValuePar():STFWCKeyValuePar(Self:aColecao[nIndice, 1], Self:aColecao[nIndice, 2])
204
205 Return oPar
206
207 //-----
208 /*{Protheus.doc} Clonar

```

```

209 Responsavel em clonar o objeto STFWCHashTable
210
211 @param
212 @see    Veja tambem a classe STFWACollection
213 @return
214 @todo
215 @obs    Metodo da classe STFWCHashTable
216 @sample
217 */
218 //-----
219
220 Method Clonar() Class STFWCHashTable
221
222     Local oRetorno := Nil           //Retorno do metodo
223     Local nCount   := 0
224
225     //Estancia o objeto STFWCColecao (recursividade)
226     oRetorno := STFWCHashTable():STFWCHashTable()
227
228     //Atribui ao array do novo objeto um clone do array original
229     For nCount := 1 To Self:Count()
230         If Valtype(Self:Elements(nCount)) == "0"
231             oRetorno:ADD(Self:aColecao[nCount, 1], Self:Elements(nCount):Clonar())
232         Else
233             oRetorno:ADD(Self:aColecao[nCount, 1], Self:Elements(nCount))
234         EndIf
235     Next
236
237 Return oRetorno
238
239 //-----
240 */{Protheus.doc} ToArray
241 Responsavel em retornar o conteudo da colecao em um array
242
243 @param
244 @see    Veja tambem a classe STFWACollection
245 @return Array
246 @todo
247 @obs    Metodo da classe STFWCHashTable
248 @sample
249 */
250 //-----
251
252 Method ToArray() Class STFWCHashTable
253
254     Local aRetorno := {}           //Retorno do metodo
255     Local nCount   := 0
256
257     //Atribui ao array o conteudo do objeto
258     For nCount := 1 To Self:Count()
259         AADD(aRetorno, Self:aColecao[nCount, 2])
260     Next

```

```
261  
262 Return aRetorno
```

Framework/Collection/STFWCKeyValuePar.prw

28 linhas após remoção de metadados de autoria/data

```
1 #INCLUDE "PROTHEUS.CH"
2
3 User Function STFWCKeyValue ; Return // "dummy" function - Internal Use
4
5 //-----
6 /*{Protheus.doc} STFWCKeyValuePar
7 Classe responsavel em guardar uma colecao de objetos.
8
9 @param
10 @see
11 @return Self
12 @todo
13 @obs
14 @sample
15 */
16 //-----
17
18 Class STFWCKeyValuePar
19     Data oKey
20     Data oValue
21
22     Method STFWCKeyValuePar()
23 EndClass
24
25 Method STFWCKeyValuePar(oKey, oValue) Class STFWCKeyValuePar
26     Self:oKey := oKey
27     Self:oValue := oValue
28 Return
```

Framework/Collection/STFWCQueue.prw

143 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "MSOBJECT.CH"
3
4 User Function STFWCQueue ; Return // "dummy" function - Internal Use
5
6 //-----
7 /*{Protheus.doc} STFWCQueue
8 Classe responsavel em guardar objetos em uma fila (First in - First Out).
9
10 @param
11 @see    Veja tambem a classe STFWACollection
12 @return Self
13 @todo
14 @obs
15 @sample
16 */
17 //-----
18
19 Class STFWCQueue From STFWACollection
20
21     Method STFWCQueue()                //Metodo construtor
22     Method Clonar()                    //Metodo que ira fazer um clone do objeto LJCQueue usando recursividade
23     Method Enqueue(oObject)            //Metodo que ira adicionar um elemento na fila
24     Method Peek()                      //Metodo que retorna o primeiro elemento da fila
25     Method Dequeue()                   //Metodo para remover um elemento da fila
26
27 EndClass
28
29 //-----
30 /*{Protheus.doc} STFWCQueue
31 Construtor da classe STFWCQueue
32
33 @param
34 @see    Veja tambem a classe STFWACollection
35 @return Self
36 @todo
37 @obs
38 @sample
39 */
40 //-----
41
42 Method STFWCQueue() Class STFWCQueue
43
44     _Super:STFWACollection()
45
46 Return Self
47
48 //-----
49 /*{Protheus.doc} Clonar
50 Responsavel em clonar o objeto STFWCQueue
51
52 @param

```

```

53 @see      Veja tambem a classe STFWACollection
54 @return   oRetorno - Contendo a quantidade de elementos
55 @todo
56 @obs
57 @sample
58 */
59 //-----
60
61 Method Clonar() Class STFWCQueue
62
63     Local oRetorno := Nil           //Retorno do metodo
64     Local nCount   := 0
65
66     //Instancia o objeto LJCQueue (recursividade)
67     oRetorno := STFWCQueue():STFWCQueue()
68
69     //Atribui ao array do novo objeto um clone do array original
70     For nCount := 1 To Self:Count()
71         If Valtype(Self:Elements(nCount)) == "0"
72             oRetorno:ADD(Self:Elements(nCount):Clonar())
73         Else
74             oRetorno:ADD(Self:Elements(nCount))
75         EndIf
76     Next
77
78 Return oRetorno
79
80 //-----
81 /*/{Protheus.doc} Enqueue
82 Adiciona um elemento na fila.
83
84 @param     Exp01 (1 - oObject) - Elemento da fila.
85 @see      Veja tambem a classe STFWACollection
86 @return   Self
87 @todo
88 @obs
89 @sample
90 */
91 //-----
92
93 Method Enqueue(oObject) Class STFWCQueue
94
95     //Adiciona um elemento na colecao
96     AADD(Self:aColecao, oObject)
97
98 Return Nil
99
100 //-----
101 /*/{Protheus.doc} Peek
102 Retorna o ultimo elemento da fila.
103
104 @param

```

```

105 @see      Veja tambem a classe STFWACollection
106 @return   Retorna o primeiro elemento da fila
107 @todo
108 @obs
109 @sample
110 /*/
111 //-----
112
113 Method Peek() Class STFWCQueue
114
115     Local oElement := Nil           //Objeto que sera retornado na funcao
116
117     //Retorna o primeiro elemento da fila
118     oElement := Self:Elements(1)
119
120 Return oElement
121
122 //-----
123 /*/{Protheus.doc} Dequeue
124 Remove o primeiro elemento da fila.
125
126 @param
127 @see      Veja tambem a classe STFWACollection
128 @return
129 @todo
130 @obs
131 @sample
132 /*/
133 //-----
134
135 Method Dequeue() Class STFWCQueue
136
137     //Apaga o primeiro elemento da colecao
138     ADel(Self:aColecao, 1)
139
140     //Redimensiona a colecao
141     ASize(Self:aColecao, ::Count() - 1)
142
143 Return Nil

```


Framework/Collection/STFWCStack.prw

143 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "MSOBJECT.CH"
3
4 User Function STFWCStack ; Return // "dummy" function - Internal Use
5
6 //-----
7 /*{Protheus.doc} STFWCStack
8 Classe responsavel em guardar objetos em uma pilha. (First In Last Out).
9
10 @param
11 @see    Veja tambem a classe STFWACollection
12 @return Self
13 @todo
14 @obs
15 @sample
16 */
17 //-----
18
19 Class STFWCStack From STFWACollection
20
21     Method STFWCStack()                //Metodo construtor
22     Method Clonar()                    //Metodo que ira fazer um clone do objeto LJCStack usando recursividade
23     Method Push(oObject)                //Metodo que ira adicionar um objeto na pilha
24     Method Peek()                       //Metodo que retorna o ultimo elemento da pilha
25     Method Poop()                       //Metodo para remover um objeto da pilha
26
27 EndClass
28
29 //-----
30 /*{Protheus.doc} STFWCStack
31 Construtor da classe STFWCStack
32
33 @param
34 @see    Veja tambem a classe STFWACollection
35 @return Self
36 @todo
37 @obs
38 @sample
39 */
40 //-----
41
42 Method STFWCStack() Class STFWCStack
43
44     _Super:STFWACollection()
45
46 Return Self
47
48 //-----
49 /*{Protheus.doc} Clonar
50 Responsavel em clonar o objeto STFWCStack
51
52 @param

```

```

53 @see    Veja tambem a classe STFWACollection
54 @return  oReturn contendo a quantidade de elementos.
55 @todo
56 @obs
57 @sample
58 */
59 //-----
60
61 Method Clonar() Class STFWCStack
62
63     Local oRetorno := Nil           //Retorno do metodo
64     Local nCount   := 0
65
66     //Instancia o objeto LJCStack (recursividade)
67     oRetorno := STFWCStack():STFWCStack()
68
69     //Atribui ao array do novo objeto um clone do array original
70     For nCount := 1 To ::Count()
71         If Valtype(Self:Elements(nCount)) == "0"
72             oRetorno:ADD(Self:Elements(nCount):Clonar())
73         Else
74             oRetorno:ADD(Self:Elements(nCount))
75         EndIf
76     Next
77
78 Return oRetorno
79
80 //-----
81 /*/{Protheus.doc} Push
82 Adiciona um elemento na pilha.
83
84 @param    Exp01 (2 - oObject) - Elemento da pilha.
85 @see      Veja tambem a classe STFWACollection
86 @return    Self
87 @todo
88 @obs
89 @sample
90 */
91 //-----
92
93 Method Push(oObject) Class STFWCStack
94
95     //Adiciona um elemento na colecao
96     AADD(Self:aColecao, oObject)
97
98 Return Nil
99
100 //-----
101 /*/{Protheus.doc} Peek
102 Retorna o ultimo elemento da pilha.
103
104 @param

```

```

105 @see      Veja tambem a classe STFWACollection
106 @return   Retorna o ultimo elemento da pilha
107 @todo
108 @obs
109 @sample
110 /*/
111 //-----
112
113 Method Peek() Class STFWCStack
114
115     Local oElement := Nil           //Objeto que sera retornado na funcao
116
117     //Retorna o ultimo elemento da pilha
118     oElement := Self:Elements():Count()
119
120 Return oElement
121
122 //-----
123 /*/{Protheus.doc} Poop
124 Remove um elemento da pilha.
125
126 @param
127 @see      Veja tambem a classe STFWACollection
128 @return   Self
129 @todo
130 @obs
131 @sample
132 /*/
133 //-----
134
135 Method Poop() Class STFWCStack
136
137     //Apaga o elemento da colecao
138     ADel(Self:aColecao, Self:Count())
139
140     //Redimensiona a colecao
141     ASize(Self:aColecao, Self:Count() - 1)
142
143 Return Nil

```

Framework/Dec2Hex/STFDex2Hex.prw

14 linhas após remoção de metadados de autoria/data

```
1 #INCLUDE "PROTHEUS.CH"
2
3 //-----
4 /*{Protheus.doc} STFDex2Hex
5 Converte um numero decimal ate' 255 para hexadecimal .
6 @return      cResult - Resultado da conversao
7 @obs         STFDex2Hex
8 */
9 //-----
10 Function STFDex2Hex(nVal)
11 Local cString:= "0123456789ABCDEF"
12 Local cRet   := Substr(cString,Int(nVal/16)+1,1)+Substr(cString,nVal-(Int(nVal/16)*16)+1,1)
13
14 Return cRet
```

Framework/DefineFont/STFDefineFont.prw

18 linhas após remoção de metadados de autoria/data

```
1 #Include 'Protheus.ch'
2
3 Function STFDefFont()
4
5 Local aFontes := {}
6 Local oFonte1
7 Local oFonte2
8 Local oFonte3
9
10 DEFINE FONT oFonte1 NAME "Arial" BOLD
11 DEFINE FONT oFonte2 NAME "Arial" SIZE 08,17 BOLD
12 DEFINE FONT oFonte3 NAME "Courier New"
13
14 aAdd(aFontes, oFonte1)
15 aAdd(aFontes, oFonte2)
16 aAdd(aFontes, oFonte3)
17
18 Return aFontes
```

Framework/Deploy/STBPAREX.prw

39 linhas após remoção de metadados de autoria/data

```
1 #Include 'Protheus.ch'
2
3 Function STBPAREX()
4
5 Local aParDefault := {} // Array com Parâmetros default que não serão atualizados
6 Local aPtParam := {} // Array com parâmetros definidos via ponto de entrada
7 Local nX := 0
8
9 Aadd(aParDefault, "MV_LJAMBIE")
10
11 Aadd(aParDefault, "MV_LJILLIP")
12
13 Aadd(aParDefault, "MV_LJILLP0")
14
15 Aadd(aParDefault, "MV_LJILLEN")
16
17 Aadd(aParDefault, "MV_LJILLCO")
18
19 Aadd(aParDefault, "MV_LJILLBR")
20
21 Aadd(aParDefault, "MV_LJILLIM")
22
23 Aadd(aParDefault, "MV_LJILLDO")
24
25 Aadd(aParDefault, "MV_LJILLAC")
26
27 Aadd(aParDefault, "MV_LJILLKT")
28
29 If ExistBlock("STParamDeploy")
30     aPtParam := ExecBlock("STParamDeploy")
31 EndIf
32
33 If Len(aPtParam) > 0
34     For nX := 1 To Len(aPtParam)
35         Aadd(aParDefault, aPtParam[nX])
36     Next nX
37 EndIf
38
39 Return aParDefault
```

Framework/Estrutura/POSDesktop.prw

716 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "protoheus.ch"
2 #INCLUDE "parmttype.ch"
3 #INCLUDE "poscss.ch"
4
5 #DEFINE FONT_NAME 1
6 #DEFINE FONT_SIZE 2
7 #DEFINE FONT_BOLD 3
8 #DEFINE FONT_ITALIC 4
9 #DEFINE FONT_UNDERLINE 5
10
11 STATIC aFontButton := { "Arial", 12, .T., .F., .F. }
12 STATIC aFontInfo := { "Arial", 12, .F., .F., .F. }
13 STATIC aFontMenu := { "Arial", 13, .T., .F., .F. }
14 STATIC aFontAlert := { "Arial", IIF(FWIsMobile(),14,32), .T., .F., .F. }
15 STATIC __ISFLY01 := FindFunction('ISFly01') .AND. ISFly01()
16
17 //-----
18 /*{Protoheus.doc} POSDesktop
19 Objetivo desta classe é preparar a interface para a utilização
20 do novo POS
21
22 */
23 //-----
24 CLASS POSDesktop
25
26     DATA cBG
27
28     DATA oMsgBarTop
29     DATA oMsgBarBottom
30     DATA oDesktop
31     DATA oOwner
32
33     METHOD New() CONSTRUCTOR
34
35     METHOD Activate()
36     METHOD Deactivate()
37
38     METHOD AddTLMenuItem()
39
40     METHOD GetDesktop()
41
42     METHOD SetBAlert()
43     METHOD SetBGImage()
44     METHOD SetBInfo( cInfo )
45     METHOD SetTInfo( cInfo )
46     METHOD SetTRMenu()
47     METHOD SetTLMenu()
48     METHOD GetPanelLogoBottom()
49
50 ENDCLASS
51
52 //-----

```

```

53 /*/{Protheus.doc} New
54 Construtor
55
56 @param oDlg Dialog container onde será construída a interface
57
58 /*/
59 //-----
60 METHOD New( oDlg ) CLASS POSDesktop
61     PARAMTYPE 00 VAR oDlg AS OBJECT
62
63     ::oOwner := oDlg
64     ::oMsgBarTop := POSMsgBarTop():New( oDlg )
65     ::oMsgBarBottom := POSMsgBarBottom():New( oDlg )
66 RETURN
67
68 //-----
69 /*/{Protheus.doc} Activate
70 Ativa a classe, este método constrói a interface
71
72 /*/
73 //-----
74 METHOD Activate() CLASS POSDesktop
75
76     Local cBG := If( ::cBG <> Nil, ::cBG, "pos_bg_geral.png" )
77
78     Local oBG
79
80     Local oOwner := ::oOwner
81
82     ::oMsgBarTop:Activate()
83     ::oMsgBarBottom:Activate()
84
85     If ::cBG == Nil
86         @ 000,000 BITMAP oBG RESOURCE "x.png" NOBORDER SIZE 000,020 OF oOwner ADJUST PIXEL
87         oBG:Align := CONTROL_ALIGN_ALLCLIENT
88         oBG:SetCSS( POSCSS( GetClassName(oBG), CSS_BG ) )
89         oOwner := oBG
90     EndIf
91
92     @ 000,000 BITMAP ::oDesktop RESOURCE cBG NOBORDER SIZE 000,020 OF oOwner ADJUST PIXEL
93     ::oDesktop:Align := CONTROL_ALIGN_ALLCLIENT
94     ::oDesktop:ReadClientCoors(.T.,.T.)
95     ::oDesktop:SetCSS( FWCSSVerify( GetClassName(::oDesktop), "Q3Frame", "Q3Frame { background-color: none; }" ) )
96
97 RETURN
98
99 //-----
100 /*/{Protheus.doc} Deactivate
101 Desativa a classe
102
103 /*/
104 //-----

```



```

105 METHOD Deactivate() CLASS POSDesktop
106
107     ::oMsgBarTop:Deactivate()
108     ::oMsgBarBottom:Deactivate()
109
110     FreeObj(::oDesktop)
111
112 RETURN
113
114 //-----
115 /*{Protheus.doc} AddTLMenuItem
116 Adiciona um item de menu popup no botão superior a esquerda
117
118 @param cDescr String de descrição do botão
119 @param bAction Bloco de ação do botão
120
121 */
122 //-----
123 METHOD AddTLMenuItem( cDescr, bAction) CLASS POSDesktop
124     PARAMTYPE 00 VAR cDescr AS CHARACTER
125     PARAMTYPE 01 VAR bAction AS BLOCK
126
127     ::oMsgBarTop:AddLMenuItem( cDescr, bAction )
128 RETURN
129
130 //-----
131 /*{Protheus.doc} GetDesktop
132 Retorna o painel disponível para a construção da area de
133 trabalho do POS
134
135 @return oDesktop Container para a construção do POS
136
137 */
138 //-----
139 METHOD GetDesktop() CLASS POSDesktop
140 RETURN ::oDesktop
141
142 //-----
143 /*{Protheus.doc} SetBAlert
144 Seta alerta na barra inferior da interface
145
146 @param cAlert String com o alerta a ser exibido
147
148 */
149 //-----
150 METHOD SetBAlert( cAlert ) CLASS POSDesktop
151     PARAMTYPE 00 VAR cAlert AS CHARACTER
152
153     ::oMsgBarBottom:SetAlert( cAlert )
154 RETURN
155
156 //-----

```

```

157 /*/{Protheus.doc} SetBGImage
158 Seta imagem de fundo da interface do POS
159
160 @param cBG Nome da imagem que deve estar compilada no repositório
161
162 */
163 //-----
164 METHOD SetBGImage( cBG ) CLASS POSDesktop
165     PARAMTYPE 00 VAR cBG AS CHARACTER
166
167     ::cBG := cBG
168 RETURN
169
170 //-----
171 /*/{Protheus.doc} SetBInfo
172 Seta informação a ser exibida na barra inferior da interface
173
174 @param cInfo String com informação a ser exibida
175
176 */
177 //-----
178 METHOD SetBInfo( cInfo ) CLASS POSDesktop
179     ::oMsgBarBottom:SetInfo( cInfo )
180 RETURN
181
182 //-----
183 /*/{Protheus.doc} SetTInfo
184 Seta informação a ser exibida na barra superior da interface
185
186 @param cInfo String com informação a ser exibida
187
188 */
189 //-----
190 METHOD SetTInfo( cInfo ) CLASS POSDesktop
191     ::oMsgBarTop:SetInfo( cInfo )
192 RETURN
193
194 //-----
195 /*/{Protheus.doc} SetTLMenu
196 Seta descrição e atalho do menu superior a esquerda
197
198 @param cDescr Descrição do menu superior a esquerda
199 @param nHotKey Numérico com o código da tecla de atalho do menu superior a esquerda
200
201 */
202 //-----
203 METHOD SetTLMenu( cDescr, nHotKey ) CLASS POSDesktop
204     ::oMsgBarTop:SetLMenu( cDescr, nHotKey )
205 RETURN
206
207 //-----
208 /*/{Protheus.doc} SetTRMenu

```

```

209 Seta descrição e atalho do menu superior a direita
210
211 @param cDescr Descrição do menu superior a diteita
212 @param nHotKey Numérico com o código da tecla de atalho do menu superior a direita
213 @param bAction Bloco com a ação a ser executada no menu superior a direita
214
215 /*/
216 //-----
217 METHOD SetTRMenu( cDescr, nHotKey, bAction ) CLASS POSDesktop
218     ::oMsgBarTop:SetRMenu( cDescr, nHotKey, bAction )
219 RETURN
220
221 //-----
222 /*/{Protheus.doc} GetPanelLogoBottom
223 Retorna o panel da barra inferior
224
225 @return oPanel barra inferior
226
227 /*/
228 //-----
229 METHOD GetPanelLogoBottom() CLASS POSDesktop
230 RETURN Self:oMsgBarBottom:GetPanelLogoBottom()
231
232
233 //-----
234 /*/{Protheus.doc} POSMsgBarTop
235 Classe responsável pela montagem da barra superior
236
237 /*/
238 //-----
239 CLASS POSMsgBarTop
240
241     DATA aLOptions
242
243     DATA bRAction
244
245     DATA cInfo
246     DATA cRMDescr
247     DATA cLMDescr
248
249     DATA nLMHotKey
250     DATA nRMHotKey
251
252     DATA oBar
253     DATA oInfo
254     DATA oLBtn
255     DATA oOwner
256     DATA oRBtn
257
258     METHOD New()
259
260     METHOD Activate()

```

```

261     METHOD Deactivate()
262
263     METHOD AddMenuItem()
264     METHOD MenuLAction()
265
266     METHOD SetInfo()
267     METHOD SetLMenu()
268     METHOD SetRMenu()
269
270
271 ENDCLASS
272
273 //-----
274 /*{Protheus.doc} New
275 Construtor
276
277 @param Container para a construção da barra
278
279 */
280 //-----
281 METHOD New( oOwner ) CLASS POSMsgBarTop
282
283     ::aLOptions := {}
284
285     ::cInfo := ""
286
287     ::oOwner := oOwner
288
289 RETURN
290
291 //-----
292 /*{Protheus.doc} Activate
293 Método para a ativação da classe
294
295 */
296 //-----
297 METHOD Activate() CLASS POSMsgBarTop
298
299     Local cTitle
300
301     Local nHeight
302     Local nWidth
303     Local nX
304     Local oFontSize      := FwFontSize():New()
305     Local oMenu
306     Local oMenuItem
307     local nLogoWidth      := 52
308     Local nLogoHeight     := 21.6
309     Local nLogoMargin     := 15
310     Local lClientHtml     := .F.
311     Local cVerWebApp      := ""
312

```

```

313 //Verifica se o SmartClient é via WebApp ou não.
314 GetRemoteType(@cVerWebApp)
315 lClientHtml:= 'HTML' $ UPPER(cVerWebApp)
316 Conout("SmartClient-> " + cVerWebApp)
317
318
319 @ 000,000 BITMAP ::oBar RESOURCE "x.png" NOBORDER SIZE 000,020 OF ::oOwner ADJUST PIXEL
320 ::oBar:Align := CONTROL_ALIGN_TOP
321 ::oBar:SetCSS( POSCSS( GetClassName(::oBar), CSS_BAR_TOP ) )
322 ::oBar:ReadClientCoors(.T.,.T.)
323
324 cTitle := ::cLMDescr + IIF(!FWIsMobile()," (" + GetKey(::nLMHotKey) + ")", '')
325 nWidth := oFontSize:getTextWidth( cTitle, aFontButton[FONT_NAME], aFontButton[FONT_SIZE], aFontButton[FONT_BOLD], aFontButton[FONT_ITALIC], aFontButton[FONT_UNDERLINE]
)/2
326
327 If FwIsMobile()
328     nWidth += Val(FWResAdjSize(26)) //adiciona a largura do icone de menu
329 EndIf
330
331 @ 000, 000 BUTTON ::oLBtn PROMPT cTitle SIZE nWidth+020,000 ACTION ::bRAction/*{|| MsgAlert("Teste") }*/ OF ::oBar PIXEL
332 ::oLBtn:Align := CONTROL_ALIGN_LEFT
333 ::oLBtn:SetCSS( POSCSS( GetClassName(::oLBtn), CSS_BAR_BUTTON, .T. ) )
334 Self:oLBtn:LCANGOTFOCUS := .F.
335
336
337 //-----
338 // Verifica qual o maior titulo do submenu
339 //-----
340 For nX := 1 To Len(::aLOptions)
341     nWidth := Max( nWidth, oFontSize:getTextWidth( OemToAnsi(::aLOptions[nX][1]), aFontMenu[FONT_NAME], aFontMenu[FONT_SIZE], aFontMenu[FONT_BOLD],
aFontMenu[FONT_ITALIC], aFontMenu[FONT_UNDERLINE] ) )
342 Next nX
343 nWidth += 100 // adiciona margem para icones
344
345 oMenu := TMenu():New(0,0,0,0,.T.)
346
347 //-----
348 // Cria submenus
349 //-----
350 For nX := 1 To Len(::aLOptions)
351     oMenuItem := TMenuItem():New(,OemToAnsi(::aLOptions[nX][1]+IIF(lClientHtml, "", Space(300))),OemToAnsi(::aLOptions[nX][1]+IIF(lClientHtml,
"",Space(300))),,,,,,,,,.T.)
352     oMenuItem:bAction := ::aLOptions[nX][2]
353
354     oMenu:Add( oMenuItem )
355 Next nX
356 ::oLBtn:SetPopupMenu(oMenu)
357 SetKey( ::nLMHotKey, {|| oMenu:Activate(::oLBtn:nLeft, ::oLBtn:nTop+::oLBtn:nHeight) } )
358 //-----
359 // Atribui css ao menu
360 //-----
361 oMenu:SetCSS( POSCSS( GetClassName(oMenu), CSS_BAR_MENU, nWidth ) )

```

```

362
363   If FWIsMobile() .And. __ISFLY01
364       //divide por 4 pq precisa dividir por 2 pela diferença de ponto e pixel e dividir por 2 pela metade da altura, para centralizar
365       ::oRBtn := TSVG():New((::oBar:nheight/4 - nLogoHeight/2 ),::oBar:nWidth/2 - (nLogoWidth +
nLogoMargin),::oBar,nLogoWidth,nLogoHeight,GETApoRes("fwskin_logo_flyrede.svg"),.T.)
366       ::oRBtn:SetCSS( FWCSSVerify( GetClassName(::oRBtn), "Q3Frame", "Q3Frame { background-color: transparent; border:none; }" ) )
367
368   Else
369       cTitle := ::cRMDescr + IIF(!FWIsMobile())," (" + GetKey(::nRMHotKey) + ")",'' )
370       nWidth := oFontSize:getTextWidth( cTitle, aFontButton[FONT_NAME], aFontButton[FONT_SIZE], aFontButton[FONT_BOLD], aFontButton[FONT_ITALIC],
aFontButton[FONT_UNDERLINE] )/2
371       @ 000, 000 BUTTON ::oRBtn PROMPT cTitle SIZE nWidth+020,000 OF ::oBar PIXEL
372       ::oRBtn:bAction := ::bRAction
373       ::oRBtn:Align := CONTROL_ALIGN_RIGHT
374       ::oRBtn:SetCSS( POSCSS( GetClassName(::oRBtn), CSS_BAR_BUTTON ) )
375       Self:oRBtn:LCANGOTFOCUS := .F.
376
377       SetKey( ::nRMHotKey, ::bRAction )
378   EndIf
379
380   If !FWIsMobile()
381       cTitle := ::cInfo+Space(10)
382       nWidth := ::oBar:nWidth-::oLBtn:nWidth-::oRBtn:nWidth
383       nHeight := oFontSize:getTextHeight( cTitle, aFontInfo[FONT_NAME], aFontInfo[FONT_SIZE], aFontInfo[FONT_BOLD], aFontInfo[FONT_ITALIC], aFontInfo[FONT_UNDERLINE] )
384       @ ( ::oBar:nHeight-nHeight)/2/2, ( ::oBar:nWidth-::oRBtn:nWidth-nWidth)/2 SAY ::oInfo PROMPT cTitle RIGHT OF ::oBar PIXEL
385       ::oInfo:Align:= CONTROL_ALIGN_RIGHT
386       ::oInfo:nWidth := nWidth
387       ::oInfo:nHeight := nHeight
388       ::oInfo:SetCSS( POSCSS( GetClassName(::oInfo), CSS_BAR_INFO ) )
389   EndIf
390
391   RETURN
392
393   Method MenuLAction(o, cID) Class POSMsgBarTop
394   Local nX := Val(cID)
395   Eval(::aLOptions[nX][2])
396   Return
397
398   //-----
399   /*{Protheus.doc} Deactivate
400   Método para a desativação da classe
401
402   */
403   //-----
404   METHOD Deactivate() CLASS POSMsgBarTop
405
406   aSize( ::aLOptions, 0 )
407   ::aLOptions := Nil
408
409   ::bRAction := Nil
410
411   FreeObj(::oBar)

```

```

412     FreeObj(::oInfo)
413     FreeObj(::oLBtn)
414     FreeObj(::oRBtn)
415
416 RETURN
417
418 //-----
419 /*{Protheus.doc} AddLMenuItem
420 Método para adicionar um item ao menu superior a esquerda
421
422 @param cDescr String com a descrição do item
423 @param bAction Ação do item do menu
424
425 */
426 //-----
427 METHOD AddLMenuItem( cDescr, bAction) CLASS POSMsgBarTop
428     aAdd( ::aLOptions, { cDescr, bAction } )
429 RETURN
430
431 //-----
432 /*{Protheus.doc} SetInfo
433 Seta informação a ser exibida na barra superior da interface
434
435 @param cInfo String com informação a ser exibida
436
437 */
438 //-----
439 METHOD SetInfo(cInfo) CLASS POSMsgBarTop
440     Local nWidth
441     Local nHeight
442
443     Local oFontSize := FWFontSize():New()
444
445     ::cInfo := cInfo
446
447     If ::oInfo <> Nil
448         nWidth := oFontSize:getTextWidth( cInfo, aFontInfo[FONT_NAME], aFontInfo[FONT_SIZE], aFontInfo[FONT_BOLD], aFontInfo[FONT_ITALIC], aFontInfo[FONT_UNDERLINE] )
449         nHeight := oFontSize:getTextHeight( cInfo, aFontInfo[FONT_NAME], aFontInfo[FONT_SIZE], aFontInfo[FONT_BOLD], aFontInfo[FONT_ITALIC], aFontInfo[FONT_UNDERLINE] )
450
451         ::oInfo:cCaption := cInfo
452
453         ::oInfo:nWidth := nWidth
454         ::oInfo:nHeight := nHeight
455         ::oInfo:Refresh()
456     EndIf
457 RETURN
458
459 //-----
460 /*{Protheus.doc} SetLMenu
461 Seta descrição e tecla de atalho do menu superior a esquerda
462
463 @param cDescr String com a descrição do menu a esquerda

```

```

464 @param nHotKey Numérico com o código da tecla de atalho do menu a esquerda
465
466 /*/
467 //-----
468 METHOD SetLMenu( cDescr, nHotKey ) CLASS POSMsgBarTop
469     ::cLMDescr := cDescr
470     ::nLMHotKey := nHotKey
471 RETURN
472
473 //-----
474 /*/{Protheus.doc} SetRMenu
475 Seta descrição, tecla de atalho e ação do menu superior a direita
476
477 @param cDescr String com a descrição do menu a direita
478 @param nHotKey Numérico com o código da tecla de atalho do menu a direita
479 @param bAction Bloco de ação do menu superior a direita
480
481 /*/
482 //-----
483 METHOD SetRMenu( cDescr, nHotKey, bAction ) CLASS POSMsgBarTop
484     ::bRAction := bAction
485     ::cRMDescr := cDescr
486     ::nRMHotKey := nHotKey
487 RETURN
488
489 //-----
490 /*/{Protheus.doc} POSMsgBarBottom
491 Classe responsável pela criação da barra inferior da interface
492
493 /*/
494 //-----
495 CLASS POSMsgBarBottom
496
497     DATA cInfo
498     DATA cRMDescr
499
500     DATA oAlert
501     DATA oBar
502     DATA oInfo
503     DATA oOwner
504     DATA oPanelLogo
505
506     METHOD New()
507
508     METHOD Activate()
509     METHOD Deactivate()
510
511     METHOD SetInfo()
512     METHOD SetAlert()
513
514     METHOD GetPanelLogoBottom()
515

```



```

516 ENDCLASS
517
518 //-----
519 /*/{Protheus.doc} New
520 Construtor
521
522 @param oOwner Container para a construção da barra
523
524 /*/
525 //-----
526 METHOD New( oOwner ) CLASS POSMsgBarBottom
527
528     ::cInfo := ""
529
530     ::oOwner := oOwner
531
532 RETURN
533
534 //-----
535 /*/{Protheus.doc} Activate
536 Método responsável por ativar a classe
537
538 /*/
539 //-----
540 METHOD Activate() CLASS POSMsgBarBottom
541
542     Local cTitle
543
544     Local nWidth
545     Local nHeight
546
547     Local oFont
548     Local oFontSize := FWFontSize():New()
549     Local oLogo
550
551
552     @ 000,000 BITMAP ::oBar RESOURCE "x.png" NOBORDER SIZE 000,030 OF ::oOwner ADJUST PIXEL
553     ::oBar:Align := CONTROL_ALIGN_BOTTOM
554     ::oBar:SetCSS( POSCSS( GetClassName(::oBar), CSS_BAR_BOTTOM ) )
555     ::oBar:ReadClientCoors(.T.,.T.)
556
557
558     @ 000,000 BITMAP ::oPanelLogo RESOURCE "x.png" NOBORDER SIZE 066,000 OF ::oBar ADJUST PIXEL
559     ::oPanelLogo:Align := CONTROL_ALIGN_RIGHT
560     ::oPanelLogo:SetCSS( POSCSS( GetClassName(::oPanelLogo), CSS_BAR_LOGO ) )
561
562     If !FWIsMobile()
563         @ 003,013 BITMAP oLogo RESOURCE FwGetImgCss('pos_logo_hor_branco') NOBORDER SIZE 040.37,015 OF ::oPanelLogo ADJUST PIXEL
564     EndIf
565
566     cTitle := ::cInfo+Space(10)
567     nWidth := oFontSize:getTextWidth( cTitle, aFontInfo[FONT_NAME], aFontInfo[FONT_SIZE], aFontInfo[FONT_BOLD], aFontInfo[FONT_ITALIC], aFontInfo[FONT_UNDERLINE] )

```

```

568     nHeight := oFontSize:getTextHeight( cTitle, aFontInfo[FONT_NAME], aFontInfo[FONT_SIZE], aFontInfo[FONT_BOLD], aFontInfo[FONT_ITALIC], aFontInfo[FONT_UNDERLINE] )
569     @ ( ::oBar:nHeight-nHeight)/2/2, ( ::oBar:nWidth-(IIF( ::oPanelLogo <> nil, ::oPanelLogo:nWidth, 0)) - nWidth)/2 SAY ::oInfo PROMPT cTitle OF ::oBar PIXEL
570     ::oInfo:nWidth := nWidth
571     ::oInfo:nHeight := nHeight
572     ::oInfo:Align := CONTROL_ALIGN_RIGHT
573     ::oInfo:SetCSS( POSSCSS( GetClassName( ::oInfo ), CSS_BAR_INFO ) )
574
575 RETURN
576
577 //-----
578 /*/{Protheus.doc} Deactivate
579 Método responsável por desativar a classe
580
581 */
582 //-----
583 METHOD Deactivate() CLASS POSMsgBarBottom
584
585     FreeObj( ::oAlert )
586     FreeObj( ::oBar )
587     FreeObj( ::oInfo )
588
589 RETURN
590
591 //-----
592 /*/{Protheus.doc} SetInfo
593 Método responsável por atribuir as informações que serão exibidas na barra inferior
594
595 @param cInfo String com as informações que serão exibidas na barra inferior
596
597 */
598 //-----
599 METHOD SetInfo( cInfo ) CLASS POSMsgBarBottom
600     Local nWidth
601     Local nHeight
602
603     Local oFontSize := FWFontSize():New()
604
605     ::cInfo := cInfo
606
607     If ::oInfo <> Nil
608         nWidth := oFontSize:getTextWidth( cInfo, aFontInfo[FONT_NAME], aFontInfo[FONT_SIZE], aFontInfo[FONT_BOLD], aFontInfo[FONT_ITALIC], aFontInfo[FONT_UNDERLINE] )
609         nHeight := oFontSize:getTextHeight( cInfo, aFontInfo[FONT_NAME], aFontInfo[FONT_SIZE], aFontInfo[FONT_BOLD], aFontInfo[FONT_ITALIC], aFontInfo[FONT_UNDERLINE] )
610
611         ::oInfo:cCaption := cInfo
612         ::oInfo:nWidth := nWidth
613         ::oInfo:nHeight := nHeight
614         ::oInfo:Refresh()
615     EndIf
616 RETURN
617
618 //-----
619 /*/{Protheus.doc} SetAlert

```

```

620 Método responsável por setar o alerta na barra inferior
621
622 @param cAlert String com o alerta a ser exibido
623
624 */
625 //-----
626 METHOD SetAlert( cAlert ) CLASS POSMsgBarBottom
627
628     Local cTitle := ""
629
630     Local nWidth
631     Local nHeight
632
633     Local oFontSize := FWFontSize():New()
634
635     cTitle := cAlert+Space(10)
636     nWidth := oFontSize:getTextWidth( cTitle, aFontAlert[FONT_NAME], aFontAlert[FONT_SIZE], aFontAlert[FONT_BOLD], aFontAlert[FONT_ITALIC], aFontAlert[FONT_UNDERLINE] )
637     nHeight := oFontSize:getTextHeight( cTitle, aFontAlert[FONT_NAME], aFontAlert[FONT_SIZE], aFontAlert[FONT_BOLD], aFontAlert[FONT_ITALIC], aFontAlert[FONT_UNDERLINE] )
638     If ::oAlert == Nil
639         @ ( ::oBar:nHeight-nHeight)/2/2, 10 SAY ::oAlert PROMPT cTitle OF ::oBar PIXEL
640         ::oAlert:SetCSS( POSCSS( GetClassName(::oAlert), CSS_BAR_ALERT) )
641         //::oAlert:Align := CONTROL_ALIGN_ALLCLIENT
642     Else
643         ::oAlert:cCaption := cTitle
644     EndIf
645     ::oAlert:nWidth := nWidth
646     ::oAlert:nHeight := nHeight
647     ::oAlert:Refresh()
648
649 RETURN
650
651 //-----
652 */{Protheus.doc} GetPanelLogoBottom
653 Retorna o panel da barra inferior
654
655 @return oPanel barra inferior
656
657 */
658 //-----
659 METHOD GetPanelLogoBottom() CLASS POSMsgBarBottom
660 RETURN Self:oPanelLogo
661
662 //-----
663 */{Protheus.doc} GetKey
664 Função responsável por retornar a descrição da tecla de atalho
665
666 @param Numérico com o código ASC da tecla de atalho
667
668 @return String com a descrição da tecla de atalho passada por parametro
669
670 */
671 //-----

```

```
672 Static Function GetKey( nKey )
673
674     Local cRet
675
676     If FWIsMobile()
677         Return ''
678     EndIf
679
680     Do Case
681         Case nKey == VK_F1
682             cRet := "F1"
683         Case nKey == VK_F2
684             cRet := "F2"
685         Case nKey == VK_F3
686             cRet := "F3"
687         Case nKey == VK_F4
688             cRet := "F4"
689         Case nKey == VK_F5
690             cRet := "F5"
691         Case nKey == VK_F6
692             cRet := "F6"
693         Case nKey == VK_F7
694             cRet := "F7"
695         Case nKey == VK_F8
696             cRet := "F8"
697         Case nKey == VK_F9
698             cRet := "F9"
699         Case nKey == VK_F10
700             cRet := "F10"
701         Case nKey == VK_F11
702             cRet := "F11"
703         Case nKey == VK_F12
704             cRet := "F12"
705     EndCase
706
707     Return cRet
708
709 //-----
710 /*/{Protheus.doc} __POSDesktop
711 Função para geração de patch
712
713 */
714 //-----
715 Function __POSDesktop()
716     Return
```

Framework/Estrutura/POSTheme.prw

752 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "protheus.ch"
2 #INCLUDE "poscss.ch"
3
4 STATIC __ISFLY01 := FindFunction('ISFly01') .AND. ISFly01()
5 STATIC __aColors := IIF(__ISFLY01, {'FF9300','F37021','F16E01','FF8000','FFA200'}, FWGetMonocromatic( "07334C" ) )
6 STATIC __aRGBColors
7 STATIC __lHTML := FindFunction( "GetRemoteType" ) .And. GetRemoteType() == REMOTE_HTML
8
9 #DEFINE GET_CSS_FOCAL "QLineEdit:Focus { color: #"+__aColors[2]+"; background-color: #FFFFFF; border: 1px solid #"+__aColors[2]+"; }"
10
11 Static lClientHtml := GetCliHTML()
12
13 //-----
14 /*/{Protheus.doc} POSBaseColor
15 Seta a cor base a ser usada pelo gerador de CSS
16
17 @param cBaseColor Cor usada a ser usada como base para o css. Ex.: "FF5476"
18
19 /*/
20 //-----
21 Function POSBaseColor( cBaseColor )
22
23     Local aRet := {}
24
25     If !__ISFLY01
26         aRet := FWGetMonocromatic( cBaseColor )
27
28         __aColors := aRet
29         CalcRgbColor(__aColors)
30     EndIf
31
32 Return
33
34 //Converte para RGB as cores padroes em hexa
35 Static Function CalcRgbColor(aColors)
36 Local aRGBColor := {}
37 Local nX
38
39 If aColors <> nil
40     For nX := 1 to len(aColors)
41
42         AADD(aRGBColor , { Str(HexToDec( Subs(aColors[nX], 1, 2) )), ;
43                             Str(HexToDec( Subs(aColors[nX], 3, 2) )), ;
44                             Str(HexToDec( Subs(aColors[nX], 5, 2) ) ) } )
45
46     Next nX
47 EndIf
48 __aRGBColors := aClone(aRGBColor)
49 aRGBColor := ASize(aRGBColor,0)
50 Return
51
52 Function GetPOSColors()

```

```

53
54 Return __aColors
55
56 //-----
57 /*{Protheus.doc} POSCSS
58 Função responsável por retornar o CSS do componente passado por parâmetro
59
60 @param nType Numérico com o tipo do CSS a ser retornado
61 @param xPar01 Parâmetro opcional utilizado em alguns casos para complemento do CSS
62
63 @return cCSS String com o CSS solicitado
64
65 */
66 //-----
67 Function POSCSS( cAdvPLComp, nType, xPar01 )
68
69     Local cCSS := ""
70
71     Local cBGColor := "FFFFFF"
72
73     Local nPxBrowse := 12 // Pixel para Browse
74     Local nPxNormal := 13 //Pixel para Label Normal(padrao)
75
76
77     If oMainWnd:nClientWidth > 1300 .AND. oMainWnd:nClientWidth < 1600 //ENTRE 1300 E 1600
78         nPxBrowse:= 14
79     Elseif oMainWnd:nClientWidth > 1600 .AND. oMainWnd:nClientWidth < 1900 //ENTRE 1900 E 1600
80         nPxBrowse:= 16
81     Elseif oMainWnd:nClientWidth > 1900 //MAIOR QUE 1900
82         nPxBrowse:= 18
83     Endif
84
85     If IsInCallStack("STIRodape")
86         nPxNormal := 18
87     Endif
88
89     If __aColors == nil
90         return cCSS
91     EndIf
92
93     If __aRGBColors == nil
94         CalcRgbColor(__aColors)
95     EndIf
96
97     Do Case
98     Case nType == CSS_BG
99         If FWIsMobile()
100             cCSS += "Q3Frame { border: none; background-color: #DDDDDD; }"
101         Else
102             cCSS += "Q3Frame { fonte-size:12px; border: none; background-color: #"+__aColors[2]+"; }"
103         EndIf
104         cCSS := FWCSSVerify( cAdvPLComp, "Q3Frame", cCSS )

```

```

105
106     Case nType == CSS_BAR_TOP
107         If FWIsMobile()
108             cCSS += "Q3Frame { border: none; background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #"+__aColors[4]+"", stop: 1 #"+__aColors[3]+""); }"
109         Else
110             cCSS += "Q3Frame { fonte-size:12px ; border: none; background-color: #"+__aColors[3]+""; }"
111         EndIf
112         cCSS := FWCSSVerify( cAdvPLComp, "Q3Frame", cCSS )
113
114     Case nType == CSS_BAR_BOTTOM
115         If FWIsMobile()
116             cCSS += "Q3Frame { border: none; border-top: "+FWResAdjSize(0.5)+"px solid #FFFFFF; background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0
#F3F3F3, stop: 1 #D9D9D9); }"
117         Else
118             cCSS += "Q3Frame {fonte-size:12px; border: none; background-color: #"+__aColors[3]+""; }"
119         EndIf
120
121         cCSS := FWCSSVerify( cAdvPLComp, "Q3Frame", cCSS )
122
123     Case nType == CSS_BAR_ALERT
124         If FWIsMobile()
125             cCSS += "QLabel { font: bold "+FWResAdjSize(14)+"px; color: #666666; }"
126         Else
127             cCSS += "QLabel { font-weight: bold; font-size: 32px; color: #FFFFFF; }"
128         EndIf
129         cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
130
131     Case nType == CSS_BAR_BUTTON
132
133         If FWIsMobile()
134             cCSS += "QPushButton{ font: bold "+FWResAdjSize(13)+"px; color: #FFFFFF; border:none; " +;
135                 "background:qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #"+__aColors[5]+"", stop: 1 #"+__aColors[4]+""); " +;
136                 "outline :none; "
137
138             If xPar01
139                 cCSS += "qproperty-icon:url(rpo:fwskin_menu_hamburguer.svg); qproperty-iconSize: "+FWResAdjSize(26)+"px;"
140                 cCSS += "border-right: "+FWResAdjSize(0.5)+"px solid #"+__aColors[3]+"";"
141             EndIf
142             cCSS += " }"
143
144         Else
145             cCSS += "QPushButton{ font-weight: bold; font-size: 12px; color: #FFFFFF; background: #"+__aColors[2]+""; border: none; }"
146             cCSS += "QPushButton:pressed{ background: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, " +;
147                 "stop: 0.0 #"+__aColors[3]+"", stop: 0.5 #"+__aColors[1]+"", stop: 0.5 #"+__aColors[1]+"", stop: 1.0 #"+__aColors[2]+""); }"
148
149         EndIf
150         cCSSs += "QPushButton::menu-indicator{ image:none; }"
151         cCSS := FWCSSVerify( cAdvPLComp, "QPushButton", cCSS )
152
153     Case nType == CSS_BAR_INFO
154         If FWIsMobile()
155             cCSS += "QLabel { qproperty-alignment:'AlignVCenter | AlignRight'; font: "+FWResAdjSize(12)+"px; color: #666666; padding-right:"+FWResAdjSize(5)+"px; }"

```

```

156     Else
157         cCSS += "QLabel { font-size: 12px; color: #FFFFFF; }"
158     EndIf
159
160     cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
161
162     Case nType == CSS_BAR_LOGO
163         If FWIsMobile()
164             cCSS += "Q3Frame { border: none; background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #F3F3F3, stop: 1 #D9D9D9); }"
165
166         Else
167             cCSS += "Q3Frame { border: none; background-color: #"+__aColors[2]+"; box-shadow: 10px 10px 5px #888888; }"
168         EndIf
169
170         cCSS := FWCSSVerify( cAdvPLComp, "Q3Frame", cCSS )
171
172     Case nType == CSS_BAR_MENU
173         If FWIsMobile()
174
175             cCSS += 'QMenu{ font: bold; font-size: '+FWResAdjSize(13)+'px; font-family: "Arial"; color: #FFFFFF; background-color: #'+__aColors[5]+'; ' + ;
176                 "border-style: solid; border-left-width: "+FWResAdjSize(1)+"px; " + ;
177                 "border-right-width: "+FWResAdjSize(1)+"px; border-bottom-width: "+FWResAdjSize(1)+"px; " + ;
178                 "border-color: #"+__aColors[2]+"; width: "+ FWResAdjSize(Int(xPar01) )+ "px; }"
179             cCSS += "QMenu::item{ padding: "+FWResAdjSize(15)+"px; padding-left: "+FWResAdjSize(25)+"px; border-style: solid; " + ;
180                 "border-bottom-width: "+FWResAdjSize(1)+"px; border-color: #"+__aColors[2]+"; }"
181
182         Else
183             cCSS += 'QMenu{ font-weight: bold; font-size: 13px; font-family: "Arial"; color: #FFFFFF; background-color: #'+__aColors[2]+'; ' + ;
184                 "border-style: solid; border-left-width: 1px; " + ;
185                 "border-right-width: 1px; border-bottom-width: 1px; " + ;
186                 "border-color: #"+__aColors[3]+"; width: "+AllTrim(Str(xPar01))+ "px; }"
187             cCSS += "QMenu::item{ padding: 10px; padding-left: 25px; border-style: solid; " + ;
188                 "border-bottom-width: 1px; border-color: #"+__aColors[3]+"; }"
189             cCSS += "QMenu::item:selected{ background-color: #"+__aColors[3]+";}"
190         EndIf
191         cCSS := FWCSSVerify( cAdvPLComp, "QMenu", cCSS )
192
193     Case nType == CSS_BTN_FOCAL
194         If FWIsMobile()
195             cCSS += "QPushButton{ font-size: "+FWResAdjSize(13)+"px; font: bold large; padding: "+FWResAdjSize(4)+"px; color: #FFFFFF; " + ;
196                 "outline :none; "+;
197                 "background:qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #"+__aColors[1]+" , stop: 1 #"+__aColors[3]+"); " + ;
198                 "border: "+FWResAdjSize(1)+"px solid #"+__aColors[2]+"; border-radius: "+FWResAdjSize(6)+"px; }"
199             cCSS += "QPushButton:pressed { padding: "+FWResAdjSize(4)+"px; color: #FFFFFF; " + ;
200                 "background:qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #"+__aColors[3]+" , stop: 1 #"+__aColors[1]+"); " + ;
201                 "border: 1px solid #"+__aColors[2]+"; border-radius: 6px; }"
202             cCSS += "QPushButton:disabled{background:qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 rgba("+__aRGBColors[1][1]+
203 ", "+__aRGBColors[1][2]+", "+__aRGBColors[1][3]+", 50%), stop: 1 rgba("+__aRGBColors[3][1]+", "+__aRGBColors[3][2]+", "+__aRGBColors[3][3]+", 50%)); " + ;
204                 " border: 1px solid rgba("+__aRGBColors[2][1]+", "+__aRGBColors[2][2]+", "+__aRGBColors[2][3]+50%); color: #FFFFFF; }"
205         Else
206             cCSS += "QPushButton{ font-size: 13px; padding: 6px; background: #"+__aColors[4]+"; " + ;
207                 "border: 1px solid #"+__aColors[4]+"; border-radius: 6px; " + ;

```



```

207         "background-image: linear-gradient(180deg, #3dafcc 0%,#0d9cbf 100%); "
208     If !_lHTML
209         cCSS += "color: #FFFFFF;"
210     EndIf
211
212     cCSS += "}"
213 EndIf
214 cCSS := FWCSSVerify( cAdvPLComp, "QPushButton", cCSS )
215
216 Case nType == CSS_BTN_NORMAL
217     If FWIsMobile()
218         cCSS += "QPushButton{ font-size: "+FWResAdjSize(12)+"px; font: bold large; padding: "+FWResAdjSize(4)+"px; color: #777777; " + ;
219             "outline :none; "+;
220             "background:qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #FFFFFF, stop: 1 #E9EBF1); " + ;
221             "border: "+FWResAdjSize(1)+"px solid #BBBBBB; border-radius: "+FWResAdjSize(6)+"px; }"
222         cCSS += "QPushButton:pressed { font: bold large; padding: "+FWResAdjSize(4)+"px; color: #777777; " + ;
223             "background:qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #E9EBF1, stop: 1 #FFFFFF); " + ;
224             "border: "+FWResAdjSize(1)+"px solid #BBBBBB; border-radius: "+FWResAdjSize(6)+"px; }"
225         cCSS += "QPushButton:disabled{background:qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 rgba(255,255,255,50%), stop: 1 rgba(233,235,241,50%));" + ;
226             " border: "+FWResAdjSize(1)+"px solid rgba(187,187,187,50%); color: rgba(119,119,119,50%); }"
227
228     Else
229
230         cCSS += "QPushButton {font-weight: bold;font-size: 12px ; " + ;
231             "padding: 4px; color: #858585; background: #FFFFFF; outline :none; " + ;
232             " border: 1px solid #D4D4D4; border-bottom-color: #A4A4A6; border-radius: 6px; }"
233         cCSS += "QPushButton:pressed { color: #FFFFFF; " + "background-image: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, " + ;
234             "stop: 0.0 #757575, stop: 0.6 #878787, stop: 0.7 #878787, stop: 1.0 #909090); " + ;
235             "border: 1px solid #787878; }"
236         cCSS += "QPushButton:disabled{ color: #BCBCBC; }"
237
238     Endif
239     cCSS := FWCSSVerify( cAdvPLComp, "QPushButton", cCSS )
240
241 Case nType == CSS_BTN_ATIVO
242     If FWIsMobile()
243         cCSS += "QPushButton{ font-size: "+FWResAdjSize(12)+"px; font: bold large; padding: "+FWResAdjSize(6)+"px; color: #FFFFFF; " + ;
244             "background-color: #757575; " + ;
245             "outline :none; "+;
246             "border: "+FWResAdjSize(1)+"px solid #787878; border-bottom-color: #A4A4A6; border-radius: "+FWResAdjSize(6)+"px; }"
247
248     Else
249
250         if lClientHtml
251             cCSS += "QPushButton{ font-weight: bold; font-size: 12px; padding: 6px; color: #b5b5b5; "
252             cCSS += "background-image: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, "
253             cCSS += "stop: 0.0 #757575, stop: 0.6 #878787, stop: 0.7 #878787, stop: 1.0 #909090); "
254             cCSS += "border: 2px solid #787878; border-bottom-color: #A4A4A6; border-radius: 6px; }"
255         Else
256             cCSS += "QPushButton{ font-weight: bold; font-size: 12px; padding: 6px; color: #FFFFFF; "
257             cCSS += "background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, "

```

```

259         cCSS += "stop: 0.0 #757575, stop: 0.6 #878787, stop: 0.7 #878787, stop: 1.0 #909090); " + ;
260         "border: 1px solid #787878; border-bottom-color: #A4A4A6; border-radius: 6px; }"
261     Endif
262
263 EndIf
264
265     cCSS := FWCSSTVerify( cAdvPLComp, "QPushButton", cCSS )
266
267 Case nType == CSS_BTN_BUY
268     cCSS += "QPushButton{ font-size: "+FWResAdjSize(14)+"px; font: bold large; padding: "+FWResAdjSize(4)+"px; color: #BD2C08; " + ;
269         "outline :none; "+;
270         "background:qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #FFFF00, stop: 1 #FFC926); " + ;
271         "border: "+FWResAdjSize(1)+"px solid #BBBBBB; }"
272
273
274     cCSS := FWCSSTVerify( cAdvPLComp, "QPushButton", cCSS )
275
276 Case nType == CSS_BTN_LUPA
277     cCSS += PosCSS( cAdvPLComp, CSS_BTN_NORMAL, xPar01)
278     cCSS += "QPushButton{ outline :none; qproperty-icon:url(rpo:fwskin_icon_lookup.svg); qproperty-iconSize: "+FWResAdjSize(30)+"px} "
279
280     cCSS := FWCSSTVerify( cAdvPLComp, "QPushButton", cCSS )
281
282 Case nType == CSS_BTN_BARCODE
283     cCSS += PosCSS( cAdvPLComp, CSS_BTN_NORMAL, xPar01)
284     cCSS += "QPushButton{ outline :none; qproperty-icon:url(rpo:fwskin_barcode.svg); qproperty-iconSize: "+FWResAdjSize(30)+"px} "
285
286
287     cCSS := FWCSSTVerify( cAdvPLComp, "QPushButton", cCSS )
288 Case nType == CSS_BROWSE
289     If FWIsMobile()
290         cCSS += "HeaderView::section { font: bold "+FWResAdjSize(12)+"px; color: #FFFFFF; background-color: #666666; " + ;
291             "border: "+FWResAdjSize(1)+"px solid #666666; height: "+FWResAdjSize(30)+"px; }"
292         cCSS += "QTableView{ font: "+FWResAdjSize(12)+"px; alternate-background-color: #EEEEEE; background: #FFFFFF; color: #444444; border: none; " + ;
293             " selection-background-color: #FFFFFF;}"
294     Else
295         cCSS += "HeaderView::section { font-weight: bold; font-size: 12px; color: #FFFFFF; background-color: #"+__aColors[3]+"; " + ;
296             "border: 1px solid #"+__aColors[3]+"; height: 30px; }"
297         cCSS += "QTableView{ font-size: "+Str(nPxBrowse)+"px; alternate-background-color: #E9E9E9; background: #FFFFFF; color: #000000; border: none; " + ;
298             "selection-background-color: #FFFFFF; gridline-color: white; }"
299     EndIf
300
301 Case nType == CSS_BROWSE_HEADER_LEFT
302     If FWIsMobile()
303         cCSS += "QFrame { margin: 0px 0px 0px "+FWResAdjSize(5)+"px; background-color: #666666; }"
304     Else
305         cCSS += "QFrame { margin: 0px 0px 0px 5px; background-color: #"+__aColors[3]+"; }"
306     EndIf
307
308     cCSS := FWCSSTVerify( cAdvPLComp, "QFrame", cCSS )
309
310 Case nType == CSS_BROWSE_HEADER_RIGHT

```

```

311     If FWIsMobile()
312         cCSS += "QFrame { margin: 0px "+FWResAdjSize(10)+"px 0px 0px; background-color: #666666; }"
313     Else
314         cCSS += "QFrame { margin: 0px "+FWResAdjSize(10)+"px 0px 0px; background-color: #"+__aColors[3]+""; }"
315     EndIf
316
317     cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
318
319 Case nType == CSS_LISTBOX
320     If FWIsMobile()
321
322
323         cCSS += "Q3ListBox{ font: "+FWResAdjSize(16)+"px; padding: 0px; background-color: #FFFFFF; margin: 0px; " + ;
324             "color: #000000; border: "+FWResAdjSize(1)+"px solid #DDDDDD; }"
325         cCSS += "Q3ListBox::item{ padding: "+FWResAdjSize(15)+"px; border-bottom: " + FWResAdjSize(1) + "px solid #DDDDDD; "+;
326             " }"
327
328         cCSS += "Q3ListBox::Item:selected{ background-color: '#"+__aColors[2]+"'; }"
329
330         cCSS += "QScrollBar::handle:vertical{width: "+FWResAdjSize(20)+"px; background: #C2C2C2; border: "+FWResAdjSize(1)+"px solid #C2C2C2; " + ;
331             "border-radius: "+FWResAdjSize(3)+"px;}"
332         cCSS += "QScrollBar:vertical{ background-color: #F2F2F2; border: "+FWResAdjSize(1)+"px solid #FFFFFF; width: "+FWResAdjSize(10)+"px; margin:
"+FWResAdjSize(10)+"px 0px "+FWResAdjSize(10)+"px "+FWResAdjSize(2)+"px;}"
333         cCSS += "QScrollBar::add-line:vertical{padding-left: "+FWResAdjSize(2)+"px; border: 0px; height: "+FWResAdjSize(8)+"px; " + ;
334             "subcontrol-position: bottom; subcontrol-origin: margin;}"
335         cCSS += "QScrollBar::sub-line:vertical{padding-left: "+FWResAdjSize(8)+"px; border: 0px; height: "+FWResAdjSize(8)+"px; " + ;
336             "subcontrol-position: top; subcontrol-origin: margin;}"
337         cCSS += "QScrollBar::up-arrow:vertical { background-image: url(rpo:pos_scroll_arrow_up.png); " + ;
338             "background-repeat: no repeat; width: "+FWResAdjSize(8)+"px; height: "+FWResAdjSize(8)+"px;}"
339         cCSS += "QScrollBar::down-arrow:vertical { background-image: url(rpo:pos_scroll_arrow_down.png); " + ;
340             "background-repeat: no repeat; width: "+FWResAdjSize(8)+"px; height: "+FWResAdjSize(8)+"px;}"
341
342         cCSS += "QWidget{ outline :none; color:'#444444'; font-family: 'Arial'; " + ;
343             " selection-color:'#FFFFFF' ; }"
344
345     Else
346
347         cCSS += "Q3ListBox{ font-size: 18px; padding: 4px; background-color: #FFFFFF; margin: 0px; border-radius: 4px; " + ;
348             "color: #000000; border: 1px solid #9B9B9B; }"
349         cCSS += "QScrollBar::handle:vertical{width: 20px; background: #C2C2C2; border: 1px solid #C2C2C2; " + ;
350             "border-radius: 3px;}"
351         cCSS += "QScrollBar:vertical{ background-color: #F2F2F2; border: 1px solid #FFFFFF; width: 10px; margin: 10px 0px 10px 2px;}"
352         cCSS += "QScrollBar::add-line:vertical{padding-left: 2px; border: 0px; height: 8px; " + ;
353             "subcontrol-position: bottom; subcontrol-origin: margin;}"
354         cCSS += "QScrollBar::sub-line:vertical{padding-left: 2px; border: 0px; height: 8px; " + ;
355             "subcontrol-position: top; subcontrol-origin: margin;}"
356         cCSS += "QScrollBar::up-arrow:vertical { background-image: url(rpo:pos_scroll_arrow_up.png); " + ;
357             "background-repeat: no repeat; width: 8px; height: 8px;}"
358         cCSS += "QScrollBar::down-arrow:vertical { background-image: url(rpo:pos_scroll_arrow_down.png); " + ;
359             "background-repeat: no repeat; width: 8px; height: 8px;}"
360         cCSS += "QWidget{ font-size: 13px; font-family: 'Arial'; }"
361

```

```

362         EndIf
363
364         cCSS := FWCSSVerify( cAdvPLComp, "Q3ListBox", cCSS )
365
366     Case nType == CSS_PANEL_LOGO
367         If lClientHtml
368             cCSS += "QFrame{ margin: 0px; background-color: #"+cBGColor+"; " + ;
369                 "border-radius: "+FWResAdjSize(8)+"px; border: "+FWResAdjSize(1)+"px solid #"+cBGColor+"; padding: "+FWResAdjSize(6)+"px; }"
370         Else
371             cCSS += "QFrame{ margin: "+FWResAdjSize(15)+"px "+FWResAdjSize(5)+"px "+FWResAdjSize(5)+"px "+FWResAdjSize(10)+"px; background-color: #"+cBGColor+"; " + ;
372                 "border-radius: "+FWResAdjSize(8)+"px; border: "+FWResAdjSize(1)+"px solid #"+cBGColor+"; padding: "+FWResAdjSize(6)+"px; }"
373         Endif
374         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
375
376     Case nType == CSS_PANEL_OPTION_TOP
377         If lClientHtml
378             cCSS += "QFrame{ margin: 0px; border: none; border-bottom: "+FWResAdjSize(1)+"px solid #F4F4F4; " + ;
379                 "border-radius: 0px; border-bottom-color: #D1D1D1; background-color: transparent; }"
380         Else
381             cCSS += "QFrame{ margin: "+FWResAdjSize(10)+"px "+FWResAdjSize(5)+"px 0px "+FWResAdjSize(10)+"px; border: none; border-bottom: "+FWResAdjSize(1)+"px solid
#F4F4F4; " + ;
382                 "border-radius: 0px; border-bottom-color: #D1D1D1; background-color: transparent; }"
383         Endif
384
385         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
386
387     Case nType == CSS_PANEL_CONTENT
388
389         cCSS += "QFrame{ background-color: transparent; margin: none; border: none; }"
390
391         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
392
393     Case nType == CSS_PANEL_OPTION_BOTTOM
394         cCSS += "QFrame{ margin: "+FWResAdjSize(10)+"px "+FWResAdjSize(5)+"px 0px "+FWResAdjSize(10)+"px; border: none; border-top: "+FWResAdjSize(1)+"px solid #F4F4F4; "
+ ;
395             "border-radius: 0px; border-top-color: #D1D1D1; background-color: transparent; }"
396
397         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
398
399     Case nType == CSS_PANEL_CONTEXT
400         If lClientHtml
401             cCSS += "QFrame{ margin: 0px; background-color: #F4F4F4; border: "+FWResAdjSize(1)+"px solid #F4F4F4; "
402         Else
403             cCSS += "QFrame{ margin: "+FWResAdjSize(5)+"px "+FWResAdjSize(5)+"px "+FWResAdjSize(15)+"px "+FWResAdjSize(10)+"px; background-color: #F4F4F4; " + ;
404                 "border: "+FWResAdjSize(1)+"px solid #F4F4F4; "
405         Endif
406
407         If Empty(xPar01)
408             cCSS += "border-radius: "+FWResAdjSize(8)+"px;"
409         Endif
410
411         cCSS += "}"

```

```

412
413         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
414
415     Case nType == CSS_PANEL_HEADER
416
417         If FWIsMobile()
418             cCSS += "QFrame{ margin: "+FWResAdjSize(15)+"px "+FWResAdjSize(10)+"px 0px "+FWResAdjSize(5)+"px; padding: "+FWResAdjSize(6)+"px; border:
"+FWResAdjSize(1)+"px solid #"+cBGColor+"; " + ;
419                 "background-color: #"+cBGColor+"; " + ;
420                 "border-top-right-radius: "+FWResAdjSize(8)+"px; border-top-left-radius: "+FWResAdjSize(8)+"px; }"
421
422         Else
423             If lClientHtml
424                 cCSS += "QFrame{ margin: 0px; padding: 6px; border: 1px solid #FFFFFF; " + ;
425                 "background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, " + ;
426                 "stop: 0.0 #FFFFFF, stop: 0.6 #F0F0F0, stop: 0.7 #F0F0F0, stop: 1.0 #E5E1DE); " + ;
427                 "border-top-right-radius: 8px; border-top-left-radius: 8px; }"
428
429             Else
430                 cCSS += "QFrame{ margin: 15px 10px 0px 5px; padding: 6px; border: 1px solid #FFFFFF; " + ;
431                 "background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, " + ;
432                 "stop: 0.0 #FFFFFF, stop: 0.6 #F0F0F0, stop: 0.7 #F0F0F0, stop: 1.0 #E5E1DE); " + ;
433                 "border-top-right-radius: 8px; border-top-left-radius: 8px; }"
434             Endif
435         EndIf
436         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
437
438     Case nType == CSS_PANEL_FOOTER
439         If lClientHtml
440             cCSS += "QFrame{ margin: 0px; border: "+FWResAdjSize(1)+"px solid #FFFFFF; " + ;
441             "background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, " + ;
442             "stop: 0.0 #FFFFFF, stop: 0.6 #f0f0f0, stop: 0.7 #f0f0f0, stop: 1.0 #E5E1DE); " + ;
443             "border-bottom-right-radius: "+FWResAdjSize(8)+"px; border-bottom-left-radius: "+FWResAdjSize(8)+"px; }"
444
445         Else
446             cCSS += "QFrame{ margin: 0px "+FWResAdjSize(10)+"px "+FWResAdjSize(15)+"px "+FWResAdjSize(5)+"px; border: "+FWResAdjSize(1)+"px solid #FFFFFF; " + ;
447             "background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1, " + ;
448             "stop: 0.0 #FFFFFF, stop: 0.6 #f0f0f0, stop: 0.7 #f0f0f0, stop: 1.0 #E5E1DE); " + ;
449             "border-bottom-right-radius: "+FWResAdjSize(8)+"px; border-bottom-left-radius: "+FWResAdjSize(8)+"px; }"
450         Endif
451         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
452
453     Case nType == CSS_PANEL_BTN_FOOTER
454
455         cCSS += "QFrame{ margin: "+FWResAdjSize(10)+"px "+FWResAdjSize(5)+"px 0px "+FWResAdjSize(10)+"px; border: none; border-top: "+FWResAdjSize(1)+"px solid #F4F4F4; "
+ ;
456                 "border-radius: 0px; border-top-color: #D1D1D1; background-color: transparent; }"
457
458         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
459
460     Case nType == CSS_PANEL_BKLOGIN
461         cCSS += "QFrame{ background-color: #EEEEEE; border-left: "+FWResAdjSize(1)+"px solid #BBBBBB !important; border-right: "+FWResAdjSize(1)+"px solid #BBBBBB; }"

```

```

462         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
463
464     Case nType == CSS_PANEL_LOGINGHEADER
465         cCSS += PosCss( cAdvPLComp, CSS_PANEL_BKGLLOGIN, xPar01)
466         cCSS += "QFrame{ border-top-right-radius: "+FWResAdjSize(8)+"px; border-top: "+FWResAdjSize(1)+"px solid #BBBBBB; border-top-left-radius: "+FWResAdjSize(8)+"px;
}"
467
468         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
469
470     Case nType == CSS_PANEL_LOGINMAIN
471         cCSS += PosCss( cAdvPLComp, CSS_PANEL_BKGLLOGIN, xPar01)
472         cCSS += "QFrame{ border-top: Transparent; }"
473
474         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
475
476     Case nType == CSS_PANEL_LOGINFOOTER
477         cCSS += PosCss( cAdvPLComp, CSS_PANEL_BKGLLOGIN, xPar01)
478         cCSS += "QFrame{ border-bottom-right-radius: "+FWResAdjSize(8)+"px; border-bottom: "+FWResAdjSize(1)+"px solid #BBBBBB; border-bottom-left-radius:
"+FWResAdjSize(8)+"px ; border-top:transparent; }"
479
480         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
481
482     Case nType == CSS_PANEL_LOGINASKREG
483         cCSS += "QFrame{ background-color: qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #EEEEEE, stop: 1 #FFFFFF); "+;
484         " border-left: "+FWResAdjSize(1)+"px solid #BBBBBB !important; border-right: "+FWResAdjSize(1)+"px solid #BBBBBB;" +;
485         " border-bottom-right-radius: "+FWResAdjSize(8)+"px; border-bottom: "+FWResAdjSize(1)+"px solid #BBBBBB; border-bottom-left-radius: "+FWResAdjSize(8)+"px ;
border-top:transparent; }"
486
487         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
488
489     Case nType == CSS_PANEL_LINEAUX
490         cCSS += "QFrame{ border-bottom: "+FWResAdjSize(0.5)+"px solid #FFFFFF; border-top: "+FWResAdjSize(0.5)+"px solid #BBBBBB; border-radius:0px;}"
491
492         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
493
494     Case nType == CSS_PANEL_WIZARD
495         cCSS += "QFrame{ background-color:#F0F0F0; }"
496
497         cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
498
499
500     Case nType == CSS_BREADCUMB
501         If FWIsMobile()
502             cCSS += "QLabel{ font: bold "+FWResAdjSize(16)+"px; color: #"+__aColors[2]+" ; background-color: transparent; border: none; margin: 0px; }"
503         Else
504             cCSS += "QLabel{ font-weight: bold; font-size: 16px; color: #C0C0C0; background-color: transparent; border: none; margin: 0px; }"
505         EndIf
506         cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
507
508     Case nType == CSS_LABEL_HEADER
509         cCSS += "QLabel{ font-size: "+FWResAdjSize(14)+"px; background-color: transparent; color: #000000; border: none; margin: 0px; }"
510

```

```

511         cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
512
513     Case nType == CSS_LABEL_FOCAL
514
515         xPar01 := IIF( Empty(xPar01), '13', xPar01)
516         IF !ValType(xPar01) == 'A'
517             xPar01 := {'13', .T.}
518         EndIf
519         if lClientHtml
520             cCSS += "QLabel{ font-size:"+ FWResAdjSize(15) + "px; color: #606060; background-color: transparent; border: none; margin: 0px; }"
521         Else
522             cCSS += "QLabel{ font:"+IIF(xPar01[2], 'bold ', ' ') + FWResAdjSize(val(xPar01[1])) + "px; color: #606060; background-color: transparent; border: none; margin:
0px; }"
523         Endif
524         cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
525
526     Case nType == CSS_LABEL_NORMAL
527         If FWIsMobile()
528             xPar01 := IIF( Empty(xPar01), '13', xPar01)
529             IF !ValType(xPar01) == 'A'
530                 xPar01 := {'13', .T.}
531             EndIf
532             cCSS += "QLabel{ font: "+IIF(xPar01[2], 'bold ', ' ') + FWResAdjSize(val(xPar01[1])) + "px; color: #444444; background-color: transparent; border: none;
margin: 0px; }"
533         Else
534             cCSS += "QLabel{ font-size: "+Str(nPxNormal)+"px; color: #B2B2B2; background-color: transparent; border: none; margin: 0px; }"
535         EndIf
536         cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
537
538     Case nType == CSS_LABEL_SUBTITLE
539         If FWIsMobile()
540             cCSS += "QLabel{ font: "+FWResAdjSize(13)+"px; color: #"+__aColors[2]+"; background-color: transparent; border: none; margin: 0px; }"
541         Else
542             cCSS += "QLabel{ font: 13px; color: #B2B2B2; background-color: transparent; border: none; margin: 0px; }"
543         EndIf
544         cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
545
546     Case nType == CSS_GET_FOCAL
547         cCSS += "QLineEdit { font-weight: bold; font-size: "+FWResAdjSize(16)+"px; color: #656565; "+;
548         " background-color: #FFFFFF; border: "+FWResAdjSize(1)+"px solid #9C9C9C; border-radius: "+FWResAdjSize(4)+"px; padding: "+FWResAdjSize(4)+"px; "
549
550         If FWIsMobile()
551             cCSS += " min-height: "+FWResAdjSize(30)+"px;}"
552
553             cCSS += " TButtonGet{ min-width: "+FWResAdjSize(35)+"px; outline :none; background:qlineargradient(x1: 0, y1: 0, x2: 0, y2: 1,stop: 0 #FFFFFF, stop: 1
#E9EBF1); " +;
554             "border: "+FWResAdjSize(1)+"px solid #BBBBBB; border-radius: "+FWResAdjSize(6)+"px; }"
555         Else
556             cCSS += "}"
557         EndIf
558
559         cCSS += "QLineEdit:disabled { color:#656565; background-color: transparent; border: none; padding: 0px; }"

```

```

560         If FWIsMobile()
561             cCSS += GET_CSS_FOCAL
562         EndIf
563     /*EndIf*/
564
565     cCSS := FWCSSVerify( cAdvPLComp, "QLineEdit", cCSS )
566
567     Case nType == CSS_GET_NORMAL
568         If FWIsMobile()
569             cCSS += "QLineEdit { min-height: "+FWResAdjSize(30)+"px; font: bold; font-size: "+FWResAdjSize(16)+"px; text-align: right; color: #888888;"+";
570             " background-color: #FFFFFF; border: "+FWResAdjSize(1)+"px solid #BBBBBB; border-radius: "+FWResAdjSize(4)+"px; padding: "+FWResAdjSize(4)+"px;}"
571             cCSS += "QLineEdit:disabled {min-height: "+FWResAdjSize(30)+"px !important; color:#444444; background-color: transparent; border: none; }"
572             cCSS += GET_CSS_FOCAL
573         Else
574             If IsInCallStack("STIRodape") .AND. lClientHtml
575                 cCSS += "QLineEdit { font-weight: bold; font-size: "+Str(nPxNormal)+"px; text-align: right; color: #656565; background-color: transparent; border: none;
margin: 0px;}"
576             Else
577                 cCSS += "QLineEdit { font-weight: bold; font-size: "+Str(nPxNormal)+"px; text-align: right; color: #656565; background-color: #FFFFFF; border: 1px solid
#9C9C9C; border-radius: 4px; padding: 4px; }"
578             Endif
579             cCSS += "QLineEdit:disabled {color:#656565; background-color: transparent; border: none; }"
580         EndIf
581
582         cCSS := FWCSSVerify( cAdvPLComp, "QLineEdit", cCSS )
583
584     Case nType == CSS_GET_TOTAL
585         If FWIsMobile()
586             cCSS += "QLineEdit { font-size: "+FWResAdjSize(16)+"px; text-align: right; color: #888888;"+";
587             " background-color: #FFFFFF; border: "+FWResAdjSize(1)+"px solid #BBBBBB; border-radius: "+FWResAdjSize(4)+"px; padding: "+FWResAdjSize(4)+"px;}"
588             cCSS += "QLineEdit:disabled {color:#444444; background-color: transparent; border: none; }"
589             cCSS += GET_CSS_FOCAL
590         Else
591             cCSS += "QLineEdit { font: bold; font-size: 13px; text-align: right; color: #656565; background-color: #FFFFFF; border: 1px solid #9C9C9C; border-radius:
4px; padding: 4px; }"
592             cCSS += "QLineEdit:disabled {color:#656565; background-color: transparent; border: none; }"
593         EndIf
594
595         cCSS := FWCSSVerify( cAdvPLComp, "QLineEdit", cCSS )
596
597     Case nType == CSS_GET_ERROR
598         If FWIsMobile()
599             cCSS += "QLineEdit {min-height: "+FWResAdjSize(30)+"px; font: bold; font-size: "+FWResAdjSize(16)+"px; color: #888888; background-color: #FFF2F0;"+";
600             " border: "+FWResAdjSize(1)+"px solid #E2938F; border-radius: "+FWResAdjSize(4)+"px; padding: "+FWResAdjSize(4)+"px; }"
601
602             cCSS += GET_CSS_FOCAL
603         EndIf
604
605         cCSS := FWCSSVerify( cAdvPLComp, "QLineEdit", cCSS )
606     Case nType == CSS_LABEL_TOTAL
607         If FWIsMobile()
608             cCSS += "QLabel{ font: "+FWResAdjSize(40)+"px; color: #444444; background-color: transparent; border: none; margin: 0px; }"

```



```

609     Else
610         cCSS += "QLabel{ font-size: 35px; color: #656565; background-color: transparent; border: none; margin: 0px; }"
611     EndIf
612     cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
613
614     Case nType == CSS_LABEL_TRIAL
615         cCSS += "QLabel{ font:bold; font-size: "+FWResAdjSize(14)+"px; color: #BD2C08; background-color: #FFD000; border: none; margin: 0px; }"
616
617         cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
618
619     Case nType == CSS_LABEL_REQUIRED
620         cCSS += "QLabel{ font:bold; font-size: "+FWResAdjSize(12)+"px; color: #DC4000; background-color:transparent; border: none; margin: 0px; }"
621
622         cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
623
624
625     Case nType == CSS_CHECKBOX_DEFAULT
626         If FWIsMobile()
627             cCSS += "QCheckBox { background-repeat: no-repeat; "+;
628                 "    outline :none; "+;
629                 "    padding-top: 0px; "+;
630                 "    font:bold; "+;
631                 "    font-size: "+FWResAdjSize(13)+"px; "+;
632                 "    color: #656565; "+;
633                 "    border: none;} "+;
634             "QWidget { border-width: 0px; }"+;
635             "    QCheckBox::indicator { "+;
636             "    width: "+FWResAdjSize(15)+"px;"+;
637             "    height: "+FWResAdjSize(15)+"px;}"
638
639             cCSS := FWCSSVerify( cAdvPLComp, "QCheckBox", cCSS )
640         EndIf
641
642     Case nType == CSS_CHOOSE_PAYMENT
643         If FWIsMobile()
644             cCSS += "QWidget{border: none;}"+;
645             "QScrollBar:horizontal{ "+;
646             "    border:none; max-height: "+FWResAdjSize(6)+"px; min-height: "+FWResAdjSize(6)+"px;"+;
647             "    background: none; border: none; margin: 0 0 0 0;"+;
648             "}"
649
650             cCSS := FWCSSVerify( cAdvPLComp, "Q3ListBox", cCSS )
651         EndIf
652
653
654     Case nType == CSS_PAYMENT_ITEM
655         If FWIsMobile()
656             cCSS += "QWidget{border: none; background-color: #FFFFFF;}"
657
658             cCSS := FWCSSVerify( cAdvPLComp, "QFrame", cCSS )
659         EndIf
660

```

```

661     Case nType == CSS_COMBOBOX
662         If FWIsMobile()
663             cCSS += "QComboBox {font-size: "+FWResAdjSize(20)+"px; ";
664             "    color: #2462A6; "+;
665             "    background-color: #ffffff; "+;
666             "    min-height: "+FWResAdjSize(40)+"px; "+;
667             "    max-height: "+FWResAdjSize(40)+"px; "+;
668             "    border-style: solid; "+;
669             "    border-width: 2px; "+;
670             "    padding-left: 0px; } "+;
671             "    QComboBox QAbstractItemView { "+;
672             "    border: "+FWResAdjSize(2)+"px solid darkgray; "+;
673             "    selection-background-color: #"+__aColors[2]+";";
674
675             cCSS := FWCSSVerify( cAdvPLComp, "TCOMBOBOX", cCSS )
676         EndIf
677
678     Case nType == CSS_STEP_WIZ
679         cCSS := "#"+__aColors[4]
680
681     Case nType == CSS_LABEL_ALERT
682         If FWIsMobile()
683             cCSS += "QLabel{ font: "+FWResAdjSize(22)+"px; color: #444444; background-color: transparent; border: none; margin: 0px; }"
684         Else
685             cCSS += "QLabel{ font-size: 22px; color: #656565; background-color: transparent; border: none; margin: 0px; }"
686         EndIf
687         cCSS := FWCSSVerify( cAdvPLComp, "QLabel", cCSS )
688
689     EndCase
690
691     Return cCSS
692
693 //-----
694 /*{Protheus.doc} POSBrwContainer
695 Função responsável por construir o container para o browse do POS
696
697 @param o Container para a construção da interface do browse
698
699 @return oContent Painei para construção do browse
700
701 */
702 //-----
703 Function POSBrwContainer(o)
704     Local oContent
705     Local oLeft
706     Local oRight
707     Local oRTop
708     Local oRBottom
709     Local oLTop

```

```

713 Local oLBottom
714 Local oLHeader
715 Local oRHeader
716 Local oSep
717
718 @ 000,000 BITMAP oLeft RESOURCE "x.png" NOBORDER SIZE 007,000 OF o ADJUST PIXEL
719 oLeft:Align := CONTROL_ALIGN_LEFT
720 oLeft:ReadClientCoors(.T.,.T.)
721
722 @ 000,000 BITMAP oLHeader RESOURCE "x.png" NOBORDER SIZE 000,016 OF oLeft ADJUST PIXEL
723 oLHeader:Align := CONTROL_ALIGN_TOP
724 oLHeader:SetCSS( POSCSS( GetClassName(oLHeader), CSS_BROWSE_HEADER_LEFT ) )
725
726 @ oLHeader:nHeight/2,oLeft:nWidth/2-4.5 BITMAP oLTop RESOURCE "pos_brw_corner_tl.png" NOBORDER SIZE 04.5,04.5 OF oLeft ADJUST PIXEL
727
728 @ oLeft:nHeight/2-4.5,oLeft:nWidth/2-4.5 BITMAP oLBottom RESOURCE "pos_brw_corner_bl.png" NOBORDER SIZE 04.5,04.5 OF oLeft ADJUST PIXEL
729
730 @ 000,000 BITMAP oRight RESOURCE "x.png" NOBORDER SIZE 010,000 OF o ADJUST PIXEL
731 oRight:Align := CONTROL_ALIGN_RIGHT
732 oRight:ReadClientCoors(.T.,.T.)
733
734 @ 000,000 BITMAP oRHeader RESOURCE "x.png" NOBORDER SIZE 000,016 OF oRight ADJUST PIXEL
735 oRHeader:Align := CONTROL_ALIGN_TOP
736 oRHeader:SetCSS( POSCSS( GetClassName(oRHeader), CSS_BROWSE_HEADER_RIGHT ) )
737
738 @ oRHeader:nHeight/2,000 BITMAP oRTop RESOURCE "pos_brw_corner_tr.png" NOBORDER SIZE 04.5,04.5 OF oRight ADJUST PIXEL
739
740 @ oRight:nHeight/2-4.5,000 BITMAP oRBottom RESOURCE "pos_brw_corner_br.png" NOBORDER SIZE 04.5,04.5 OF oRight ADJUST PIXEL
741
742 @ 000,000 BITMAP oContent RESOURCE "x.png" NOBORDER SIZE o:nWidth/2-20,o:nHeight/2-20 OF o ADJUST PIXEL
743 oContent:Align := CONTROL_ALIGN_ALLCLIENT
744
745 @ 000,000 BITMAP oSep RESOURCE "pos_brw_degrade.png" NOBORDER SIZE 000,2.5 OF oContent ADJUST PIXEL
746 oSep:Align := CONTROL_ALIGN_BOTTOM
747 oSep:SetCSS( FWCSSVerify( GetClassName(oSep), "QFrame", "QFrame { background-color: #FFFFFF; }" ) )
748
749 @ 000,000 BITMAP oContent RESOURCE "x.png" NOBORDER SIZE o:nWidth/2-20,o:nHeight/2-20 OF oContent ADJUST PIXEL
750 oContent:Align := CONTROL_ALIGN_ALLCLIENT
751
752 Return oContent

```

Framework/EventBus/STFEventBus.prw

152 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2
3 Static _aHandlers := {}
4
5
6 //-----
7 /*{Protheus.doc} STFAddEventHandler
8 Add evento
9 @param      cFunName      Nome da funcao
10 @param      cEventName    Nome do evento
11 @param      bHandler      Bloco de manipulacao
12 @return     Nil
13 @obs
14 @sample
15 */
16 //-----
17 Function STFAddEventHandler(cFunName, cEventName, bHandler)
18
19 Local nPosClass := 0 //Posicao da class
20 Local nPosEvent := 0 //Posicao do evento
21
22 Default cFunName      := ""
23 Default cEventName    := ""
24 Default bHandler      := "{| | }"
25
26 cFunName      := Lower(cFunName)
27 cEventName    := Lower(cEventName)
28
29 nPosClass := AScan( _aHandlers, { |x| x[1] == cFunName } )
30 If nPosClass > 0
31     nPosEvent := AScan( _aHandlers[nPosClass][2], { |x| x[1] == cEventName } )
32     If nPosEvent > 0
33         AAdd( _aHandlers[nPosClass][2][nPosEvent][2], bHandler )
34     Else
35         AAdd( _aHandlers[nPosClass][2], { cEventName, { bHandler } } )
36     EndIf
37 Else
38     AAdd( _aHandlers, { cFunName, { { cEventName, { bHandler } } } } )
39 EndIf
40
41 Return Nil
42
43
44 //-----
45 /*{Protheus.doc} STFRemoveEventHandler
46 Remove evento
47 @param      cFunName      Nome da funcao
48 @param      cEventName    Nome do evento
49 @param      bHandler      Bloco de manipulacao
50 @return     Nil
51 @obs
52 @sample

```

```

53 /*/
54 //-----
55 Function STFRemoveEventHandler(cFunName, cEventName, bHandler)
56
57 Local nPosClass := 0 //Posicao da class
58 Local nPosEvent := 0 //Posicao do evento
59
60 Default cFunName := ""
61 Default cEventName := ""
62 Default bHandler := "{|| }"
63
64 cFunName := Lower(cFunName)
65 cEventName := Lower(cEventName)
66
67 nPosClass := AScan( _aHandlers, { |x| x[1] == cFunName } )
68
69 If nPosClass > 0
70     nPosEvent := AScan( _aHandlers[nPosClass][2], { |x| x[1] == cEventName } )
71     If nPosEvent > 0
72         nPosHandler := AScan( _aHandlers[nPosClass][2][nPosEvent][2], { |x| GetCbSource(x) == GetCbSource(bHandler) } )
73         If nPosHandler > 0
74             ADel( _aHandlers[nPosClass][2][nPosEvent][2], nPosHandler )
75             ASize( _aHandlers[nPosClass][2][nPosEvent][2], Len( _aHandlers[nPosClass][2][nPosEvent][2] ) - 1 )
76         EndIf
77     EndIf
78 EndIf
79
80 Return
81
82
83 //-----
84 /*/{Protheus.doc} STFFireEvent
85 Start evento
86 @param      cFunName      Nome da funcao
87 @param      cEventName    Nome do evento
88 @param      bHandler      Bloco de manipulacao
89 @return Nil
90 @obs
91 @sample
92 */
93 //-----
94 Function STFFireEvent(cFunName, cEventName, uEventData)
95
96 Local nPosClass := 0 //Posicao da class
97 Local nPosEvent := 0 //Posicao do evento
98 Local aRets := {} //Array retornos
99
100 Default cFunName := ""
101 Default cEventName := ""
102 Default uEventData := "{|| }"
103
104 cFunName := Lower(cFunName)

```

```

105 cEventName := Lower(cEventName)
106
107 nPosClass := AScan( _aHandlers, { |x| x[1] == cFunName .Or. x[1]=="*" } )
108
109 If nPosClass > 0
110     nPosEvent := AScan( _aHandlers[nPosClass][2], { |x| x[1] == cEventName .Or. x[1]=="*" } )
111     If nPosEvent > 0
112         AEval( _aHandlers[nPosClass][2][nPosEvent][2], { |x| AAdd(aRets, Eval(x, cFunName, cEventName, uEventData)) } )
113     EndIf
114 EndIf
115
116 Return aRets
117
118
119 //-----
120 /*{Protheus.doc} STFClearEvents
121 Limpa evento
122 @param
123 @return Nil
124 @obs
125 @sample
126 */
127 //-----
128 Function STFClearEvents()
129 _aHandlers := {}
130 Return
131
132
133 //-----
134 Function STFExistEvent(cFunName, cEventName)
135
136 Local nPosClass := 0    //Posicao da class
137 Local nPosEvent := 0    //Posicao do evento
138
139 Default cFunName      := ""
140 Default cEventName    := ""
141
142 cFunName      := Lower(cFunName)
143 cEventName    := Lower(cEventName)
144
145 nPosClass := AScan( _aHandlers, { |x| x[1] == cFunName .Or. x[1]=="*" } )
146
147 If nPosClass > 0
148     nPosEvent := AScan( _aHandlers[nPosClass][2], { |x| x[1] == cEventName .Or. x[1]=="*" } )
149
150 EndIf
151
152 Return nPosEvent > 0

```

Framework/Exit/STFExit.prw

127 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "PARMTYPE.CH"
3 #INCLUDE "STFEXIT.CH"
4
5 Static lPerExit := .F.      // Permite sair do sistema. Usado para conferencia de caixa
6
7 //-----
8 /*{Protheus.doc} STFExit
9 Finalização do Sistema
10 @param
11 @return lRet Retorna se pode sair do sistema
12 @obs
13 @sample
14 */
15 //-----
16 Function STFExit()
17
18 Local lRet      := .T.      // Retorna se pode sair do sistema
19 Local nTotSale  := 0        // Valor total da venda
20 Local oTotal    := STFGetTot() // Recebe o Objeto totalizador
21 Local lSTExitPOS := ExistBlock("STEXITPOS") // Verifica se existe o PE FTVD7019
22 Local lCloseCxAut := SuperGetMv("MV_LJFECCX",,T.) // Indica se usa Fechamento automatico de caixa
23 Local lljConff     := SuperGetMv("MV_LJCONFF",,T.,.F.) // Fechamento de caixa
24 Local lObgFecha    := SuperGetMv("MV_LJOBGCF",,T.) // Obrigatorio o fechamento de caixa.
25 Local lMobile      := STFGetCfg( "lMobile" , .F. ) // Valida se é versao mobile
26 Local cCesta       := STDGPBasket("SL2", "L2_SITUA") // Recebe a situação do item da venda
27 Local lOpenCash    := STBOpenCash() // Verifico se o caixa esta aberto
28
29 If ValType(oTotal) == "0"
30     nTotSale := oTotal:GetValue( "L1_VLRTOT" )
31
32     If nTotSale > 0 .OR. cCesta == "05"
33         STFMessage("STFExit", "STOP", STR0001 ) //"Nao e possivel sair, venda ja iniciada. Fechar ou cancelar a venda."
34         lRet := .F.
35     EndIf
36 EndIf
37
38 If lRet
39     If lOpenCash .AND. lljConff
40         If lObgFecha .AND. !STFGetPerExit()
41             STFMessage("STFExit", "STOP", STR0003) //"Conf. de Caixa Ativa. É obrigatório confirmar o Encerramento de Caixa - Menu(F2).
42             STFShowMessage("STFExit")
43             lRet = .F.
44         ElseIf !STFGetPerExit()
45             STFMessage("STFExit", "YESNO", STR0004 ) //"Deseja abandonar a conferência de caixa e sair do sistema?"
46             lRet := STFShowMessage("STFExit")
47             STFSetPerExit(lRet)
48         EndIf
49     ElseIf !STFGetPerExit() .AND. STBOpenCash()
50         STFMessage("STFExit", "YESNO", STR0002 ) //"Deseja realmente sair do sistema?"
51         lRet := STFShowMessage("STFExit")
52

```

```

53      STFSetPerExit(lRet)
54  EndIf
55
56 EndIf
57
58 If lSTExitPOS
59   lRet := ExecBlock( "STEXITPOS",.F.,.F.,{} )
60 EndIf
61
62 // Se puder sair do POS faz validacoes necessarias antes
63 If lRet
64
65   //Fecha comunicacao com perifericos
66   STWCloseDevice()
67
68   //Ao Sair Fecha o Caixa Automaticamente se o Parametro estiver configurado
69   If lCloseCxAut
70     lRet := STWPOSCloseCash()
71   EndIf
72
73   //Libera o objeto de totais da venda
74   If ExistFunc("STFTotRelease")
75     STFTotRelease()
76   EndIf
77   //Versao Mobile sai do sistema direto
78   //sem voltar para o menu do Protheus
79   If ValType(lMobile) == "L" .AND. lMobile
80     MS_QUIT()
81   EndIf
82
83   If lRet
84     FreeUsedCode()
85   Endif
86
87 EndIf
88
89 If lRet
90   PutGlbVars("lPosOpen",.F.) // -- Indica que o PDV esta fechado
91 EndIf
92
93 STI7InfCPF(.F.)
94
95 STFShowMessage("STFExit")
96
97 Return lRet
98
99
100 //-----
101 /*{Protheus.doc} STFSetPerExit
102 Seta variavel statica que controla a saida do sistema. Usado para conferencia de caixa
103 @param lExit Valida se pode sair do sistema
104 @return Nil

```



```
105 @obs
106 @sample
107 /*/
108 //-----
109 Function STFSetPerExit(lExit)
110
111 Default lExit := .F.
112
113 lPerExit := lExit    // Permite sair do sistema. Usado para conferencia de caixa
114
115 Return Nil
116
117
118 //-----
119 /*/{Protheus.doc} STFGetPerExit
120 Retorna a variavel estatica que controla a saida do sistema. Usado para conferencia de caixa
121 @return  lPerExit
122 @obs
123 @sample
124 /*/
125 //-----
126 Function STFGetPerExit()
127 Return lPerExit
```

Framework/Facilitator/STFMobWizard.prw

767 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE 'Protheus.ch'
2 #INCLUDE "TBICONN.CH"
3 #INCLUDE "POSCSS.CH"
4 #INCLUDE "STPOS.CH"
5 #INCLUDE "FILEIO.CH"
6 #INCLUDE "STFMOBWIZARD.CH"
7
8
9 Static oStepWiz      := Nil //Objto tipo Wizard
10 Static oNewPag      := Nil //Objeto que adiciona nova pagina no wizard
11 Static oDlg         := Nil //Dialog bkg
12 Static oPanelBkg    := Nil //Panel de BackGround
13 Static oListImp     := Nil //List para impressoras
14 Static oListTef     := Nil //List TEFs
15 Static lImpWifi     := .T. //Indica se e impressora Wifi
16 Static lImpBlu      := .F. //Indica se e impressora Wifi
17 Static lTefAtivo    := .F. //Indica se TEF esta ativo
18 Static lFimWizard   := .F. //Controla se pode finalizar do Wizard
19
20 Static aSLG         := {} //Array que guarda os dados do cadastro de estacao
21 Static aMDG         := {} //Array que guarda os dados do cadastro de estacao TEF
22
23 Static cLG_CODIGO   := "" //Codigo da estacao
24 Static cLG_PDV      := "" //Numero do PDV
25 Static cLG_SERIE    := "" //Serie para estacao atual
26 Static cMV_CODREG   := "" //Regime tributario para NFC-E
27 Static cMV_NFCEIDT  := "" //Token do sefaz
28 Static cMV_LJEMPCK  := "" //Codigo da empresa cadastrado no daruma Migrate
29 Static cMV_LJFILIN  := "" //Codigo da Filial do Servidor
30 Static cLG_WSSRV    := "" //endereço ws do servidor
31
32 Static cLG_IMPFISC  := "" //Modelo da Impressora
33 Static cLG_TSCPORT  := "" //porta para impressora de Rede | Wifi . Se Bluetooth sera gravado o timeout
34 Static cLG_TSCSRV   := "" //Porta IP da impressora | Se Bluetooth nome do equipamento Case-Sensitive
35 Static cLG_PORTIF   := "" //Porta da impressora | Se Bluetooth sera gravado 'BLU'
36
37
38 Static cMDG_TEFATV  := "" //Ativa o TEF
39 Static cMDG_CARPAY  := "" //Usa TEF PAYGO
40
41 Static cImpAnterior := "" //Armazena impressora anterior configurada para controle de mudanca
42
43 Static oImg         := Nil //Obejeto de imagem
44
45 //-----
46 /*{Protheus.doc} STFMobWizard
47 Cria wizard de Configurações para mobile
48
49 @param
50 @return .T.
51 @obs
52 @sample

```

```

53  /*/
54  //-----
55  Function STFMobWizard()
56
57  //Instancia a classe FWWizard
58  oStepWiz:= FWWizardControl():New(,)// Define o tamanho do wizard  ex: {600,800}
59  oStepWiz:ActiveUISteps()
60
61
62  // 01 ----- "Primeiros passos"
63  oNewPag := oStepWiz:AddStep("1")//Adiciona a primeira tela do wizard
64  oNewPag:SetStepDescription(STR0001) //Altera a descrição do step //"Primeiros passos"
65  oNewPag:SetConstruction({|Panel|cria_pg1(Panel)}) //Define o bloco de construção
66  oNewPag:SetNextAction({||valida_pg1()})//Define o bloco ao clicar no botão Próximo
67  oNewPag:SetCancelAction({|FWAlertInfo(STR0002), .F.})//Valida acao cancelar, nao deixa sair do wizard //"Wizard não pode ser cancelado!"
68
69
70  // 02 ----- "Impressora"
71  oNewPag := oStepWiz:AddStep("2", {|Panel|cria_pg2(Panel)})
72  oNewPag:SetStepDescription(STR0003)//"Impressora"
73  oNewPag:SetNextAction({||valida_pg2()})
74  oNewPag:SetCancelWhen({||.F.})
75
76
77  // 03 ----- "TEF"
78  oNewPag := oStepWiz:AddStep("3", {|Panel|cria_pn3(Panel)})
79  oNewPag:SetStepDescription(STR0004)//"TEF"
80  oNewPag:SetNextAction({||valida_pg3()})
81  oNewPag:SetCancelWhen({||.F.})
82
83
84  // 04 ----- "Leitor via Câmera"
85  oNewPag := oStepWiz:AddStep("4", {|Panel|cria_pn4(Panel)})
86  oNewPag:SetStepDescription(STR0005)//"Leitor via Câmera"
87  oNewPag:SetNextAction({||valida_pg4()})
88  oNewPag:SetCancelWhen({||.F.})
89
90
91  // 05 ----- "Marca da Empresa"
92  oNewPag := oStepWiz:AddStep("5", {|Panel|cria_pn5(Panel)})
93  oNewPag:SetStepDescription(STR0006)//"Marca da Empresa"
94  oNewPag:SetNextAction({|| lFimWizard := FimWizard(), lFimWizard })
95  oNewPag:SetCancelWhen({||.F.})
96
97
98  //Ativa Wizard
99  oStepWiz:Activate()
100
101  //Desativa Wizard
102  oStepWiz:Destroy()
103
104

```

```

105 Return .T.
106
107
108 //-----
109 /*{Protheus.doc} cria_pgl
110 Cria wizard de Configurações para mobile pgl
111
112 @param oPanel - Panel de Fundo
113 @return Nil
114 @obs
115 @sample
116 */
117 //-----
118 Static Function cria_pgl(oPanel)
119
120
121 Local oSayPdv          := Nil //Numero da estacao PDV
122 Local oTGetPDV         := Nil //Numero da estacao PDV
123 Local oSaySerie        := Nil //Serie da estacao
124 Local oTGetSerie       := Nil //Serie da estacao
125 Local oSayEndWs        := Nil //Endereco do Webservice
126 Local oTGetEndWs       := Nil //Endereco do Webservice
127 Local oCbxBxRegTrib    := Nil //Regime tributario para NFC-E
128 Local aCbxBxRegTrib    := {} //Regime tributario para NFC-E
129 Local oSayRegTrib      := Nil //Regime de tributacao
130 Local oSayToken        := Nil //Token SEFAZ
131 Local oTGetToken       := Nil //Token SEFAZ
132 Local oSayFilServer    := Nil //Filial do servidor
133 Local oSayExEndWs      := Nil //Say de exemplo de end WS
134 Local oTGetFilServer   := Nil //Filial do servidor
135
136 Default oPanel         := Nil // Panel Bkg
137
138
139 //----- Codigo da estacao e numero do PDV
140
141 cLG_PDV := PADR(STFGetStation("PDV"),TamSx3("LG_PDV")[1])
142
143 oSayPdv:= TSay():New(05,10,{|| STR0007 },oPanel,,,,,T,,,100,25) //"Código do PDV"
144 oSayPdv:SetCSS( POSCSS (GetClassName(oSayPdv), CSS_LABEL_NORMAL ))
145
146 oTGetPDV := TGet():New(15,10,{|u| if( PCount() > 0, cLG_PDV := u, cLG_PDV ) },oPanel,130,025, "@E " + Replicate("9",TamSx3("LG_PDV")[1] )
,0,,,F,,,T,,,F,,,F,,,F,,,F,,,F,,,cLG_PDV,,, )
147 oTGetPDV:SetCSS( POSCSS (GetClassName(oTGetPDV), CSS_GET_NORMAL ))
148
149
150 //----- Serie do PDV
151 cLG_SERIE := PADR(STFGetStation("SERIE"),TamSx3("LG_SERIE")[1])
152 oSaySerie:= TSay():New(05,150,{||STR0008},oPanel,,,,,T,,,300,25) //"Série do PDV"
153 oSaySerie:SetCSS( POSCSS (GetClassName(oSaySerie), CSS_LABEL_NORMAL ))
154
155 oTGetSerie := TGet():New( 15,150,{|u| if( PCount() > 0, cLG_SERIE := u, cLG_SERIE ) },oPanel,80,025,"@E " + Replicate("9",TamSx3("LG_SERIE")[1]

```

```

    ),,0,,,F,,,T,,,F,,,F,,,F,,,F,,,cLG_SERIE,,, )
156 oTGetSerie:SetCSS( POSCSS (GetClassName(oTGetSerie), CSS_GET_NORMAL ))
157
158
159 //----- Regime Tributario para NFC-E
160 aCbxRegTrib := { STR0009 ; ; //"1 - Simples Nacional"
161                 STR0010 ; ; //"2 - Simples Nacional - Excesso de sub-limite de receita bruta"
162                 STR0011 } //"3 - Regime Nacional"
163
164 cMV_CODREG := AllTrim(SuperGetMV( "MV_CODREG", .F., ""))
165 IIF(cMV_CODREG $ "123",cMV_CODREG := aCbxRegTrib[Val(cMV_CODREG)],"")
166 oSayRegTrib:= TSay():New(45,10,{||STR0012 },oPanel,,,,,T,,,100,25)//"Regime Tributário"
167 oSayRegTrib:SetCSS( POSCSS (GetClassName(oSayRegTrib), CSS_LABEL_NORMAL ))
168
169 oCbxRegTrib := TComboBox():Create( oPanel, {|u| if( Pcount( )>0, cMV_CODREG := u, cMV_CODREG ) }, 55, 10, ;
170                                aCbxRegTrib, 285, 25,,,,,T,,,,,cMV_CODREG)
171 oCbxRegTrib:SetCSS( POSCSS (GetClassName(oCbxRegTrib), CSS_COMBOBOX))
172
173
174 //----- Token SEFAZ
175 cMV_NFCEIDT := PADR(SuperGetMV( "MV_NFCEIDT", .F., ""),100)
176 oSayToken:= TSay():New(85,10,{||STR0013},oPanel,,,,,T,,,100,25) //"Token SEFAZ"
177 oSayToken:SetCSS( POSCSS (GetClassName(oSayToken), CSS_LABEL_NORMAL ))
178
179 oTGetToken := TGet():New( 95,10,{|u| if( PCount() > 0, cMV_NFCEIDT := u, cMV_NFCEIDT ) },oPanel,130,025,,,0,,,F,,,T,,,F,,,F,,,F,,,F,,,cMV_NFCEIDT,,, )
180 oTGetToken:SetCSS( POSCSS (GetClassName(oTGetToken), CSS_GET_NORMAL ))
181
182
183 //----- Codigo da empresa cadastrado no daruma Migrate
184 cMV_LJEMPCK := PADR(SuperGetMV( "MV_LJEMPCK", .F., ""),100)
185 oSayToken:= TSay():New(85,150,{||STR0014},oPanel,,,,,T,,,150,25) //"Código Empresa Daruma Migrate"
186 oSayToken:SetCSS( POSCSS (GetClassName(oSayToken), CSS_LABEL_NORMAL ))
187
188 oTGetToken := TGet():New( 95,150,{|u| if( PCount() > 0, cMV_LJEMPCK := u, cMV_LJEMPCK ) },oPanel,130,025,,,0,,,F,,,T,,,F,,,F,,,F,,,F,,,cMV_LJEMPCK,,, )
189 oTGetToken:SetCSS( POSCSS (GetClassName(oTGetToken), CSS_GET_NORMAL ))
190
191
192 //----- Codigo da Filial da integracao
193
194 cMV_LJFILIN := PADR(SuperGetMV( "MV_LJFILIN", .F., ""),100)
195 oSayFilServer:= TSay():New(125,10,{||STR0015},oPanel,,,,,T,,,150,25) //"Filial Servidor"
196 oSayFilServer:SetCSS( POSCSS (GetClassName(oSayFilServer), CSS_LABEL_NORMAL ))
197
198 oTGetFilServer := TGet():New( 135,10,{|u| if( PCount() > 0, cMV_LJFILIN := u, cMV_LJFILIN ) },oPanel,130,025,,,0,,,F,,,T,,,F,,,F,,,F,,,F,,,cMV_LJFILIN,,, )
199 oTGetFilServer:SetCSS( POSCSS (GetClassName(oTGetFilServer), CSS_GET_NORMAL ))
200
201
202 //----- Endereco Ws do servidor
203 cLG_WSSRV := AllTrim(STFGetStation("WSSRV"))
204
205 If !Empty(cLG_WSSRV)
206

```

```

207 While "/WSPDV" $ UPPER(cLG_WSSRV)
208   cLG_WSSRV := SUBSTR(cLG_WSSRV, 1, Len(cLG_WSSRV) - 6 )
209 End
210
211 EndIf
212
213 cLG_WSSRV := PADR( cLG_WSSRV ,TamSx3("LG_WSSRV")[1])
214
215 oSayEndWs:= TSay():New(165,10,{||STR0016},oPanel,,,,,T,,,300,25) //"End. e Porta WebService Servidor"
216 oSayEndWs:SetCSS( POSCSS (GetClassName(oSayEndWs), CSS_LABEL_NORMAL ))
217
218 oSayExEndWs:= TSay():New(185,145,{||"ex: 172.16.70.110:190"},oPanel,,,,,T,,,300,25) //"ex: 172.16.70.110:190"
219 oSayExEndWs:SetCSS( POSCSS (GetClassName(oSayExEndWs), CSS_LABEL_FOCAL ))
220
221 oTGetEndWs := TGet():New( 175,10,{||u| if( PCount() > 0, cLG_WSSRV := u, cLG_WSSRV ) } ,oPanel,130,25,"@!",,0,,,F,,,T,,,F,,,F,,,F,,,F,,,F,,,cLG_WSSRV,,, )
222 oTGetEndWs:SetCSS( POSCSS (GetClassName(oTGetEndWs), CSS_GET_NORMAL ))
223
224
225 Return Nil
226
227
228 //-----
229 /*{Protheus.doc} valida_pg1
230 Cria Validacao do wizard de Configurações para mobile pg1
231
232 @param
233 @return Nil
234 @obs
235 @sample
236 */
237 //-----
238 Static Function valida_pg1()
239
240 Local lRet := .T. //Retorno da validacao
241
242 IIf(lRet , lRet := !Empty(cLG_PDV) ,)
243 IIf(lRet , lRet := !Empty(cLG_SERIE) ,)
244 IIf(lRet , lRet := !Empty(cMV_CODREG) ,)
245 IIf(lRet , lRet := !Empty(cMV_NFCEIDT) ,)
246 IIf(lRet , lRet := !Empty(cMV_LJEMPCK) ,)
247 IIf(lRet , lRet := !Empty(cLG_WSSRV) ,)
248
249 If !lRet
250   MsgWzValid()//Mensagem validacao
251 EndIf
252
253 Return lRet
254
255
256 //-----
257 /*{Protheus.doc} cria_pg2
258 Cria wizard de Configurações para mobile pg2

```

```

259
260 @param oPanel - Panel de Fundo
261 @return Nil
262 @obs
263 @sample
264 /**
265 //-----
266 Static Function cria_pg2(oPanel)
267
268 Local nPnlHeight      := 40 //Altura panel de botoes
269 Local nPlnWidth       := 40 //Largura panel de botoes
270 Local nBtnPosHoriz    := 0 //Posicao horizontal dos Botoes
271 Local aItems          := {} //Impressoras disponiveis
272 Local oPanelBtn       := TPanel():New(0,0,"",oPanel,,,,,nPlnWidth,nPnlHeight)//Panel de botoes
273 Local oBtnBlu        := Nil //Botao Bluetooth
274 Local oBtnWifi       := Nil //Botao Wifi
275 Local oSayCab        := Nil //Say de Inforcao de cabecalho
276 Local oSayIPName     := Nil //Say de IP(Wifi) | ou nome da impressora(Bluetooth)
277 Local oSayPort       := Nil //Say de porta da impressora
278 Local oGetIPName     := Nil //Get de IP(Wifi) | ou nome da impressora(Bluetooth)
279 Local oGetPort       := Nil //Get de porta da impressora
280 Local nPosImp        := 0 //Posicao da impressora
281
282 Default oPanel := Nil // Panel Bkg
283
284 aItems          := {'DARUMA DR700 V02.10.01','DATECS DPP-350 BT'}
285
286 //valida se impressora Bluetooth ou wifi
287 cLG_PORTIF := STFGetStation("PORTIF") //Porta da impressora | Se Bluetooth sera gravado 'BLU'
288 If UPPER(AllTrim(cLG_PORTIF)) == "BLU" //Impressora Bluetooth
289     lImpBlu := .T.
290     lImpWifi := .F.
291 EndIf
292
293 oPanelBtn:SetCss((POSCSS (GetClassName(oPanelBtn), CSS_PANEL_WIZARD, .T. )))
294 oPanelBtn:Align := CONTROL_ALIGN_TOP
295
296 oSayCab:= TSay():New(017,010,{||STR0017},oPanelBtn,,,,,.T.,,200,25)//"CONECTAR IMPRESSORA"
297 oSayCab:SetCSS( POSCSS (GetClassName(oSayCab), CSS_LABEL_HEADER ))
298
299
300 //----- Botao Bluetooth
301 nBtnPosHoriz := ((oPanel:nClientWidth/2) - 200 )//Calcula posicao horizontal
302 oBtnBlu := TButton():New( 010, nBtnPosHoriz, STR0018,oPanelBtn,; //"Bluetooth"
303     {||oBtnBlu:SetCSS( POSCSS (GetClassName(oBtnBlu), CSS_BTN_ATIVO )) ,;
304     oBtnWifi:SetCSS( POSCSS (GetClassName(oBtnWifi), CSS_BTN_NORMAL )) ,;
305     lImpWifi := .F. , lImpBlu := .T. ,;
306     cLG_TSCSRV := PADR("",TamSx3("LG_TSCSRV")[1]) ,;
307     cLG_TSCPORT := PADR("",TamSx3("LG_TSCPORT")[1]) },;
308     LARGBTN,ALTURABTN,,,F.,.T.,.F.,,F.,,F. )
309 oBtnBlu:SetCSS( POSCSS (GetClassName(oBtnBlu), IIF(lImpBlu,CSS_BTN_ATIVO,CSS_BTN_NORMAL) ))
310

```

```

311
312 //----- Botao Wifi
313 nBtnPosHoriz := ((oPanel:nClientWidth/2) - 100 )//Calcula posicao horizontal
314 oBtnWifi := TButton():New( 010, nBtnPosHoriz, STR0019 ,oPanelBtn, ; //"Rede Sem Fio (Wi-Fi)"
315         {||oBtnBlu:SetCSS( POSCSS (GetClassName(oBtnBlu), CSS_BTN_NORMAL ))      ;;
316         oBtnWifi:SetCSS( POSCSS (GetClassName(oBtnWifi), CSS_BTN_ATIVO ))        ;;
317         lImpWifi := .T. , lImpBlu := .F.                                         ;;
318         cLG_TSCSRV := PADR("",TamSx3("LG_TSCSRV"))[1])                          ;;
319         cLG_TSCPORT := PADR("",TamSx3("LG_TSCPORT"))[1])                        },,;
320         LARGBTN,ALTURABTN,,,F.,.T.,.F.,,.F.,,.F. )
321 oBtnWifi:SetCSS( POSCSS (GetClassName(oBtnWifi), IIF(lImpWifi,CSS_BTN_ATIVO,CSS_BTN_NORMAL) ))
322
323
324 //----- ListBox das Impressoras
325 oListImp := TListBox():Create(oPanel,nPnlHeight+1,0,,aItems,oPanel:nClientWidth,100,,,,.T.)
326 oListImp:SetCSS( POSCSS (GetClassName(oListImp),CSS_LISTBOX ))
327
328 cLG_IMPFIISC := AllTrim(STFGetStation("IMPFIISC"))
329 cImpAnterior := cLG_IMPFIISC //Armazena impressora configurada
330 nPosImp := ASCAN(aItems, cLG_IMPFIISC )
331
332 //Posiciona na impressora cadastrada
333 If nPosImp > 0
334     oListImp:Select(nPosImp)
335 Else
336     oListImp:GoTop()//Posiciona no item 1
337 EndIf
338
339
340 //----- IP ou Nome da impressora
341 oSayIPName:= TSay():New(nPnlHeight+115,010,{||IIF(lImpWifi,STR0020,STR0021)},oPanel,,,,.T.,,300,25) //"Endereço IP da impressora na rede:### "Nome da Impressora
    Bluetooth. Case-sensitive"
342 oSayIPName:SetCSS( POSCSS (GetClassName(oSayIPName), CSS_LABEL_NORMAL ))
343
344 cLG_TSCSRV := PADR(STFGetStation("TSCSRV",,.F.),TamSx3("LG_TSCSRV"))[1])
345 oGetIPName := TGet():New( nPnlHeight+125,10,{|u| if( PCount() > 0, cLG_TSCSRV := u, cLG_TSCSRV ) } ,oPanel,120,25,"",,0,,,F.,.T.,.F.,.F.,.F.,.F.,.F.,cLG_TSCSRV,,, )
346 oGetIPName:SetCSS( POSCSS (GetClassName(oGetIPName), CSS_GET_NORMAL ))
347
348
349 //----- Porta da impressora
350 oSayPort:= TSay():New(nPnlHeight+115,150,{||IIF(lImpWifi,STR0022,"")},oPanel,,,,.T.,,300,25) //"Porta da impressora:"
351 oSayPort:SetCSS( POSCSS (GetClassName(oSayPort), CSS_LABEL_NORMAL ))
352
353 cLG_TSCPORT := PADR(IIF(lImpBlu,"",STFGetStation("TSCPORT")),TamSx3("LG_TSCPORT"))[1])
354 oGetPort := TGet():New( nPnlHeight+125,150,{|u| if( PCount() > 0, cLG_TSCPORT := u, cLG_TSCPORT ) } ,oPanel,40,25,"@!",,0,,,F.,.T.,.F.,.F.,.F.,.F.,.F.,cLG_TSCPORT,,, )
355 oGetPort:SetCSS( POSCSS (GetClassName(oGetPort), CSS_GET_NORMAL ))
356 oGetPort:bWhen := {|| lImpWifi }
357
358
359 Return Nil
360
361 //-----

```



```

362 /*{Protheus.doc} valida_pg2
363 Cria Validacao do wizard de Configurações para mobile pg2
364
365 @param
366 @return lRet - Retorno da validacao
367 @obs
368 @sample
369 */
370 //-----
371 Static Function valida_pg2()
372
373 Local lRet := .T.      //Retorno da validacao
374
375 IIf(lRet          , lRet := oListImp:GetPos() > 0      ,)//impressora selecionada
376 IIf(lRet          , lRet := !Empty(cLG_TSCSRV)         ,)//IP ou Nome informado
377 IIf(lRet .AND. lImpWifi , lRet := !Empty(cLG_TSCPORT)   ,)//Porta informada apenas Wifi Rede
378
379 If !lRet
380     MsgWzValid()//Mensagem validacao
381 EndIf
382
383 Return lRet
384
385
386 //-----
387 /*{Protheus.doc} cria_pn3
388 Cria wizard de Configurações para mobile pn3
389
390 @param oPanel - Panel de Fundo
391 @return Nil
392 @obs
393 @sample
394 */
395 //-----
396 Static Function cria_pn3(oPanel)
397
398 Local nPnlHeight      := 40 //Altura panel
399 Local nPlnWidth       := 40 //Largura panel
400 Local aItems          := {} //TEFs disponiveis
401 Local oPanelSup       := TPanel():New(0,0,"",oPanel,,,,,nPlnWidth,nPnlHeight)//Panel Superior
402 Local oSayCab         := Nil //Say de Inforcao de cabecalho
403 Local oSayAtvTEF      := Nil //Say de Ativa TEF
404 Local nBtnAtiva       := Nil //Botao Ativa/Desativa TEF
405 Local nPosTEF         := 0  //Posicao do TEF
406
407 Default oPanel := Nil      // Panel Bkg
408
409
410 oPanelSup:SetCss((POSCSS (GetClassName(oPanelSup), CSS_PANEL_WIZARD, .T. )))
411 oPanelSup:Align := CONTROL_ALIGN_TOP
412
413 oSayCab:= TSay():New(017,010,{||STR0023},oPanelSup,,,,,.T.,,200,25) //"ATIVA TRANSAÇÕES TEF"

```

```

414 oSayCab:SetCSS( POSCSS (GetClassName(oSayCab), CSS_LABEL_HEADER ))
415
416 //----- ListBox dos TEFs
417 aItems      := {"PAYGO"}
418 oListTEF := TListBox():Create(oPanel,nPnlHeight+1,0,,aItems,oPanel:nClientWidth,100,,,,.T.)
419 oListTEF:SetCSS( POSCSS (GetClassName(oListTEF),CSS_LISTBOX ))
420
421 cMDG_CARPAY := AllTrim(STFGetStation("CARPAY",.T.))
422 nPosTEF := IIF(cMDG_CARPAY == "1",1,0)
423
424 //Posiciona no TEF cadastrado
425 If nPosTEF > 0
426     oListTEF:Select(nPosTEF)
427 Else
428     oListTEF:GoTop()//Posiciona no item 1
429 EndIf
430
431
432 //----- Ativa TEF
433 cMDG_TEFATV := AllTrim(STFGetStation("TEFATV",.T.))
434 lTefAtivo := cMDG_TEFATV == "1"
435 nBtnAtiva := TButton():New( nPnlHeight+125,10, STR0024,oPanel, ; //"Ativa / Desativa TEF"
436     {||,;
437     IIF( lTefAtivo , lTefAtivo := .F. , lTefAtivo := .T. ) ,;
438     nBtnAtiva:SetCSS( POSCSS (GetClassName(nBtnAtiva), IIF(lTefAtivo,CSS_BTN_ATIVO,CSS_BTN_NORMAL) )) ,;
439     cMDG_TEFATV := IIF( lTefAtivo , "1" , "2" ),;
440     LARGBTN,ALTURABTN,,,F.,.T.,.F.,.F.,.F.,.F. )
441 nBtnAtiva:SetCSS( POSCSS (GetClassName(nBtnAtiva), IIF(lTefAtivo,CSS_BTN_ATIVO,CSS_BTN_NORMAL) ))
442
443 oSayAtvTEF:= TSay():New(nPnlHeight+135,100,{|| IIF(lTefAtivo,STR0025,STR0026)},oPanel,,,,.T.,,100,25) //"TEF Ativado!" ### "TEF Desativado!"
444 oSayAtvTEF:SetCSS( POSCSS (GetClassName(oSayAtvTEF), CSS_LABEL_NORMAL ))
445
446
447 Return Nil
448
449
450 //-----
451 /*/{Protheus.doc} valida_pg3
452 Cria Validacao do wizard de Configurações para mobile pg3
453
454 @param
455 @return lRet - Retorno da validacao
456 @obs
457 @sample
458 */
459 //-----
460 Static Function valida_pg3()
461
462 Local lRet := .T.      //Retorno da validacao // essa pag nao precisa validar
463
464 Return lRet
465

```

```

466
467 //-----
468 /*{Protheus.doc} cria_pn4
469 Cria wizard de Configurações para mobile pn4
470
471 @param oPanel - Panel de Fundo
472 @return Nil
473 @obs
474 @sample
475 */
476 //-----
477 Static Function cria_pn4(oPanel)
478
479 Local nPnlHeight      := 40 //Altura panel
480 Local nPlnWidth       := 40 //Largura panel
481 Local oPanelSup       := TPanel():New(0,0,"",oPanel,,,,,nPlnWidth,nPnlHeight)//Panel Superior
482 Local oSayCab         := Nil //Say de Inforcao de cabecalho
483 Local oSayCodBar      := Nil //Say do código de barras
484 Local oSayBCodBar     := Nil //Say do botao código de barras
485 Local oBtnCam         := Nil //Botao da Camera
486
487 Default oPanel := Nil
488
489 oPanelSup:SetCss((POSCSS (GetClassName(oPanelSup), CSS_PANEL_WIZARD, .T. )))
490 oPanelSup:Align := CONTROL_ALIGN_TOP
491
492 oSayCab:= TSay():New(017,010,{||STR0027},oPanelSup,,,,,T,,,200,25) //"LEITOR VIA CÂMERA"
493 oSayCab:SetCSS( POSCSS (GetClassName(oSayCab), CSS_LABEL_HEADER ))
494
495
496 oSayCodBar:= TSay():New(nPnlHeight+5,10,{||STR0028 + chr(13)+chr(10) + ; //"Você pode escanear os códigos de barras de seus produtos"
497                               STR0029 },oPanel,,,,,T,,,300,50) //"usando a câmera do seu dispositivo"
498 oSayCodBar:SetCSS( POSCSS (GetClassName(oSayCodBar), CSS_LABEL_HEADER ))
499
500 //"Botao teste da camera"
501 oBtnCam := TButton():New( nPnlHeight+35,10,"",oPanel,{|| TesteCam() },40,ALTURABTN,,,,T,,,,)
502 oBtnCam:SetCSS( POSCSS (GetClassName(oBtnCam), CSS_BTN_BARCODE ))
503
504 oSayBCodBar:= TSay():New(nPnlHeight+65,10,{||STR0030 },oPanel,,,,,T,,,300,50) //"Pressione o Botão e teste"
505 oSayBCodBar:SetCSS( POSCSS (GetClassName(oSayBCodBar), CSS_LABEL_NORMAL ))
506
507
508 Return Nil
509
510
511 //-----
512 /*{Protheus.doc} valida_pg4
513 Cria Validacao do wizard de Configurações para mobile pg4
514
515 @param
516 @return lRet - Retorno da validacao
517 @obs

```

```

518 @sample
519 /*/
520 //-----
521 Static Function valida_pg4()
522 Local lRet := .F. //Retorno
523
524 lRet := .T.
525
526 Return lRet
527
528
529 //-----
530 /*/{Protheus.doc} TesteCam
531 Teste de camera dso dispositivos mobiles
532
533 @param
534 @return cRet - retorna o codigo lido
535 @obs
536 @sample
537 /*/
538 //-----
539 Static Function TesteCam()
540
541 Local oMbl          := Nil      //Objeto Barcode
542 Local cBarType       := ""      //Define os Tipos de Barcode lidos
543 Local aBarResult     := {}      //Array de resultados
544 Local cRet           := ""      //retorna o codigo lido
545
546 oMbl:= TMobile():New()
547
548 aBarResult:= oMbl:BarCode(cBarType)
549 If aBarResult[2] = ""
550     cRet := ""
551 Else
552     cRet := aBarResult[2]
553 EndIf
554
555 Return cRet
556
557 //-----
558 /*/{Protheus.doc} cria_pn5
559 Cria wizard de Configurações para mobile pn5
560
561 @param oPanel - Panel de Fundo
562 @return Nil
563 @obs
564 @sample
565 /*/
566 //-----
567 Static Function cria_pn5(oPanel)
568
569 Local nPnlHeight      := 0      //Altura panel

```

```

570 Local nPlnWidth      := 0      //Largura panel
571 Local oPanelL        := Nil    //Panel Esquerdo
572 Local oPanelR        := Nil    //Panel direito
573 Local oSayCab        := Nil    //Say de Inforcao de cabecalho
574 Local oSayFormato    := Nil    //Say Formato das imagens
575 Local oImgExemplo    := Nil    //Imagem de exemplo da direita
576 Local oSayInfo1      := Nil    //Say de Inforcao da imagem tela Esquerda
577 Local oSayInfo2      := Nil    //Say de Inforcao da imagem tela Superior Direita
578 Local oSayInfo3      := Nil    //Say de Inforcao da imagem tela Inferior Direita
579 Local oBtnAltImg     := Nil    //Botao alterar imagem
580 Local bTrocaImg      := {}     //Bloco de codigo para troca da imagem
581 Local cImgName       := PADR("LOGOPOS",20) //Nome da imagem para troca
582 Local lIncluiuImg    := .F.    //controla se conseguiu incluir a imagem
583
584 Default oPanel := Nil
585
586 nPnlHeight := (oPanel:nClientHeight/2) /2
587 nPlnWidth  := (oPanel:nClientWidth/2) /2
588
589
590 //----- Lado esquerdo -----
591 //-----
592 //-----
593 oPanelL := TPanel():New(0,0,"",oPanel,,,,,nPlnWidth,nPnlHeight)//Panel Esquerdo
594 oPanelL:SetCss((POSCSS (GetClassName(oPanelL), CSS_PANEL_WIZARD, .T. )))
595 oPanelL:Align := CONTROL_ALIGN_LEFT
596
597 oSayCab:= TSay():New(010,010,{||STR0031},oPanelL,,,,,T,,,200,25) //"MARCA DA EMPRESA"
598 oSayCab:SetCSS( POSCSS (GetClassName(oSayCab), CSS_LABEL_HEADER ))
599
600 //----Imagem do Repositorio
601 @ 030,010 REPOSITORY oImg OF oPanelL SIZE 130,65 PIXEL
602 If !oImg:LoadBMP(cImgName)
603     //Verifica se nao tem o arquivo sem os espacos
604     oImg:LoadBMP(AllTrim(cImgName))
605 EndIf
606 oImg:lStretch := .T.
607
608 //---- Says
609 oSayFormato:= TSay():New(100,010,{||STR0032 + chr(13)+chr(10) + "LOGOPOS.JPG"},oPanelL,,,,,T,,,200,25) //"Formato:"
610 oSayFormato:SetCSS( POSCSS (GetClassName(oSayFormato), CSS_LABEL_FOCAL ))
611
612 oSayInfo1:= TSay():New(130,010,{||"* " + STR0033},oPanelL,,,,,T,,,200,25) //"Utilizar imagem com fundo branco"
613 oSayInfo1:SetCSS( POSCSS (GetClassName(oSayInfo1), CSS_LABEL_FOCAL ))
614
615 //"Botao Trocar Imagem"
616 bTrocaImg := {} TrocaImg(oImg ,cImgName ) }
617
618 oBtnAltImg := TButton():New( 150,010,STR0034,oPanel, bTrocaImg ,LARGBTN,ALTURABTN,,,,.T.,,,)//"Trocar imagem" //Refresh na Dialog para alteração da imagem
619 oBtnAltImg:SetCSS( POSCSS (GetClassName(oBtnAltImg), CSS_BTN_NORMAL ))
620
621

```

```

622 //----- Lado Direito -----
623 //-----
624 //-----
625 oPanelR      := TPanel():New(0,0,"",oPanel,,,,,nPlnWidth,nPlnHeight)//Panel direito
626 oPanelR:Align := CONTROL_ALIGN_RIGHT
627
628 oSayInfo2:= TSay():New(010,010,{||STR0035 + chr(13)+chr(10) + STR0036},oPanelR,,,,,T,,,200,25) //"Aplicação da marca da" ### "empresa no aplicativo"
629 oSayInfo2:SetCSS( POSCSS (GetClassName(oSayInfo2), CSS_LABEL_FOCAL ))
630
631
632 //---Imagem do Repositorio
633 @ 030,010 REPOSITORY oImgExemplo OF oPanelR NOBORDER SIZE 130,100 PIXEL
634 oImgExemplo:Load("posmarca")
635 oImgExemplo:lStretch := .T.
636
637 oSayInfo3:= TSay():New(135,010,{|| STR0037 + chr(13)+chr(10) +; //"A marca da empresa irá aparecer na tela"
638                               STR0038 + chr(13)+chr(10) +; //"do aplicaativo, na parte superior esquerda,"
639                               STR0039 },oPanelR,,,,,T,,,200,25) //"conforme imagem acima"
640 oSayInfo3:SetCSS( POSCSS (GetClassName(oSayInfo3), CSS_LABEL_FOCAL ))
641
642
643 Return Nil
644
645
646 //-----
647 /*/{Protheus.doc} TrocaImg
648 Troca imagem de logo do PDV
649
650 @param oImage - Objeto de imagem
651 @param cImgName - Nome da imagem
652 @return Nil
653 @obs
654 @sample
655 */
656 //-----
657 Static Function TrocaImg(oImage, cImgName)
658
659 Local nMaskDef      := 2                // Mascara JPEG como default
660 Local cDirAtu       := ""              // Diretoria atual
661 Local cFile         := ""              // path + arquivo
662 Local lInsertImage  := .T.             // Foi concluida a insercao da imagem?
663 Local lRet          := .T.             // Retorno da funcao
664 Local cDrive        := Space(255)      // Drive onde esta o arquivo
665 Local cDir          := Space(255)      // Diretorio onde esta o arquivo
666 Local cMask         := STR0032 + " JPEG" + "(*.JPG) |*.jpg|" // "Formato" "JPEG" + "(*.JPG) |*.jpg|"
667
668 Default oImage := Nil
669 Default cImgName := ""
670
671 cFile := cGetFile( cMask, STR0040 + " LOGOP0S.JPG...", @nMaskDef, GetSrvProfString("RootPath",""), .F. ) // "Selecione o arquivo"
672
673 If Empty(cFile) .OR. Empty(cImgName)

```

```

674     lRet := .F.
675 Else
676
677     If Upper(RIGHT(AllTrim(cFile),11) ) == "LOGOP0S.JPG"
678         cFile := UPPER(cFile)
679
680         If ( File( cFile) )
681             oImage:DeleteBmp(cImgName)
682             cFile := oImage:InsertBmp(cFile,cImgName,.T.)
683             oImage:LoadBmp(cImgName)
684             oImage:Refresh()
685         EndIf
686     Else
687         FWAlertError( STR0041 + " LOGOP0S.JPG") //"Imagem Inválida. A imagem deve estar no formato .JPG e com o nome"
688     EndIf
689
690 EndIf
691
692 Return lRet
693
694
695 //-----
696 /*{Protheus.doc} MsgWzValid
697 Mensagem validacao padrao wizard
698
699 @param
700 @return Nil
701 @obs
702 @sample
703 */
704 //-----
705 Static Function MsgWzValid()
706     FWAlertError(STR0042)//"Campos obrigatórios não informados!"
707 Return Nil
708
709
710 //-----
711 /*{Protheus.doc} FimWizard
712 Finaliza wizard de Configurações.
713
714 @param
715 @return Nil
716 @obs
717 @sample
718 */
719 //-----
720 Static Function FimWizard()
721
722 Local lRet := .T.    //Retorno
723
724
725 aSLG      := {}    //Array que guarda os dados do cadastro de estacao

```

```

726 aMDG      := {}    //Array que guarda os dados do cadastro de estacao TEF
727
728
729 //----- Informacoes da pag 1
730
731 //Parametros
732 PutMV("MV_CODREG"      , LEFT(AllTrim(cMV_CODREG),1)    )//Regime tributario para NFC-E
733 PutMV("MV_NFCEIDT"     , AllTrim(cMV_NFCEIDT) )//Token do sefaz
734 PutMV("MV_LJEMPCK"     , AllTrim(cMV_LJEMPCK) )//Codigo da empresa cadastrado no daruma Migrate
735 PutMV("MV_LJFILIN"     , AllTrim(cMV_LJFILIN) )//Codigo da Filial da integracao
736
737
738 //Cadastro de Estacao
739 Add( aSLG , { "PDV"      , cLG_PDV      } )//Numero do PDV
740 Add( aSLG , { "SERIE"    , cLG_SERIE    } )
741 cLG_WSSRV := AllTrim(cLG_WSSRV) + "/wspdv" //Exemplo de como fica "127.0.0.1:190/wspdv"
742 Add( aSLG , { "WSSRV"   , cLG_WSSRV    } )
743
744
745 //----- Informacoes da pag 2
746 cLG_IMPFIISC := oListImp:GetSelText() //Modelo da Impressora
747 cLG_TSCPORT  := IIf(lImpWifi,cLG_TSCPORT,"500") //porta para impressora de Rede, Wifi . Se Bluetooth sera gravado o timeout 500 default
748 cLG_PORTIF   := IIf(lImpBlu,"BLU",cLG_PORTIF) //Porta da impressora | Se Bluetooth sera gravado 'BLU'
749
750 Add( aSLG , { "IMPFIISC" , cLG_IMPFIISC } )
751 Add( aSLG , { "TSCSRV"   , cLG_TSCSRV   } )
752 Add( aSLG , { "TSCPORT"  , cLG_TSCPORT  } )
753 Add( aSLG , { "PORTIF"   , cLG_PORTIF   } )
754
755 lRet := STFSetStat( aSLG )
756
757
758 //----- Informacoes da pag 3
759 Add( aMDG , { "TEFATV"   , cMDG_TEFATV   } )//Ativa o TEF
760 Add( aMDG , { "CARPAY"   , cMDG_CARPAY   } )//Usa TEF PAYGO
761 lRet := STFSetTefStat( aMDG )
762
763 FWAlertSuccess(STR0043)//"Wizard Configurado com sucesso!"
764
765 // funções não compiladas - retirado o bloc de codigo
766
767 Return .T.

```


Framework/Facilitator/STFSTCFG.CH

41 linhas após remoção de metadados de autoria/data

```

1 #Define _FATHER_NODE 1
2 #Define _CODE_NODE 2
3 #Define _DESCRIPTION_NODE 3
4 #Define _ORDER_NODE 4
5 #Define _ENVIRONMENT_NODE 5
6 #Define _VALID_NODE 6
7 #Define _EDITED_NODE 7
8 #Define _VALID_FUNCTION 8
9 #Define _IS_FATHER 9
10 #Define _PREDECESSOR_NODE 10
11 #Define _RECORD_TABLE_ONCE 11 //Grava todos os campos da tabela uma vez
12 #Define _INSERT_TABLE 12 //Insero o registro de uma tabela, caso o registro ainda não exista
13 #Define _COMPS_NODE 15 ///COMPONENTES DO NÓ - NO CASO DE SER UM PAI
14 #Define _POS_RECORDED 13 //Função a ser executada após a gravação do nó
15 #Define _WRITE_FUNCTION 14 //Função de gravação
16 #Define _COMP_NODE 01
17 #Define _COMP_ID 02
18 #Define _COMP_ENVIRONMENT 03
19 #Define _COMP_TYPE 04
20 #Define _COMP_NAME 05
21 #Define _COMP_TITLE 06
22 #Define _COMP_F3 07
23 #Define _COMP_DEFAULT 08
24 #Define _COMP_TABLE_FILE 09 //NOME DA TABELA OU ARQUIVIO CONFIG
25 #Define _COMP_INDEX 10
26 #Define _COMP_KEY 11 //SE FOI CAMPO DÁ UM EVAL
27 #Define _COMP_INITYPE 12
28 #Define _COMP_VALID 13 //EXPRESSO DE VALIDAÇÃO
29 #Define _COMP_LEN 14 // Tamanho do componente -
30 #Define _COMP_LIST 15 // LISTA DE VALORES //
31 #Define _COMP_FIELD_TYPE 16// - TIPO DE CAMPO // Default C
32 #Define _COMP_CTRL_TYPE 17 // - Tipo de CONTROLE // Default MsGet ("G") // M - Memo //L List
33 #Define _COMP_OLDVALUE 18 // - Valor antigo do controle no caso de rollback dos dados
34 #Define _COMP_POSICA 19 //Picture do campo
35 #Define _COMP_OBJECT 20 // - objeto do controle
36 #Define _TOTAL_NODES Len(aData) //TOTAL DOS NÓS
37 #Define _TOTAL_CAMPOS 18 //TOTAL DE CAMPOS
38 #Define _CAMPO "1"
39 #Define _PARAMETRO "2"
40 #Define _ARQUIVO_INI "3"
41 #Define _ROTINA "4"

```

Framework/Facilitator/STFSTTaxCalc.PRW

179 linhas após remoção de metadados de autoria/data

```

1 #INCLUDE "PROTHEUS.CH"
2 #INCLUDE "STFStTaxCalc.CH"
3 //-----
4 /*{Protheus.doc} STFStTaxCalc
5 Calcula os impostos cadastrados para a tabela de ECF
6 @return lRet Status de execução da rotina
7 @obs
8 @sample
9 */
10 //-----
11
12 Function STFStTaxCalc()
13     Local aArea := GetArea()
14     Local cMsg := ""
15     Local lRet := .F.
16
17     MsgRun(STR0001,STR0002,{||lRet := STFStTaxCl(@cMsg)})    //"Calculando os impostos para o cliente tipo padrão"#"Aguarde!"
18
19     If lRet
20         MsgInfo(cMsg)
21     Else
22         MsgAlert(cMsg)
23     EndIf
24
25     RestArea(aArea)
26
27
28
29 Return lRet
30 //-----
31 /*{Protheus.doc} STFStTaxCalc
32 Calcula os impostos cadastrados para a tabela de ECF
33 @param cMsg Mensagem
34 @return lRet Status de execução da rotina
35 @obs
36 @sample
37 */
38 //-----
39 Static Function STFStTaxCl(cMsg)
40     Local aArea := GetArea()    //WorkArea Anterior
41     Local aAreaSF4 := SF4->(GetArea())    //WorkArea SF4
42     Local aAreaSB1 := SB1->(GetArea())    //WorkArea SB1
43     Local aAreaSA1 := SA1->(GetArea())    //WorkArea SA1
44     Local aAreaSB0 := SB0->(GetArea())    //WorkArea SB0
45     Local aAreaMFD := {}    //WorkArea MFD
46     Local cCliPad := PadR( SuperGetMV("MV_CLIPAD"), TamSX3("A1_COD")[1])    //Cliente Padrão
47     Local cLojaPad := PadR( SuperGetMV("MV_LOJAPAD"), TamSx3("A1_LOJA")[1])    //Loja Padrao
48     Local cItemTES := ""    //Tes do Produto
49     Local cTpSolCf := SuperGetMv("MV_TPSOLCF")    //Parametro para validacao do tipo de cliente para o calculo do solidario
50     Local cFilSB1 := SB1->(xFilial())    //Filial SB1
51     Local cFilSA1 := SA1->(xFilial())    //Filial SA1
52     Local cFilSB0 := SB0->(xFilial())

```

```

53 Local cFilSF4 := SF4->(xFilial())
54 Local cFilMFD := ""
55 Local lMVDescISS := SuperGetMv("MV_DESCISS",,.F.)
56 Local nMVALiqISS := SuperGetMv("MV_ALIQISS")
57 Local cTipoImp := ""
58 Local nAliq := 0
59 Local nMFDA11 := 0
60 Local nMFDA12 := 0
61 Local lRet := .F. //Retorno da Rotina
62 Local lFunc := FindFunction("STBTaxTES")
63 Local nALIQUISS := 0
64 Local nPICMRET := 0
65 Local cTS := ""
66
67 Default cMsg := ""
68
69 cMsg := STR0003 //"Registros Gerados com sucesso"
70
71 If nModulo == 12 .AND. AliasIndic("MFD") .AND. lFunc
72   aAreaMFD := MFD->(GetArea())
73   cFilMFD := MFD->(xFilial())
74   cTabela := "SB1"
75
76   SA1->(DbSetOrder(1)) //A1_FILIAL+A1_COD+A1_LOJA
77   SB1->(DbSetOrder(1))//B1_FILIAL+B1_COD
78   SB0->(DbSetOrder(1)) //B0_FILIAL+B0_COD
79   SF4->(DbSetOrder(1))//F4_FILIAL+F4_CODIGO
80   MFD->(DbSetOrder(1)) //MFD_FILIAL + MFD_IMPOST + STR(MFD_ALIQ,5,2)
81 //ÚÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁ
82 //³Posiciona o SA1
83 //ÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁ
84 If SA1->(DbSeek( cFilSA1 + cCliPad + cLojaPad ) )
85
86 //ÚÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁ
87 //³Posiciona o SB1
88 //ÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁ
89 SB1->(DbSeek( cFilSB1 ) )
90
91 nMFDA11 := MFD->(TamSX3("MFD_ALIQ"))[1]
92 nMFDA12 := MFD->(TamSX3("MFD_ALIQ"))[2]
93
94 Do While SB1->(!Eof() .AND. B1_FILIAL == cFilSB1)
95
96   nALIQUISS := RetFldProd(SB1->B1_COD,"B1_ALIQISS")
97   nPICMRET := RetFldProd(SB1->B1_COD,"B1_PICMRET")
98   cTS := RetFldProd(SB1->B1_COD,"B1_TS")
99
100   cItemTES := STBTaxTES( 2 , "01" , cCliPad , cLojaPad ,
101                         "C" , SB1->B1_COD , Nil , If(Empty(cTS), SB1->B1_TS, cTS) )
102
103 //ÚÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁÁ
104 //³Posiciona o SB0

```

```

105      //AAAAAAAAAAAAAAAAAAAA
106      SB0->(DbSeek(cFilSB0 +SB1->B1_COD)) //B0_FILIAL+B0_COD
107
108      //UUUUUUUUUUUUUUUUUUUU
109      //³Posiciona o SF4      ³
110      //AAAAAAAAAAAAAAAAAAAA
111      If SF4->(DbSeek( cFilSF4+cItemTES ) )
112
113          cTipoImp      := "1"
114          nAliq          := 0
115
116          nPICMRET      := Iif(Empty(nPICMRET ), SB1->B1_PICMRET, nPICMRET )
117
118          If SF4->F4_ISS == "S" //.AND. ( ( ( SA1->A1_RECIS == "1" .AND. lMVDescISS ) .OR. ( SA1->A1_RECIS <> "1" ) ) )
119              nALIQUISS := Iif(Empty(nALIQUISS), SB1->B1_ALIQUISS, nALIQUISS)
120              If nALIQUISS > 0
121                  cTipoImp := "2"
122                  nAliq := nALIQUISS
123              Else
124                  cTipoImp := "2"
125                  nAliq := nMVALIQUISS
126              EndIf
127          ElseIf nPICMRET > 0 .AND. SA1->A1_TIPO $ cTpSolCf .AND. SF4->F4_BSICMST <> 100 // Substituicao tributaria (Icms Solidario)
128              nAliq := 0
129          ElseIf SF4->F4_BASEICM > 0 .AND. SF4->F4_BASEICM < 100 // Com redução de Icms na Base de Cálculo do Imposto
130              nAliq := SB0->B0_ALIQRED
131          ElseIf SF4->F4_LFICM == "I" // Isento
132              nAliq := 0
133          ElseIf SF4->F4_LFICM == "N" .OR. SF4->F4_LFICM == "0" // N,,o sujeito a ICMS
134              nAliq := 0
135          Else // Com ICMS
136              nAliq := AliqIcms( "N" ,"S",SA1->A1_TIPO,"I", NIL, .T.)
137          EndIf
138
139      EndIf
140
141      If nAliq > 0 .AND. !MFD->(DbSeek(cFilMFD + cTipoImp + Str(nAliq, nMFDA11, nMFDA12) ) )
142          RecLock("MFD", .T.)
143          MFD->MFD_FILIAL := cFilMFD
144          MFD->MFD_IMPOST := cTipoImp
145          MFD->MFD_ALIQ := nAliq
146          MFD->(MsUnLock())
147      EndIf
148
149
150      SB1->(DbSkip())
151      End
152      lRet := .T.
153  Else
154      cMsg := STR0004 //"Cliente Padrão não localizado"
155  EndIf
156

```

```
157
158     RestArea(aAreaMFD)
159 Else
160     If !AliasInDic("MFD")
161         cMsg := STR0005 //"Para a execução da apuração das alíquotas do ECF, é necessário que a tabela MFD esteja cadastrada"
162     Else
163         If !lFunc
164             cMsg := STR0006 //"Função não compilada no repositório: STBTaxTES"
165         Else
166             cMsg := STR0007 //"Esta rotina deve ser executada somente no Sigaloja"
167         EndIf
168     EndIf
169 EndIf
170 EndIf
171
172
173 RestArea(aAreaSB0)
174 RestArea(aAreaSA1)
175 RestArea(aAreaSB1)
176 RestArea(aAreaSF4)
177 RestArea(aArea)
178
179 Return lRet
```